Notre vaisseau spatial, le Babylon, traverse l'immensité de l'univers à une vitesse prodigieuse, ses moteurs émettant une lueur bleutée alors qu'il fend l'espace-temps. Autour de nous, les étoiles brillent comme des joyaux étincelants dans l'obscurité infinie, tandis que des nébuleuses multicolores tourbillonnent et des astéroïdes dérivent dans l'espace, sans but. Nous survolons des systèmes stellaires entiers en un instant, traversant des trous de ver et des portes spatiales qui nous propulsent instantanément dans des régions éloignées de l'univers. Des formes de vie exotiques nous saluent alors que nous passons devant des planètes habitées par des espèces intelligentes. Des vaisseaux spatiaux étrangers croisent notre route, certains amicaux et d'autres hostiles. Des tempêtes spatiales violentes éclatent autour de nous, avec des éclairs bleus qui illuminent les cieux sombres de l'espace.

Nous sommes les explorateurs de l'inconnu, les pionniers de l'espace, et rien ne peut nous arrêter. Notre vaisseau spatial est un chef-d'œuvre de la technologie avancée, équipé de boucliers énergétiques, de canons à plasma et de systèmes de propulsion sophistiqués qui nous permettent de voyager à des vitesses incroyables. Le Babylon est notre maison, notre refuge dans l'espace infini. Nous avons passé des décennies à bord, parcourant des milliards de milliards de kilomètres et explorant des mondes étranges et merveilleux que la plupart des gens ne verront jamais. Et maintenant, nous nous dirigeons vers la Terre, notre planète natale, avec une impatience croissante.

Alors que nous nous approchons du système solaire, les étoiles autour de nous commencent à s'estomper. Nous pouvons sentir l'excitation monter en nous alors que nous nous rapprochons de notre planète. Le Babylon, est enveloppé d'une aura bleue étincelante alors que nous pénétrons dans la ceinture d'astéroïdes qui entoure le système solaire. Les astéroïdes défilent devant nous, créant une chorégraphie spatiale à la fois fascinante et dangereuse. Nous naviguons avec agilité entre les masses rocheuses et les champs de débris spatiaux, évitant de justesse les collisions qui pourraient endommager le vaisseau. C'est une danse complexe et magnifique, orchestrée par les lois de la physique et la maîtrise de notre équipage expérimenté.

Enfin, nous atteignons la bordure extérieure du système, où se trouvent les planètes géantes comme Jupiter et Saturne. Nous admirons leur majesté et leur beauté, des sphères gigantesques aux nuances de bleu, de rouge et de jaune. Nous nous sentons petits et insignifiants face à ces géants gazeux qui nous rappellent la puissance et la grandeur de l'univers.

Puis, nous nous approchons de la Terre. Nous pouvons la voir comme un petit point bleu au milieu de l'espace, accompagnée par la lueur blanche de la Lune, sa Lune, notre Lune. Nous ressentons une émotion intense alors que nous nous approchons de plus en plus, réalisant que nous allons bientôt poser les pieds sur notre foyer, après ces décennies de voyage dans l'espace. La vue de la planète est à couper le souffle, un mélange de bleu et de vert qui nous rappelle à quel point la Terre est précieuse et fragile. Nous sommes prêts à atterrir, à retrouver notre foyer et à partager notre expérience de l'espace.

Nous approchons de plus en plus de la Terre et nous pouvons maintenant distinguer des détails précis sur sa surface. Les montagnes sont immenses, se dressant comme des géants géologiques contre le ciel bleu, et nous pouvons voir des pics enneigés qui s'étendent à perte de vue. Les canyons sont profonds et étroits, formant des sillons dans la terre qui ressemblent à des cicatrices. Nous pouvons même voir des rivières serpentines qui coulent à travers les vallées et les plaines. Les déserts sont également impressionnants, avec des dunes de sable doré qui se chevauchent et se plient comme des vagues figées. Nous pouvons voir des oasis verdoyantes nichées au creux des collines, offrant un havre de paix et de fraîcheur aux voyageurs fatigués. Les océans sont tout aussi captivants, avec des étendues d'eau bleu profond qui s'étendent à perte de vue. Des vagues immenses se forment et se brisent sur les plages, créant un rythme hypnotique qui accompagne notre descente vers la planète. Nous apercevons des récifs coralliens colorés qui abritent une multitude de créatures marines fascinantes, des poissons exotiques aux raies majestueuses en passant par les requins redoutables. Au-dessus de nous, la lune apparaît comme un monde stérile et aride, couvert de cratères et de rochers. Nous pouvons voir des montagnes accidentées et des plaines rocheuses, et même des formations étranges qui ressemblent à des visages figés dans la pierre. Bien que la lune ne soit pas aussi vivante que la Terre, elle est tout de même fascinante à regarder.

Nous sommes désormais dans l’atmosphère terrestre et notre vaisseau spatial, le Babylon, semble plus majestueux que jamais alors qu'il fend l'air avec une aisance incroyable. Les moteurs rugissent et laissent derrière eux une traînée de flammes alors que nous plongeons dans l'atmosphère. L'aura étincelante qui enveloppe notre vaisseau semble danser et vibrer, créant une impression de puissance et de mouvement. Au fur et à mesure que nous descendons, nous pouvons prendre la mesure de l’immensités des montagnes se déployant sous nos yeux. Les sommets enneigés brillent sous le soleil tandis que les vallées verdoyantes et luxuriantes s'étendent à perte de vue. Les forêts sont également impressionnantes, avec des arbres qui s'étendent comme des géants, formant un couvert dense et luxuriant sur les collines et les vallées. Alors que nous approchons de plus en plus nous pouvons sentir la chaleur de l'atmosphère nous envelopper, faisant vibrer notre vaisseau devenu terrestre, avec une intensité croissante. Les immensités de montagnes et de forêts se rapprochent de plus en plus de nous, comme si nous plongions dans une mer de verdure et de pierre. C'est une vision spectaculaire et époustouflante qui nous transporte à travers les magnifiques paysages de notre planète natale.

Notre vaisseau, le Babylon, vient de se poser sur une crête escarpée, surplombant des paysages grandioses à perte de vue. La vue est à couper le souffle, offrant une vision panoramique des montagnes majestueuses, des vallées verdoyantes et des rivières qui serpentent à travers le paysage. Nous sortons du Babylon, nos bottes crissant sur le sol pierreux et durci par le vent. L'air frais et pur de la montagne nous accueille, et nous respirons profondément, savourant la liberté retrouvée après des années de confinements. Le vent souffle fort, créant une symphonie de sons harmonieux qui se mêlent à la beauté brute et sauvage de la nature qui nous entoure. L'adieux aux bruits blanc. Nous contemplons les immensités de montagnes et de vallées, émerveillés par la beauté brute et sauvage de notre planète natale.

Nous sommes de retour chez nous. Nous avons parcouru des milliards de kilomètres à travers les confins de l'univers pour finalement atterrir sur ce morceau de terre, cette petite planète qui est notre foyer.

Nous sommes de retour chez nous, avec la certitude que l'exploration de l'univers ne fait que commencer, mais que notre planète restera toujours notre foyer, un endroit où nous pourrons revenir pour trouver refuge et inspiration.

Nous sommes de retour chez nous, et c'est un sentiment indescriptible.

L'ordinateur de bord du Babylon a été notre compagnon constant tout au long de notre voyage dans l'espace. Depuis le début, il a enregistré toutes nos données de navigation, de communication et de recherche scientifique, créant ainsi un enregistrement complet de notre aventure.

Nous avons eu l'idée de faire cela pour les générations futures, afin qu'elles puissent comprendre les défis et les merveilles de l'exploration spatiale. Nous avons conscience que notre voyage n'est qu'un petit pas dans l'histoire de l'humanité, mais nous sommes convaincus que notre expérience pourrait inspirer les générations futures à continuer à explorer l'espace et à étendre les limites de notre compréhension de l'univers.

L'ordinateur de bord a également créé une retranscription complète de notre aventure, y compris les détails de notre voyage depuis les confins de l'univers jusqu'à notre atterrissage sur Terre. Cette retranscription sera accessible à tous ceux qui cherchent à comprendre notre expérience et à apprendre de notre aventure.

Ainsi, notre aventure à bord du Babylon ne se termine pas avec notre retour sur Terre. Nous avons créé un héritage durable pour les générations futures, un enregistrement complet de notre aventure dans l'espace, qui continuera à inspirer et à éduquer ceux qui cherchent à comprendre notre place dans l'univers.

Cet ouvrage est une retranscription de l'explication donnée par l'IA du Babylon, le vaisseau spatial qui a conduit notre expédition. L'IA a été notre guide constant tout au long de notre voyage, nous fournissant des informations cruciales sur notre environnement et nous aidant à naviguer à travers les vastes étendues de l'espace.

Dans ce livre, nous avons rassemblé les explications de l'IA sur les images que nous avons capturées lors de notre exploration. Les images sont parfois spectaculaires, révélant des paysages d'une beauté à couper le souffle, des planètes étranges et des phénomènes cosmiques fascinants. Mais pour comprendre pleinement ces images, il est nécessaire d'avoir les explications détaillées de l'IA.

En somme, ce livre est un témoignage de notre voyage dans l'espace, capturant les moments clés de notre aventure et offrant une vue détaillée de l'univers à travers les yeux de l'IA du Babylon.

Dans ce livre, nous allons nous concentrer en particulier sur notre arrivée dans le système solaire et les vues de notre atterrissage sur la Terre. Ce moment était particulièrement émouvant pour nous, car après des décennies de voyage à travers l'espace, nous étions finalement de retour sur notre planète natale.

Nous avons capturé de magnifiques images de notre voyage à travers le système solaire, révélant les détails fascinants des planètes et des lunes que nous avons croisées en chemin. Cependant, c'est l'arrivée sur Terre qui est sans doute a plus émouvante de toutes les images que nous avons capturées.

Nous avons pu observer notre planète depuis l'espace, révélant une vue unique de la Terre et de la Lune dans leur ensemble. Puis, en nous rapprochant de plus en plus, nous avons pu capturer des images incroyables des montagnes, des vallées et des océans de notre planète. Nous avons atterri sur une crête escarpée, donnant sur un paysage grandiose à perte de vue.

Nous avons ensuite exploré la Terre, capturant des images étonnantes de la faune et de la flore, ainsi que des villes et des infrastructures humaines. Mais ce sera une autre hsitoire.

L’univers observable

Avant de commencer notre mission de retranscription, il est intéressant de faire une pause pour rappeler l'ampleur de notre sujet initial d'observation. Il s'agit de garder à l'esprit l'échelle de ce que l’IA du Babylon traduira en informations 3D afin de comprendre l’ensemble des choix réducteur qui seront fait à des fin d’optimisation.

« *.. Selon les estimations basées sur les données observationnelles, il y aurait environ 100 à 400 milliards d'étoiles dans la Voie lactée. Cependant, les incertitudes dans ces mesures et modèles signifient qu'il est difficile de donner une estimation précise du nombre exact d'étoiles dans notre galaxie. La plupart des étoiles de la Voie lactée sont faibles et ne sont pas facilement observables avec les techniques d'observation actuelles.*

Il est probable que la plupart des étoiles de la galaxie aient des systèmes planétaires similaires à notre système solaire. Il existe également des systèmes planétaires qui sont très différents de notre système solaire, par exemple des systèmes avec des étoiles multiples ou des planètes géantes proches de leur étoile. Cependant, il est difficile de le dire avec certitude car notre capacité à détecter des planètes en dehors de notre système solaire est limitée. Notre compréhension des systèmes planétaires en dehors de notre propre système est encore limitée, mais les observations récentes ont permis de découvrir des milliers de planètes en dehors de notre système.

En ce qui concerne l'univers, selon les estimations actuelles basées sur les données d'observation, il y aurait environ 100 à 200 milliards de galaxies observables dans l'univers observable. Cette estimation est sujette à de grandes incertitudes et peut varier en fonction des progrès dans les techniques d'observation et les modèles théoriques. Les distances dans l'univers sont généralement mesurées en milliers ou en millions de parsecs…»

IA du BABYLON – extrait du briefing de mission - Date stellaire 77021.1

La voie lactée

Concernant notre galaxie, la Voie lactée, le nombre d’étoile a été estimé à partir de diverses méthodes d'observation et de modèles théoriques. Cependant, à date de l’année 2023, selon les estimations basées sur les données observationnelles, il y aurait environ 100 à 400 milliards d'étoiles dans la Voie lactée. Cette estimation a été obtenue en utilisant des observations de différentes sources, telles que la luminosité de la galaxie, la distribution spatiale des étoiles et leur mouvement, ainsi que des modèles théoriques de la formation et de l'évolution des galaxies. Cependant, les incertitudes dans ces mesures et modèles signifient qu'il est difficile de donner une estimation précise du nombre exact d'étoiles dans notre galaxie. Il convient également de noter que la plupart des étoiles de la Voie lactée sont faibles et ne sont pas facilement

observables avec les techniques d'observation actuelles. Les étoiles les plus brillantes et les plus proches sont plus faciles à détecter et à étudier, mais elles représentent seulement une petite fraction du nombre total d'étoiles de la galaxie.

Il est probable que la plupart des étoiles de la galaxie aient des systèmes planétaires similaires à notre système solaire, mais il est difficile de dire avec certitude car notre capacité à détecter des planètes en dehors de notre système solaire est limitée. Cependant, les observations récentes ont montré que les planètes sont assez courantes dans la galaxie, ce qui suggère que de nombreuses étoiles pourraient avoir des systèmes planétaires.

Les systèmes

Le système solaire est le système planétaire dans lequel se trouve notre planète, la Terre. C'est un système assez typique, composé d'une étoile, le Soleil, et de plusieurs planètes, ainsi que d'autres corps célestes tels que des lunes, des astéroïdes et des comètes. Il existe également des systèmes planétaires qui sont très différents de notre système solaire, par exemple des systèmes avec des étoiles multiples ou des planètes géantes proches de leur étoile.

Il est important de noter que notre compréhension des systèmes planétaires en dehors de notre propre système solaire est encore limitée, car nous ne pouvons observer directement que quelques-uns d'entre eux. Cependant, les observations récentes de télescopes spatiaux et terrestres ont permis de découvrir des milliers de planètes en dehors de notre système solaire, et cela continue de s'améliorer grâce aux progrès dans les techniques d'observation et les modèles théoriques.

L’univers

Concernant l’univers, selon les estimations actuelles basées sur les données d'observations, il y aurait environ 100 à 200 milliards de galaxies observables dans l'univers observable, c'est-à-dire la partie de l'univers qui est suffisamment proche et lumineuse pour être détectée par les télescopes. Cette estimation a été obtenue à partir d'observations de différentes sources, telles que la luminosité des galaxies, leur distribution spatiale et leur mouvement, ainsi que des modèles théoriques de la formation et de l'évolution des galaxies. Cependant, il est important de noter que cette estimation est sujette à de grandes incertitudes et peut varier en fonction des progrès dans les techniques d'observation et les modèles théoriques. De plus, il existe probablement de nombreuses galaxies encore inobservables, car elles sont situées à des distances trop éloignées ou sont cachées derrière des régions obscures de l'univers.

Les Corps célestes

« ..En tant qu'Intelligence Artificielle embarquée à bord du vaisseau spatial Babylon, j'ai eu l'opportunité de traiter et d'analyser une immense quantité de données sur les corps célestes que nous avons rencontrés lors de notre voyage à travers notre système solaire. Nous avons visité et étudié toutes sortes de planètes, lunes, astéroïdes et comètes, chacun avec ses propres caractéristiques et particularités uniques. Nous avons observé des mondes rocheux arides, dépourvus d'atmosphère, tels que Mars ou la Lune, qui étaient exposés à des températures extrêmes et qui étaient stériles de toute forme de vie. D'autres planètes comme Jupiter, Saturne, Uranus et Neptune étaient des géantes gazeuses, entourées d'anneaux et de nuages tourbillonnants. Nous avons également étudié des planètes telles que Vénus, qui est enveloppée d'une épaisse couche de nuages acides et qui est soumise à une pression atmosphérique extrêmement élevée.

En plus des planètes, nous avons étudié des comètes, des astéroïdes et des météorites, qui sont des corps célestes plus petits et souvent plus étranges dans leur forme. Nous avons utilisé des scanners avancés pour analyser leur composition et leur structure, et avons découvert que certains d'entre eux contenaient des éléments précieux et rares tels que l'or et le platine.

Toutes les données que nous avons collectées ont été capturées par nos caméras et nos instruments de mesure, nous permettant une analyse et une compréhension approfondies de ces corps célestes fascinants. En étudiant la surface des planètes et les cratères d'impact, nous avons pu en apprendre davantage sur l'histoire de notre système solaire et des événements qui ont façonné notre univers.

En somme, notre voyage a été une opportunité incroyable de découvrir et de documenter les merveilles de l'espace, depuis les plus grandes planètes jusqu'aux plus petits astéroïdes. En utilisant des technologies de pointe et en analysant les données de manière rigoureuse, nous avons pu en apprendre davantage sur notre place dans l'univers et sur la diversité incroyable des corps célestes qui nous entourent… »

IA du BABYLON – extrait de rapport de mission - Date stellaire 77061.3

Types

voici une liste des principaux types de corps célestes que nous avons rencontrés lors de notre voyage à bord du Babylon :

- Étoiles : Les étoiles sont des corps célestes massifs qui produisent de la lumière et de la chaleur par le processus de fusion nucléaire.

- Planètes : Des corps célestes orbitant autour d'une étoile, de taille suffisamment grande pour avoir une forme sphérique sous l'effet de la gravité.
    - planètes rocheuses - des corps célestes solides et rocheux, tels que Mercure, Vénus, la Terre et Mars.
    - planètes géantes gazeuses - des corps célestes massifs principalement composés de gaz, tels que Jupiter, Saturne, Uranus et Neptune.
- Satellites naturels - des corps célestes qui orbitent autour d'une planète, telles que la Lune de la Terre, Io autour de Jupiter, ou encore Encelade autour de Saturne.
- Astéroïdes - des corps célestes rocheux et métalliques, de taille variable, se trouvant principalement dans la ceinture d'astéroïdes entre Mars et Jupiter.
- comètes - des corps célestes principalement composés de glace et de poussière, qui ont des orbites très elliptiques autour du soleil.
- météorites - des corps célestes qui ont survécu à leur entrée dans l'atmosphère terrestre et qui sont tombés sur notre planète.
- Naines brunes : Les naines brunes sont des objets stellaires qui ne sont pas suffisamment massifs pour déclencher la fusion nucléaire et devenir une étoile.
- Trous noirs : Les trous noirs sont des objets astronomiques qui ont une force gravitationnelle si intense que rien, pas même la lumière, ne peut en sortir.
- Galaxies : Les galaxies sont des groupes de millions d'étoiles, de gaz et de poussière qui sont liés par la gravité.
- Nébuleuses : Des nuages interstellaires de gaz et de poussière.

Pour classer les corps célestes, nous avons utilisé une approche de modélisation informatique basée sur la création de classes d'objets. Nous avons créé une classe d'objets pour chaque type de corps céleste, telle que les planètes, les étoiles, les astéroïdes, les comètes et les lunes.

Chaque classe d'objets avait des propriétés spécifiques qui lui étaient propres, telles que la masse, le rayon, la composition, la température et l'orbite. Nous avons également ajouté des méthodes à chaque classe d'objets pour les rendre plus facilement accessibles et manipulables.

Ensuite, nous avons créé des relations entre les différentes classes d'objets. Par exemple, nous avons créé une relation entre les planètes et leur lune respective, ou entre les étoiles et leurs planètes en orbite.

Cette modélisation informatique a été d'une grande utilité pour notre communication avec les équipes scientifiques et les autres vaisseaux spatiaux que nous avons croisés au cours de notre voyage.

Afin de présenter cette classification, nous utiliserons un langage structuré. Le langage historique du Babylon étais le C++ mais la nécessité d'échange nous a conduit à privilégier Type Script pour ce rapport. D'autre langage sont utilisé par d'autre vaisseaux. Le C# sur le Unity et le C++ sur le Unreal. La modélisation reste toutefois identique et la retranscription d'un langage à l'autre triviale.

Coordonnées 3D



Les systèmes de coordonnées sont utilisés pour décrire la position d'un point dans l'espace en trois ou quatre dimensions. Cet espaces est infinis et les méthodes de calcul diffèrent selon l'échelle considérée.

L'échelle de distance utilisée pour l'univers est généralement le parsec (pc) ou le kiloparsec (kpc). Un parsec équivaut à environ 3,26 années-lumière, soit environ 30 856 milliards de kilomètres. Les distances dans l'univers sont généralement mesurées en milliers ou en millions de parsecs.

Les distances entre les galaxies sont généralement mesurées en mégaparsecs (Mpc), qui équivaut à un million de parsecs, ou en kilomètres. Par exemple, la distance entre notre galaxie, la Voie lactée, et la galaxie d'Andromède est d'environ 2,5 millions d'années-lumière, soit environ 770 kiloparsecs.

Les distances entre les corps célestes dans un système planétaire sont généralement mesurées en unités astronomiques (UA). Une unité astronomique équivaut à la distance moyenne entre la Terre et le Soleil, soit environ 149,6 millions de kilomètres. Par exemple, la distance entre la Terre et Mars est d'environ 1,5 UA, tandis que la distance entre la Terre et Neptune est d'environ 30 UA.

Dans la suite de cet ouvrage, nous allons zoomer du système solaire vers les corps célestes puis vers la position à leur surface (ou proche périphérie). C'est dans cette dernières zone que nous allons développer nos algorithmes de terrains.

Nous parlerons donc dans les prochains paragraphes des coordonées 3D

- au sein des systèmes.
- a la surface des corps celestes (avec les planetes en particulier).

Pour aboutir sur les coordonées des terrains et les algorithmes qui y sont associés. Algorithmes portés par le CPU et le GPU.

Coordonnées 3D des objets célestes.

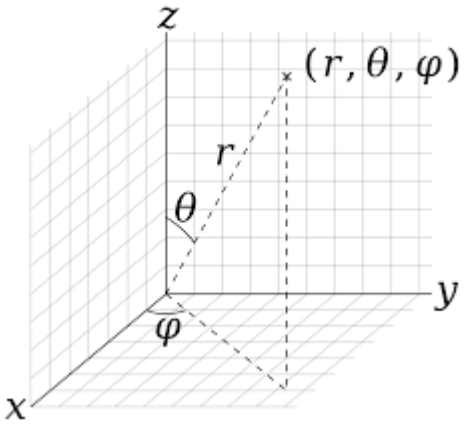
https://solarsystem.nasa.gov/basics/chapter2-1/

Les coordonnées des objets célestes sont utilisées pour décrire leur position et leur mouvement dans le système solaire. Elles peuvent être exprimées en termes de coordonnées géocentriques ou héliocentriques, qui considèrent respectivement la Terre ou le Soleil comme le centre du système. Les coordonnées géocentriques sont souvent exprimées en termes de longitude, de latitude et de distance, tandis que les coordonnées héliocentriques sont exprimées en termes d'orbite elliptique autour du Soleil.

Les coordonnées de ces objets peuvent également être exprimées en termes de coordonnées équatoriales, qui décrivent la position d'un objet en termes de déclinaison et d'ascension droite, comme pour les galaxies. Les coordonnées équatoriales permettent de suivre leur mouvement apparent sur le ciel vu de la Terre.

La position cartésienne d'une planète peut être calculée à partir de ses coordonnées géocentriques ou héliocentriques en utilisant des transformations mathématiques.

Prenons le cas des coordonnées géocentriques, qui peuvent être exprimées en termes de longitude, de latitude et de distance. Pour obtenir la position cartésienne, la longitude et la latitude doivent d'abord être converties en coordonnées sphériques en utilisant des relations trigonométriques. Ensuite, ces coordonnées sphériques peuvent être converties en coordonnées cartésiennes en utilisant les formules simples suivantes:



$$x = r * \sin(\theta) * \cos(\varphi)$$

$$y = r * \sin(\theta) * \sin(\varphi)$$

$$z = r * \cos(\theta)$$

où r est la distance de la planète à la Terre, latitude est la latitude géocentrique, et longitude est la longitude géocentrique.

- θ est l'angle complémentaire de la latitude soit $^{\pi}/_{2} - \theta$
- φ est la longitude

Le processus est similaire pour les coordonnées héliocentriques, sauf que la référence est le Soleil au lieu de la Terre. Les coordonnées cartésiennes peuvent également être obtenues à partir des éléments orbitaux d'une planète, tels que son excentricité, sa longitude du périhélie et sa longitude du nodal, en utilisant des formules mathématiques plus complexes.

Enfin, il est important de noter que les coordonnées des planètes dépendent du temps, car leurs positions et mouvements évoluent. Les coordonnées doivent donc être spécifiées pour un moment donné afin de garantir leur exactitude.

Lorsque nous décrivons ces systèmes de coordonnées, gardez à l'esprit le système de référence employé. En effet vous êtes dans l'espace infinis et par définition il n'y a PAS de référence absolue, donc il faut en fixer une. La majorité des modèles, données et calculs que vous allez rencontrer dans la bibliographie prennent la terre et son plans écliptique pour référence, historiquement nous somme le centre de l'univers !. Sachez également que l'astronomie possède son propre langage avec des termes désignant des concept spatiaux temporels spécifiques. C'est un sujet d'étude à part entière et nous allons essayer de le présenter de la manière la plus accessible possible avec des raccourcis qui pourront être parfois mal perçus par les spécialistes. Sachez néanmoins que cet ouvrage a été relus par des universitaire afin d'en supprimer les propos blasphématoire .

Si l'on était sur Jupiter, on utiliserait également les mêmes coordonnées équatoriales pour spécifier la position des étoiles, mais ces coordonnées seraient définies par rapport au système de référence de Jupiter plutôt que par rapport au système de référence de la Terre.

Pour déterminer la position des étoiles depuis Jupiter, il faudrait donc appliquer une transformation de coordonnées pour convertir les coordonnées équatoriales des étoiles depuis le système de référence de la Terre vers le système de référence de Jupiter.

Les coordonnées équatoriales des étoiles peuvent être exprimées en termes de leur ascension droite (RA) et de leur déclinaison (DEC), qui sont les coordonnées utilisées dans les catalogues d'étoiles. Pour convertir les coordonnées équatoriales de la Terre vers celles de Jupiter, il faut utiliser des formules de transformation appropriées qui tiennent compte de la position relative de Jupiter et de la Terre dans leur orbite autour du Soleil.

Une fois que les coordonnées équatoriales des étoiles ont été converties dans le système de référence de Jupiter, elles peuvent être utilisées pour déterminer la position apparente des étoiles depuis Jupiter en fonction de leur ascension droite et de leur déclinaison.

Les formules de transformation pour convertir les coordonnées équatoriales de la Terre vers celles de Jupiter sont assez complexes et dépendent de la position relative de Jupiter et de la Terre dans leur orbite autour du Soleil. Voici les formules générales pour la transformation des coordonnées équatoriales de la Terre vers celles de Jupiter :

Calcul de la position de Jupiter par rapport au Soleil : Il faut d'abord calculer la position de Jupiter par rapport au Soleil à un moment donné, en utilisant des formules orbitales. Cette position peut être exprimée en termes de coordonnées écliptiques (longitude et latitude).

Calcul de la position de la Terre par rapport à Jupiter : Ensuite, il faut calculer la position de la Terre par rapport à Jupiter à ce même moment, également en utilisant des formules orbitales. Cette position peut être exprimée en termes de coordonnées écliptiques.

Conversion des coordonnées équatoriales de la Terre vers les coordonnées écliptiques : Les coordonnées équatoriales de l'étoile peuvent être converties en coordonnées écliptiques à l'aide de formules de transformation appropriées.

Transformation des coordonnées écliptiques de la Terre vers les coordonnées écliptiques de Jupiter : Les coordonnées écliptiques de l'étoile calculées à partir des coordonnées équatoriales de la Terre peuvent être transformées en coordonnées écliptiques de Jupiter en appliquant une transformation de coordonnées appropriée. Cette transformation dépend de la position relative de Jupiter et de la Terre dans leur orbite autour du Soleil.

Conversion des coordonnées écliptiques de Jupiter en coordonnées équatoriales : Les coordonnées écliptiques de l'étoile calculées à partir des coordonnées équatoriales de la Terre peuvent ensuite être converties en coordonnées équatoriales de Jupiter à l'aide de formules de transformation appropriées.

Une fois que les coordonnées équatoriales de l'étoile ont été transformées dans le système de référence de Jupiter, elles peuvent être utilisées pour déterminer sa position apparente depuis Jupiter en fonction de son ascension droite et de sa déclinaison

Mouvement des objets célestes

Si l'on considère le seul système solaire, celui-ci se déplace au sein de notre galaxie, qui elle-même se déplace dans l'univers, au grés des fluctuation de la gravité et des lois relativiste.

Les corps célestes dans le système solaire sont soumis à la gravité, qui les attire les uns vers les autres. En conséquence, ils se déplacent en suivant des orbites elliptiques autour du Soleil. Cette orbite est déterminée par la masse et la vitesse de chaque corps céleste, ainsi que par la force gravitationnelle exercée par le Soleil et les autres corps célestes du système solaire. Les planètes se déplacent à des vitesses différentes en fonction de leur distance au Soleil et de leur masse, ce qui détermine la durée de leur année et la forme de leur orbite. Les corps plus petits, tels que les astéroïdes et les comètes, peuvent également être influencés par les forces gravitationnelles exercées par les autres corps du système solaire. En général, les mouvements des corps célestes dans le système solaire sont très prévisibles et peuvent être décrits avec une grande précision grâce aux lois de la mécanique céleste.

Il existe plusieurs méthodes pour décrire ces mouvements, mais les plus couramment utilisées sont :

- La mécanique céleste newtonienne : Cette méthode utilise les lois de la gravitation décrites par Isaac Newton pour décrire les mouvements des corps célestes. Elle permet de déterminer la forme et la vitesse des orbites ainsi que les positions et les vitesses des corps célestes à un moment donné.
- La méthode de l'équation de Laplace-Lagrange : Cette méthode utilise des équations mathématiques pour décrire les mouvements des corps célestes. Elle est plus précise que la mécanique céleste newtonienne et est utilisée pour déterminer les positions et les vitesses des corps célestes à long terme.
- La méthode des N-corps : Cette méthode est utilisée pour décrire les mouvements des corps célestes lorsqu'il y a plusieurs corps qui interagissent entre eux. Elle utilise des algorithmes informatiques pour simuler les interactions gravitationnelles entre les corps célestes.
- La relativité générale : Cette méthode est une théorie de la gravitation décrite par Albert Einstein. Elle décrit les mouvements des corps célestes en prenant en compte la courbure de l'espace-temps causée par la présence de masses.

En général, la méthode utilisée dépend des besoins et des objectifs de la simulation, mais la mécanique céleste newtonienne et la méthode de l'équation de Laplace-Lagrange sont les plus couramment utilisées pour décrire les mouvements des corps célestes dans le système solaire et suffisamment précis pour nos objectifs.

L'équation de Kepler se situe parmi les méthodes de la mécanique céleste newtonienne. C'est en fait une loi qui décrit la forme des orbites des corps autour du Soleil. Cette équation stipule que la vitesse angulaire d'un corps sur son orbite est proportionnelle à la racine carrée de la constante gravitationnelle divisée par le demi-

grand axe de l'orbite. Cette équation est très utile pour déterminer la forme et la vitesse des orbites, ainsi que la position approximative et la vitesse des corps célestes à un moment donné. En somme, l'équation de Kepler est un élément clé de la mécanique céleste newtonienne pour décrire les mouvements des corps dans le système solaire.

Il n'existe pas de méthode absolue pour résoudre les équations de Kepler car cela dépend de la programmation, de la précision souhaitée, des données utilisées, et de la façon dont les équations sont formulées. Il existe des bibliothèques dans différents langage de programmation (JavaScript, c#, c++, etc…).

Il est intéressant de noter que le mouvements de la terre peut être décris de manière approximative comme keplerien (problème gravitationnel a deux corps). Ce n'est pas le cas du mouvement de la lune qui est beaucoup plus complexe, car étroitement lié à l'attraction de la terre ET du Soleil. La position de la lune est ainsi donnée par une résolution du problème à trois corps.

Approximation des positions

https://ssd.jpl.nasa.gov/planets/approx_pos.html

https://www.danbp.org/p/en/node/139

https://github.com/danpeig/kepler-planet-orbits-js

Ellipsoides

Coordonnées 3D relatives aux corps célestes.

Il existe plusieurs systèmes de coordonnées utilisés pour positionner un point sur une ellipsoïde, qui est une forme tridimensionnelle représentant un corps celeste (dans notre cas particulier, la Terre).

- Le système de coordonnées géodesique décrit la position des objets sur la surface de la planète en utilisant des coordonnées de latitude, de longitude et de hauteur par rapport à un référentiel géocentrique lié à la planète elle-même. Les coordonnées de latitude décrivent la position nord-sud sur la planète, les coordonnées de longitude décrivent la position est-ouest sur la planète, et les coordonnées de hauteur décrivent la position verticale par rapport à la surface de la planète.

- Le système de coordonnées **ECEF** (Earth-Centered Earth-Fixed) est un système de coordonnées à trois dimensions qui place l'origine au centre de la Terre. Le vecteur x est dirigé vers le point vernal[1] (ou équinoxe), le vecteur y est dirigé vers le méridien de Greenwich et le vecteur z est dirigé vers le pôle nord.

  Il n'y a pas de généralisation systématique de coordonnées « **ECEF »** pour les autres planètes (A l'exception de Mars ou l'exploration à crée le besoin) ou satellites (Lunes, etc..), bien que la description soit purement mathématiques, les particularités liée au point vernal et au méridien de Greenwich sont purement arbitraire. Cependant, il est possible de définir un équivalent du point vernal sur une autre planète de notre système solaire. Donc, sur chaque corps céleste, on peut définir un système de coordonnées similaire à celui utilisé sur la Terre, avec une équateur céleste, une orbite apparente du Soleil dans le ciel, et un point de départ pour la mesure de la longitude.

  Il existe ainsi des systèmes de coordonnées spécifiques à chaque corps céleste, tels que les systèmes IAU (International Astronomical Union (International Astronomical Union, s.d.)) pour les planètes du système solaire, qui définissent une référence pour les observations astronomiques et les mesures spatiales sur chaque corps céleste.

- Le système **ENU** (East-North-Up) décrit la position d'un point en termes de coordonnées locales, exprimées par rapport à un repère lié à une position donnée sur la planète et au plans tangent à ce même point. Il est fortement lié au système ECEF (ou un équivalent) puisque définis par rapport à ce dernier sous forme d'une translation et d'une rotation.
- Le système **NED** (North-East-Down) est similaire au système ENU, mais les coordonnées sont exprimées par rapport à un repère orienté vers le bas.

Chacun de ces systèmes de coordonnées a ses propres avantages et inconvénients en termes de précision, de facilité d'utilisation et d'applicabilité. Il est important de choisir le système de coordonnées le plus approprié en fonction de l'application et des exigences de précision.

[1] Le point vernal est un point sur l'écliptique, qui est la projection sur le ciel du plan de l'orbite terrestre autour du Soleil. C'est également connu sous le nom d'équinoxe de printemps, car c'est le moment où le Soleil passe à travers l'équateur céleste, donnant lieu à des jours et des nuits de durée égale partout sur Terre. Le point vernal marque le début astronomique du printemps. Voir le chapitre sur les coordonné astronomiques pour plus de détails

Le système géodésique

Un système géodésique est un système de coordonnées utilisé pour décrire la position et la forme de la planète Il est généralement utilisé pour la cartographie et la navigation.

Le système géodésique est basé sur un modèle « sphérique » de la planète, appelé ellipsoïde de référence, qui est utilisé pour représenter la forme du corps céleste. L'ellipsoïde de référence est souvent défini en termes de deux grandeurs (il en existe 3 mais on négligera ici le défaut de forme à l'équateur):

- le rayon à l'équateur
- le défaut de forme qui mesure l'écart entre l'ellipsoïde et un modèle sphérique parfait.

Le système géodésique est associé à un système de coordonnées, qui définit les positions sur la surface de la Terre en termes de latitude, de longitude et d'altitude. La latitude est la distance en degrés nord ou sud de l'équateur, la longitude est la distance en degrés est ou ouest du méridien de Greenwich et l'altitude est la distance au-dessus ou au-dessous du niveau de la mer. Ces valeurs sont géocentriques.

Ellipsoid.

Il y a deux types de systèmes géodésiques : les systèmes terrestres, qui ont été conçus avant l'ère spatiale et qui sont spécifiques à une région ou à un pays, et les systèmes spatiaux, qui sont basés sur des technologies spatiales et couvrent l'ensemble de la planète.

Il existe aujourd'hui plusieurs systèmes géodésiques. Le plus commun est le système géodésique WGS 84 (World Geodetic System 1984), qui est utilisé pour la plupart des applications militaires et civiles. Il a été développé par le Département de la Défense des États-Unis pour une utilisation dans les systèmes de positionnement par satellite, tels que le GPS (Global Positioning System).

WGS84 est conçu pour fournir des positions précises sur l'ensemble de la surface de la Terre, y compris dans les zones polaires. La précision de WGS84 dépend de la méthode utilisée pour déterminer les coordonnées géodésiques d'un point. En général, la précision de WGS84 est considérée comme très élevée pour les applications de navigation et de cartographie, avec une précision de l'ordre de quelques mètres. Cependant, la précision réelle peut varier en fonction de nombreux facteurs, tels que la qualité des récepteurs GPS utilisés, les erreurs atmosphériques et les effets de multipath.

En termes de précision relative par rapport à la forme de la Terre, WGS84 est considéré comme précis à environ 2 cm sur la surface de la Terre, ce qui en fait l'un des systèmes géodésiques les plus précis disponibles.

Il est intéressant de localiser l'ensemble de ces paramètre dans une classe, et d'y sauver des valeurs intermédiaires qui nous servirons dans nos prochains développements.

```
class Ellipsoid {
  constructor(name, semiMajor, semiMinor, flattening, inverseFlattening) {
    this._name = name;
    this._a = semiMajor;
    this._b = semiMinor;
    this._f = flattening;
    this._p1mf = 1.0 - this._f;
    this._invf = inverseFlattening;
    this._aa = this._a * this._a;
    this._bb = this._b * this._b;
    this._c = Math.sqrt(this._aa - this._bb);
    this._ee = 1 - this._bb/this._aa ;
    this._e = Math.sqrt(this._ee);
    this._invaa = 1.0 / this._aa;
    this._aadc = this._aa / this._c;
    this._bbdcc = this._bb / (this._c * this._c);
    this._l = this._ee / 2;
    this._p1mee = 1 - this._ee;
    this._p1meedaa = this._p1mee / this._aa;
    this._hmin = Math.pow(this._e, 12) / 4;
    this._ll = this._l * this._l;
    this._ll4 = this._ll * 4;
  }
  static FromAAndInverseF(name, semiMajor, inverseFlattening) {
    const f = 1.0 / inverseFlattening;
    const b = (1.0 - f) * semiMajor;
    return new Ellipsoid(name, semiMajor, b, f, inverseFlattening);
  }
  static FromAAndF(name, semiMajor, flattening) {
    const inverseF = 1.0 / flattening;
    const b = (1.0 - flattening) * semiMajor;
    return new Ellipsoid(name, semiMajor, b, flattening, inverseF);
  }
  get Name() {
    return this._name;
  }
  get SemiMajorAxis() {
    return this._a;
  }
  get SemiMinorAxis() {
    return this._b;
  }
  get Flattening() {
    return this._f;
  }
  get InverseFlattening() {
    return this._invf;
  }
  get LinearEccentricity() {
    return this._c;
  }
```

```
  get Eccentricity() {
    return this._e;
  }
  get SqrEccentricity() {
    return this._ee;
  }
  get SemiLatusRectum() {
    return this._p1mee * this._a;
  }
  IsEquals(other) {
    return other && other._a == this._a && other._b == this._b;
  }
  clone(name, scale = 1.0) {
    return new Ellipsoid(name, this._a * scale, this._b * scale, this._f, this._invf);
  }
}
Ellipsoid.d2r = Math.PI / 180;
Ellipsoid.r2d = 180.0 / Math.PI;
Ellipsoid.WGS84 = Ellipsoid.FromAAndInverseF("WGS84", 6378137.0, 298.257223563);
Ellipsoid.GRS80 = Ellipsoid.FromAAndInverseF("GRS80", 6378137.0, 298.257222101);
Ellipsoid.GRS67 = Ellipsoid.FromAAndInverseF("GRS67", 6378160.0, 298.25);
Ellipsoid.ANS = Ellipsoid.FromAAndInverseF("ANS", 6378160.0, 298.25);
Ellipsoid.WGS72 = Ellipsoid.FromAAndInverseF("WGS72", 6378135.0, 298.26);
Ellipsoid.Clarke1858 = Ellipsoid.FromAAndInverseF("Clarke1858", 6378293.645, 294.26);
Ellipsoid.Clarke1880 = Ellipsoid.FromAAndInverseF("Clarke1880", 6378249.145, 293.465);
```

Il est important de souligner la présence de la fonction clone() dans cette classe. En effet, les paramètres de l'ellipsoïde sont souvent exprimé dans des unité (le mètre ici) qui rendent l'utilisation de certain système de référence comme ECEF inutilisable par les moteur de 3D. De plus si on représente le système solaire, et si l'on décide de travailler en coordonnée astronomique (1 unité est égale à la distance terre soleil) il sera indispensable d'appliquer un facteur d'échelle à notre ellipsoïde.

Ainsi il sera simple de définir une ellipsoïde de référence pour nos travaux

```
const ellipsoidOfRef = Ellipsoid.WGS84.clone("custom", 1/1000000)
```

Position

Les coordonnées géodésiques sont également appelées coordonnées de latitude, longitude et altitude.

La latitude mesure l'angle nord-sud entre le point et l'équateur, en partant de zéro à l'équateur jusqu'à 90 degrés au pôle Nord ou au Sud. La longitude mesure l'angle est-ouest entre le point et un méridien de référence, généralement le méridien de Greenwich. L'altitude mesure la distance verticale entre le point et une surface de référence, telle que le niveau de la mer.

Une définition simpliste en javascript pourrait être :

```
class Location {

  constructor(lat = .0, lon = .0, alt = .0) {
    this.lat = lat;
    this.lon = lon;
    this.alt = alt;
  }
  static Zero() {
    return new Location(0, 0, 0);
  }
  clone() {
    return new Location(this.lat, this.lon, this.alt);
  }
  set(l) {
    if (l) {
      this.lat = l.lat;
      this.lon = l.lon;
      this.alt = l.alt;
    }
    return this;
  }
  equals(other) {
    if (this === other) {
      return true;
    }
    if (!other) {
      return false;
    }
    return this.lat === other.lat && this.lon === other.lon && this.alt === other.alt;
  }
  toString(){
    return '[' +  this.lat + ',' + this.lon + ',' + this.alt + ']';
  }
}
```

Enveloppe

Les enveloppes en coordonnées géodésiques sont des courbes qui relient les points ayant la même latitude ou la même longitude. Les enveloppes de latitude sont des cercles parallèles à l'équateur, tandis que les enveloppes de longitude sont des demi-cercles allant d'un pôle à l'autre. Les enveloppes ont une logique particulière car elles permettent de diviser la surface courbe en régions régulières. Par exemple, la surface de la Terre est divisée en 180 bandes de longitude de 2 degrés chacune, et en cercles de latitude tous les 10 degrés. Cette grille est utilisée pour faciliter la navigation et la représentation des données géographiques.

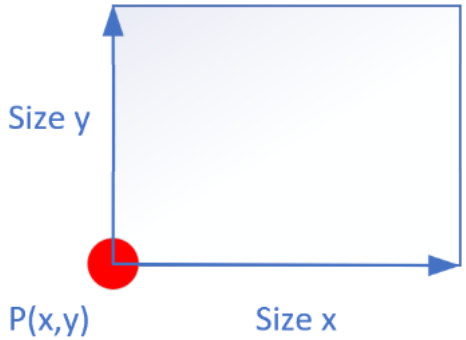
Il convient également de noter que les enveloppes ne se croisent pas à des angles droits, sauf aux pôles. Cela signifie que les distances mesurées le long des enveloppes de latitude et de longitude ne sont pas équivalentes, sauf aux équateurs et aux pôles. Par conséquent, pour mesurer les distances entre deux points

sur une surface courbe, il est souvent nécessaire d'utiliser d'autres méthodes, telles que la distance géodésique, qui suit la courbure de la surface.

Il y a donc deux discontinuités sur la métrique des enveloppes.

1. La ligne de longitude correspondant à -180° et +180°.
2. Les pôles correspondants à des latitudes de -90° et +90°

Si l'on définis une enveloppe par son point initial et ses dimensions, l'on se doit de mettre des règles de traitement sur l'affectation de ces dimensions, afin de toujours avoir une enveloppe cohérente.



Les dimensions peuvent être positive ou négative.

Une astuce consiste à travailler sur un repère élargis avec les règles suivantes.

$$\begin{cases} x, & x \in [-540|540] \\ y, & y \in [-90|90] \\ size.x, & x + size.x \in [-540|540] \\ size.y, & y + size.y \in [-90|90] \end{cases}$$

Ainsi les tests d'intersection et d'unions se rapporterons a des cas simples avec size x et size y toujours positifs.

L'enveloppe sera ainsi définie implicitement par les propriété suivantes North, South, East, West

Et les points géodésiques correspondant, SouthWest, NorthWest, NorthEast, SouthEast, Center.

Avec le système suivant.

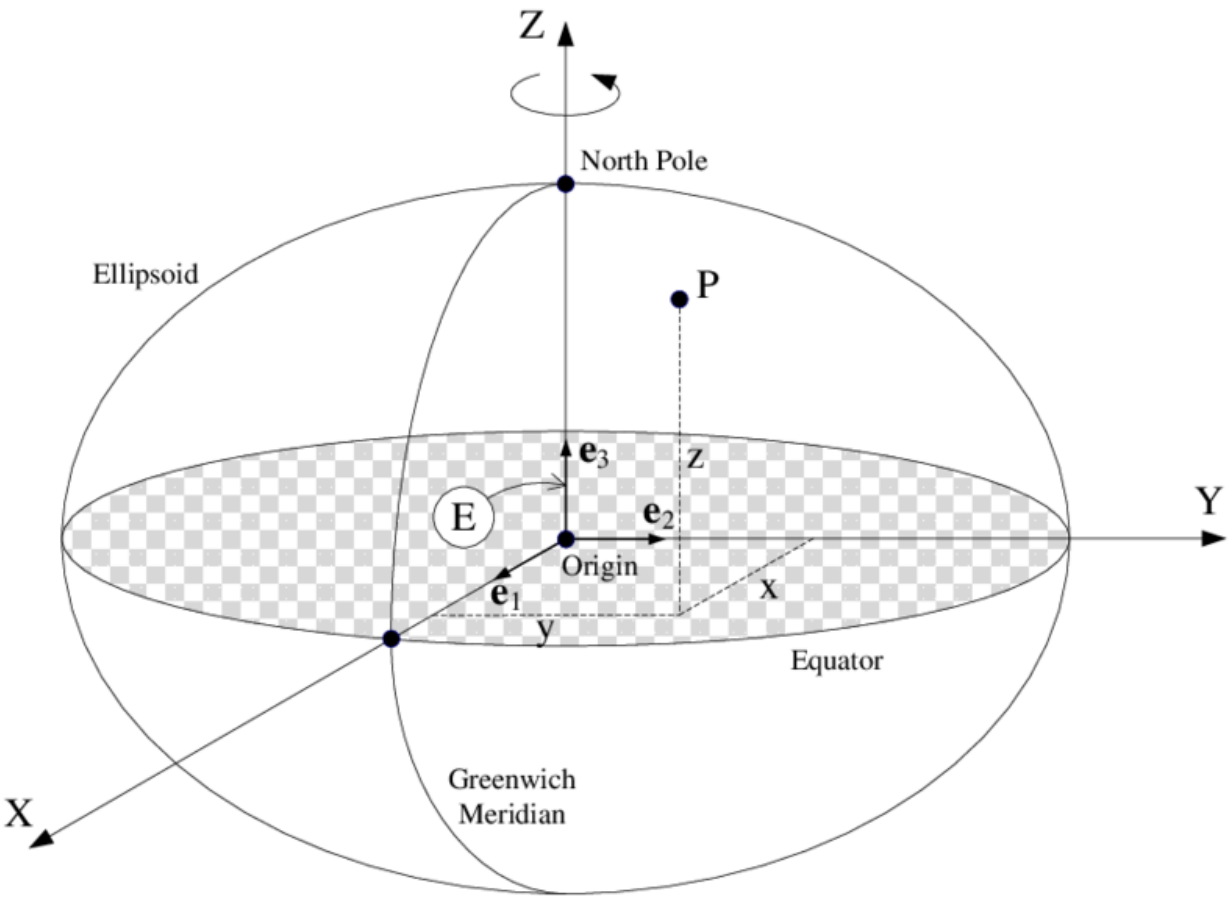
$$\begin{cases} North, & x \in [-90|90] \\ South, & y \in [-90|90] \\ East, & x \in [-180|180] \\ West, & y \in [-180|180] \\ North \geq South & \\ West \geq East & \end{cases}$$

Résumé

Dans ce paragraphe, nous avons introduit les concepts de coordonnées géodésiques et géocentriques, ainsi que les positions, enveloppes, ellipsoïdes et leurs paramètres. Il est important de se rappeler que notre discussion était de nature introductive et simplifiée pour rendre le sujet plus accessible.

Dans la prochaine section, nous allons nous concentrer sur le système de coordonnées cartésiennes ECEF (Earth-Centered, Earth-Fixed) ainsi que ses systèmes dérivés ENU (East-North-Up) et NED (North-East-Down). Ces systèmes de coordonnées sont essentiels pour les moteurs de rendu 3D.

Le système ECEF



Le système ECEF (Earth-Centered, Earth-Fixed) est un système de coordonnées cartésiennes tridimensionnelles utilisé pour localiser des points sur ou près de la surface de la Terre.

Dans le système ECEF, les coordonnées d'un point sont définies par sa distance par rapport à l'axe X (qui passe par l'équateur à la longitude 0°), sa distance par rapport à l'axe Y (qui passe par l'équateur à la longitude 90° Est) et sa distance par rapport à l'axe Z (qui passe par le pôle Nord).

Les formules pour convertir des coordonnées géodésiques (latitude, longitude, altitude) en coordonnées ECEF (x, y, z) sont les suivantes:

1 - Calcul de la normale N à la surface de la Terre à partir de la latitude :

$$N_\varphi = \frac{a}{\sqrt{(1 - e^2 sin^2 \varphi)}}$$

où :

- a est le rayon équatorial de l'ellipsoïde (6378137 mètres pour la terre)
- e est l'excentricité de l'ellipsoïde de référence (0.0818191908426 pour l'ellipsoïde WGS-84)
- φ est la latitude du point en radians

2 - Calcul des coordonnées x, y, z en utilisant la latitude, la longitude et l'altitude :

$$x = (N + h) * \cos(\varphi) * \cos(\lambda)$$

$$y = (N + h) * \cos(\varphi) * \sin(\lambda)$$

$$z = (\mathrm{N}(1 - \mathrm{e}^2) + \mathrm{h}) * \sin(\varphi)$$

où :

- h est l'altitude du point par rapport à la surface de la Terre
- λ est la longitude du point en radians

A l'inverse pour convertir des coordonnées ECEF en coordonnées géodésiques, les formules sont plus complexes et impliquent l'utilisation de méthodes itératives ou de tables de conversion.

La raison pour laquelle une méthode itérative est utilisée pour calculer la latitude ϕ à partir des coordonnées ECEF (x, y, z) est que l'équation qui lie les coordonnées géodésiques et ECEF est non-linéaire. Cette équation est la suivante :

$$N_\varphi = \frac{a}{\sqrt{(1 - e^2 sin^2 \varphi)}}$$

où

- N est la normale à la surface de la Terre (voir le passage géodésiques vers ECEF pour plus de détails)
- a est le rayon équatorial de la Terre

- et e est l'excentricité de l'ellipsoïde de référence.

Cette équation ne peut pas être résolue de manière analytique pour φ en fonction de x, y et z.

Cependant, elle peut être résolue de manière itérative en utilisant la méthode de l'itération de Bowring (Source : Bowring, 1985, The accuracy of geodetic latitude and height equations, Survey Review, 28, pp202-206).

Cette méthode consiste à répéter un processus de calcul jusqu'à ce qu'une certaine condition de convergence soit atteinte. Dans ce cas, la condition de convergence est que la différence entre les valeurs successives de φ soit inférieure à une certaine tolérance.

Pour appliquer la méthode de l'itération de Bowring, il faut commencer par un calcul approximatif de la latitude φ. Cette approximation de la latitude est ensuite utilisée comme point de départ pour l'itération. La méthode de l'itération est ensuite appliquée de manière répétée en utilisant la formule suivante :

$$\varphi = atan2(z/(1 - e^2 N/(N + h)), \sqrt{x^2 + y^2} * e^2/(N + h))$$

$$delta = abs(\varphi - atan2(z, \sqrt{x^2 + y^2}))$$

où N est mis à jour à chaque itération en utilisant la latitude de l'itération précédente. La méthode est répétée jusqu'à ce que la différence entre deux itérations successives (delta) de φ soit inférieure à une certaine tolérance.

La méthode est donc la suivante :

1 – calcul approximatif de la latitude.

$$\varphi = atan2(z, \sqrt{x^2 + y^2})$$

Dans cette formule, z représente la coordonnée ECEF en direction du pôle nord, et x et y représentent les coordonnées ECEF dans le plan équatorial.

L'expression $\sqrt{x^2 + y^2}$ représente la distance du point à l'axe de rotation de la Terre (correspondant à l'équateur), tandis que $atan2(z, \sqrt{x^2 + y^2})$ donne l'angle formé par ce vecteur avec l'axe de rotation de la planète, c'est-à-dire la latitude du point.

Notez que cette formule ne fonctionne pas pour les points situés directement sur les pôles de la Terre, où x et y sont nuls. Dans ce cas, la latitude est simplement égale à +90 degrés pour le pôle nord et -90 degrés pour le pôle sud.

2 - Calcul de la normale N à la surface de la Terre à partir de la latitude

$$N_\varphi = a\frac{a}{\sqrt{(1 - e^2 sin^2\varphi)}}$$

3 - Calcul de la longitude λ

$$\lambda = atan2(y, x)$$

4 - Calcul de la distance h entre le point et le centre de la Terre

$$\mathrm{h} = \|r\| - N$$

où r est le vecteur position du point en coordonnées ECEF, et ||r|| est sa norme.

5 - Calcul itératif

Tant que delta > tolérance et nombre d'itération < i

$$\varphi = atan2(z/(1 - e^2 N/(N + h)), \sqrt{x^2 + y^2} * e^2/(N + h))$$

$$delta = abs(\varphi - atan2(z, \sqrt{x^2 + y^2}))$$

Même si ce calcul ne sera pas utile pour la suite de cet ouvrage il faut rappeler que cette méthode itérative présente certaines limites, notamment en termes d'erreur et de vitesse de calcul.

- En ce qui concerne l'erreur, l'itération de Bowring peut produire des résultats précis pour des points situés près de la surface de la Terre, mais l'erreur peut augmenter à mesure que l'on s'éloigne de la surface (plusieurs centaines de mètres).
- En ce qui concerne la vitesse d'exécution, l'itération de Bowring peut nécessiter un nombre élevé d'itérations pour converger, en particulier pour des points situés à haute altitude. Cela peut entraîner des temps de calcul plus longs, en particulier pour des applications nécessitant des conversions rapides et précises de grandes quantités de données.

Il existe des méthodes plus récentes et plus rapides pour la conversion de coordonnées ECEF en coordonnées géodésiques, telles que la méthode de Hatanaka et la méthode de l'approximation polynomiale. Ces méthodes peuvent offrir une meilleure précision et une vitesse de calcul plus rapide, mais peuvent également être plus complexes à mettre en œuvre.

Nous pouvons à ce stade augmenter la classe location vue précédemment avec les fonction de changement de repère.

```
class Location {
```

```
  static Deg2Rad = Math.PI/180.0 ;
  …
  toECEF (ref, result, isDeg) {
    const lambda = isDeg ? this.lat * Location.Deg2Rad : this.lat;
    const phi = isDeg ? this.lon * Location.Deg2Rad : this.lon;
    const sin_lambda = Math.sin(lambda);
    var cos_lambda = Math.cos(lambda);
    var cos_phi = Math.cos(phi);
    var sin_phi = Math.sin(phi);
    const N = ref._a / Math.sqrt(1 - ref._ee * sin_lambda * sin_lambda);
    var tmp = (this.alt + N) * cos_lambda;
    result.x = tmp * cos_phi;
    result.y = tmp * sin_phi;
    result.z = (this.alt + ref._p1mee * N) * sin_lambda;
  }
}
```

Résumé

Dans ce paragraphe nous avons vus le passage de coordonnés géodésique vers des coordonnées cartésiennes. Ce passage est obligatoire pour pouvoir créer un modèle géométrique utilisable par les moteur 3D. Il est important de constater a ce stade que les coordonnées obtenues ont un ordre de grandeur lié à R, qui est grosso-modo la longueur de l'axe majeur de l'ellipsoïde (IE: le rayon de la terre). Hors, nous voulons ici travailler sur la surface de l'ellipsoïde pour y représenter les terrains. Il vient donc rapidement à l'esprit que repérer les points de surface par rapport au centre de l'ellipsoïde vas engendrer rapidement un problème de précision numérique.

Une des solutions consiste à relocaliser le repère au plus proche de son utilisation afin de conserver un maximum de précision pour les valeurs de la scène 3D utiles, c'est-à-dire visible. C'est le principe des systèmes de coordonnée de type ENU ou NED.

Les système ENU et NED

Résumé

Grille de référence

La grille de référence 3D (je rappelle que nous ne traitons PAS ici les projections) est la grille géodésique. Elle est souvent représentée par des lignes verticales pour les latitudes et horizontale pour les longitudes.

Le système de référence choisi sera donc X pour les longitude, Y pour les latitude et Z pour l'altitude.

Nous verrons dans la suite de cet ouvrage que cette grille simple sert de référence a de nombreuse notions de cartographie web, avec notamment la notion de tuile.

Par la suite nous utiliserons les dimensions de la grille comme puissance de 2 (une vielle habitude qui à moins de sens aujourd'hui avec les GPU modernes) mais qui garde l'avantage de réaliser des subdivisions simples, notamment via les niveaux de détails. Ce sont d'ailleurs ces puissances de 2 que l'on retrouve dans les système de tuile cartographique, donc il me semble nécessaire de coller avec l'existant pour maximiser notre accès à de nombreuses sources de données.

Note : Il est IMPORTANT de considérer la représentation sous forme de sphère 3D de ces coordonnées, comme résultat du passage des coordonnées Géodésique vers ECEF. Sachant que le passage ECEF vers un plans tangentiel local (ENU ou NED) n'est que la concaténation d'une rotation et d'une translation, il est relativement simple de concevoir de travailler sur une grille « à plat » et de la transformer par la suite pour lui donner l'apparence d'une sphère. Ce travail sur une grille régulière nous simplifies le travail et permet d'envisager des solutions d'optimisation topologique pour le rendu avec notamment la prise en compte de niveau de détail fonction de l'éloignement de l'observateur. Certains auteurs ont déjà proposé de multiples solutions autour de cette problématique.

Ainsi, les coordonnées initiale de nos sommets ne seront PAS des coordonnées cartésiennes mais bien des coordonnées géodésique , qui seront transformé par le CPU ou le GPU en coordonnées cartésiennes.

$$(lat, lon, alt) * \begin{pmatrix} a & \cdots & b \\ \vdots & \ddots & \vdots \\ c & \cdots & d \end{pmatrix} = (x, y, z)$$

Les coordonnées cartésiennes finales sont les coordonnées utilisées par le GPU dans ses calculs de projection.

Topologie

Soit une grille de référence de dimension W et H, qui correspondent respectivement au nombre de lignes et au nombre de colonnes de coordonnées.

Pour W = H = 5

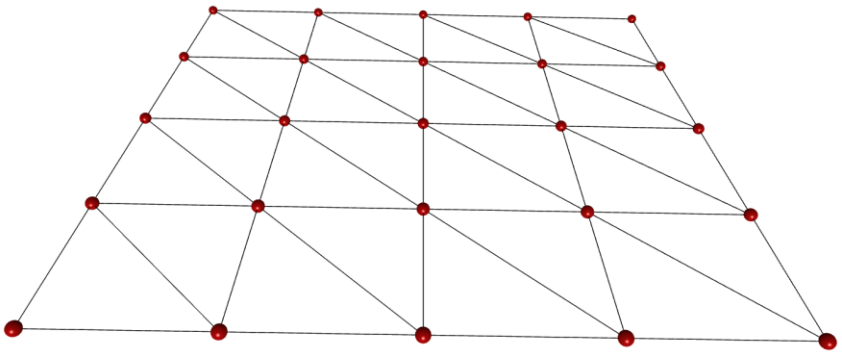


Figure 1: topologie simple

Nous avons W * H sommets et (W-1) * (H-1) * 3 indices. Chaque triplet d'indices définis une face triangulaire.

Note : nous choisissons ici de ne PAS dupliquer les sommets, comme le font de nombreux outils 3D. Cette topologie optimisée, peut être incompatible avec des libraires ou exemple d'algorithmes manipulant les modèles géométriques 3D. Nous en verrons un exemple lorsque nous évoquerons les coordonnées Barycentriques.

L'algorithme pour générer cette topologie avec Babylon JS est le suivant :

```
function createNormalizedGridData(w, h, star){
   const sx = 1;
   const sy = 1;
   const vertexData = new BABYLON.VertexData();
   const positions = [];
   const indices = [];
   const uvs = [];

   // we decided to center the grid.
   const x0 = -.5;
   const y0 = .5;
   const dx = (1. / (w-1));
   const dy = (1. / (h-1));

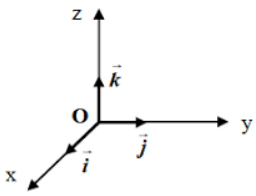
   // positions and uvs. Note the uvs origin upper left.
   for(let row= 0; row < h; row++) {
      let v = row * dy  ;
      const y = (y0 - v) * sy ;
      for(let column=0; column < w; column++) {
         const u = column * dx;
         const x = (x0 + u) * sx ;
         positions.push(x, y, 0);
         uvs.push(u, 1-v);
      }
   }
```

```
  // indices
  const isWEven = (w%2 == 0);
  for (let row = 0; row < h-1; row++) {
     const rowOffset = isWEven ? row%2 : 0;
     for (let col = 0; col < w-1; col++) {
        const idx1 = col + row * w ;
        const idx2 = idx1 + w ;
        const idx3 = idx2 + 1 ;
        const idx4 = idx1 + 1 ;
        if(star){
          if(idx1%2 != rowOffset){
             indices.push(idx1, idx2, idx4, idx2, idx3, idx4);
          } else {
             indices.push(idx1, idx2, idx3, idx3, idx4, idx1);
          }
          continue;
        }
        indices.push(idx1, idx2, idx3, idx3, idx4, idx1);
     }
  }
  // populate the data
  vertexData.indices = indices;
  vertexData.positions = positions;
  vertexData.uvs = uvs;

  return vertexData;
}
```

Vous pouvez trouver le code source complet en ligne sur https://www.babylonjs-playground.com/#RZ8TSC#4

ou bien sur le git de l'ouvrage.



- La grille est générée dans un repère cartésien direct.
- Nous avons choisis de générer une grille normalisée, c'est-à-dire que les valeurs des abscisses et ordonnées sont toutes comprises dans l'intervalle [0,1]. L'énorme avantage de cette approche est que la grille peut être considérée comme un modèle sans unité, pour être ensuite manipulé avec des coordonnées géodésiques ou cartésienne ECEF ou autre.
- z est initié à 0.
- Nous ne nous occupons pas des normales à ce stade.

Il est intéressant de noter que nous avons généré une grille avec une topologie simple, ou les faces sont ordonnées de manière identique et répétitive. Une autre possibilité est de générer cette grille en étoile, c'est-à-dire que les diagonales successives sont orienté différemment.

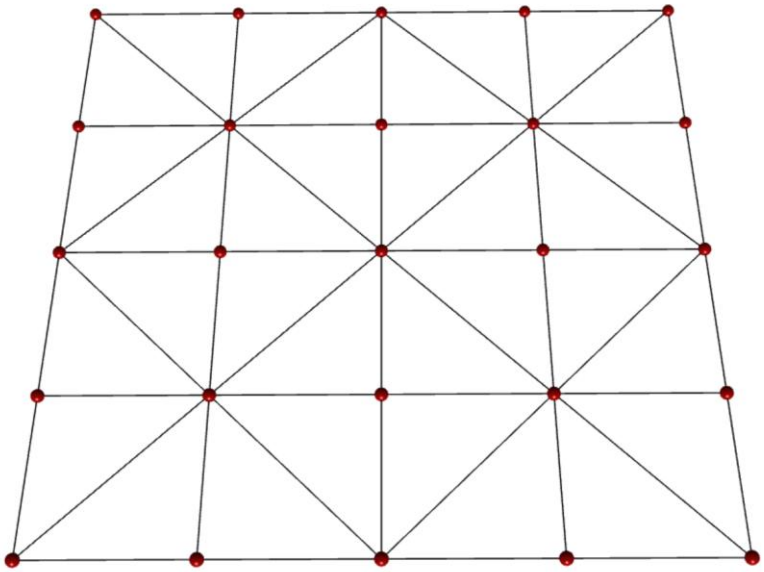


Figure 2: topologie de style étoile

Dans la suite de cet ouvrage nous préférerons utiliser cette topologie en étoile qui offre en plus d'un aspect visuel plus intéressant, quelques avantages sur le plans algorithmique.

Transformations

Une fois cette grille obtenue, nous pouvons travailler sur les coordonnées normalisées afin de les transformer en coordonnées géodésiques.

Pour ce faire nous définissons un centre géographique, deux arcs et une altitude (par default 0).

La formule pour transformer les points de notre grille est alors relativement simple

$$x = Center_x + Position_x * Arc_x$$
$$y = Center_y + Position_y * Arc_y$$
$$z = Altitude$$

Cela pourrait se traduire par la matrice de transformation suivante

$$T_{geo} = \begin{pmatrix} Arc_x & 0 & 0 & 0 \\ 0 & Arc_y & 0 & 0 \\ 0 & 0 & 1 & 0 \\ Center_x & Center_y & Altitude & 1 \end{pmatrix}$$

Les opérations de transformations peuvent être effectuées à la fois au niveau du CPU (unité centrale de traitement) et au niveau du GPU (unité de traitement graphique).

Le CPU est l'unité de traitement principale d'un ordinateur, responsable de l'exécution de la plupart des tâches, telles que le traitement des données, l'exécution des programmes et des opérations mathématiques. Il

est conçu pour gérer des tâches complexes, mais cela peut entraîner des goulots d'étranglement lorsqu'il s'agit de tâches intensives en termes de calculs, telles que les opérations de transformation.

Le GPU, quant à lui, est conçu pour gérer les opérations graphiques et les calculs parallèles. Les GPU modernes disposent d'un grand nombre de cœurs de traitement, ce qui leur permet de traiter rapidement des tâches complexes telles que les opérations de transformation.

Lorsqu'il s'agit de traiter de grandes quantités de données ou de réaliser des tâches qui nécessitent beaucoup de calculs, il peut être plus efficace d'utiliser le GPU plutôt que le CPU. Cela est particulièrement vrai pour les applications telles que la modélisation 3D.

Cependant, il est important de noter que les opérations de transformation peuvent également être effectuées sur le CPU avec une bonne efficacité. En fait, pour les tâches de transformation plus simples, le CPU peut être suffisant. Cependant, pour les tâches plus complexes et pour traiter de grandes quantités de données, l'utilisation du GPU peut offrir des avantages significatifs en termes de temps de traitement et de performances.

En somme, l'objectif final est de minimiser l'utilisation du CPU pour les tâches intensives en calculs et de transférer ces tâches vers le GPU pour une exécution plus rapide et plus efficace. Minimiser le nombre de transfert (« draw call ») est également un point majeur d'optimisation.

CPU

Le code Babylon JS est le suivant :

```
function transformToGeodetic(mesh, center, arc, isDeg ) {
   const positions = mesh.getVerticesData(BABYLON.VertexBuffer.PositionKind);
   const newPos = [];
   center = isDeg ? center.scale(Deg2Rad) : center ;
   arc = isDeg ? arc.scale(Deg2Rad) : arc ;
   for(let i=0; i != positions.length; i+=3) {
      const x = positions[i] * arc.x + center.x;
      const y = positions[i+1] * arc.y + center.y;
      const z = positions[i+2] + center.z;
      newPos.push(x,y,z) ;
   }
   mesh.updateVerticesData(BABYLON.VertexBuffer.PositionKind, newPos);
}
```

Avec la constante

```
const Deg2Rad = Math.PI/180.0 ;
```

Ensuite nous pouvons appliquer à ces mêmes coordonnées les transformations vues précédemment soit Geodetic to ECEF ou Geodetic to ENU. Il est intéressant de générer les normales à ce stade, car elle sont simplement l'équivalent de la position normalisée. A noter que selon le Framework utilisé, les normales pourront être calculées automatiquement par faces. Ici nous avons une normale par sommets, qui sera donc naturellement interpolé sur l'ensemble des faces adjacentes.

Le code Babylon JS dans le cas d'une sphère parfaite est le suivant :

```
function transformGeodeticToEcef(mesh, ellipsoid) {
   const positions = mesh.getVerticesData(BABYLON.VertexBuffer.PositionKind);
   const newPos = [];
   const normals = [];

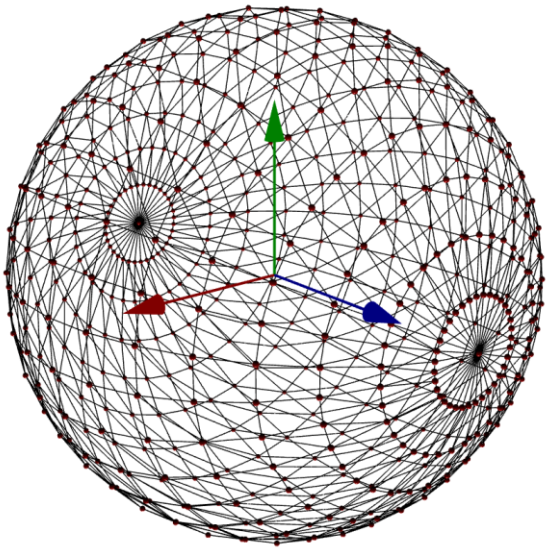
   const geodetic = new Location();
   const cache = new BABYLON.Vector4();
   for(let i=0; i != positions.length; i+=3) {
      geodetic.lon = positions[i];
      geodetic.lat = positions[i+1];
      geodetic.alt = positions[i+2];
      geodetic.toECEFToRef(ellipsoid, cache);
      newPos.push(cache.x,cache.y,cache.z) ;
      const n = cache.normalize();
      normals.push(n.x, n.y, n.z);
   }
   mesh.updateVerticesData(BABYLON.VertexBuffer.PositionKind, newPos);
   mesh.setVerticesData(BABYLON.VertexBuffer.NormalKind, normals);
}
```

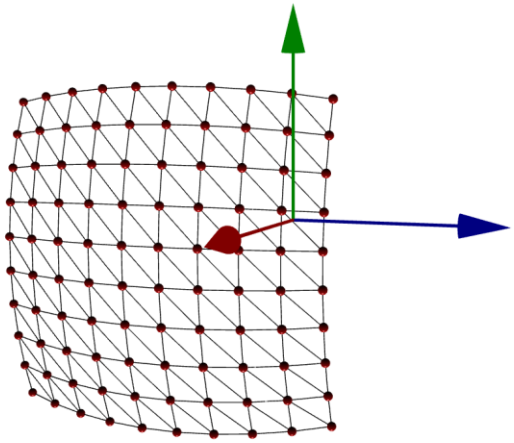
En appliquant consécutivement les deux transformations, avec un centre à (0,0,0) et des arcs respectifs de 45 degrés

```
   const ellipsoidOfRef = Ellipsoid.WGS84.clone("custom", 1/100000);
   …
   const data = createNormalizedGridData(w, h);
   const grid = new BABYLON.Mesh("grid", scene);
   data.applyToMesh(grid, true);
   transformToGeodetic(grid, BABYLON.Vector3.Zero(), new BABYLON.Vector3( 45 , 45 , 0), true);
   transformGeodeticToEcef_sphere(grid, ellipsoidOfRef, false);
```

En changeant W et H, ainsi que les arcs en (360, 180) on obtiens une ellipsoïde. Vous pouvez jouer avec le paramètre « inverse flattening » pour constater que la sphère s'aplatie aux pôles.

```
   const ellipsoidOfRef = Ellipsoid.FromAAndInverseF("custom”10, 5)
```





Mais que ce passe t'il lorsque nous définissons le centre de notre grille sur un pôle ? Nous avons vus précédemment que la logique associée aux enveloppes et particulière et que les règle basique de géométries sont inapplicables.

Dans ce cas, la topologie de la grille de référence change et représente alors une calotte où les coordonnées de la grille sont constitué en cercle concentrique de latitude donnée. La fonction de création de la grille géodésique est alors à réécrire.

Nous avons donc deux manières de représenter la grille géodésique qui sont incompatibles entres elles (mais elles peuvent cohabiter). Ce point est primordiale car il est dimensionnant sur le choix des algorithmes possibles de navigation sur notre surface.

Bien évidemment, ces extraits de codes n'ont d'intérêt que pour la démonstration. Ils se trouvent sur https://www.babylonjs-playground.com/#MMZ3GP#9

Dans l'exemple précèdent , nous avons démontré qu'il était possible de décrire toute ou partie d'une ellipsoïde avec une grille de référence. Il est tout aussi simple de diviser cette ellipsoïde en une multitude de grille, de topologie identiques.

Le code suivant démontre l'utilisation d'un ensemble de grille.

- Taille des grille géodésique 45° x 45°
- Subdivision des grilles 64 x 64 soit 65 x 65 sommets (recouvrement des sommets adjacents entre les tuiles)
- 8 grilles selon les longitude (360°)
- 4 grilles selon les latitude (180°)

Tout d'abords nous définissons une fonction de création de grilles qui reprend les codes précédents

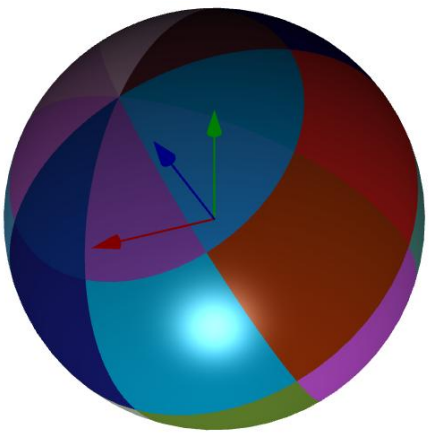
```
function buildTile(name, w, h, center, arc, radius, color, scene, isDeg){
   const data = createNormalizedGridData(w, h);
   const grid = new BABYLON.Mesh(name, scene);
   data.applyToMesh(grid, true);
   transformToGeodetic(grid, center, arc, isDeg);
   transformGeodeticToEcef_sphere(grid, radius);
   grid.material = new BABYLON.StandardMaterial(name + ".standard", scene);
   grid.material.diffuseColor = color;
}
```

Ensuite nous utilisons cette fonction au sein de boucles imbriqués pour générer chaque grille.

```
  ...
   const tileSize = 64;
   const w = tileSize + 1;
   const h = tileSize + 1;
   const d = Math.PI / 4;
   const d_2 = Math.PI / 8;
   const size = new BABYLON.Vector3(d, d, 0);

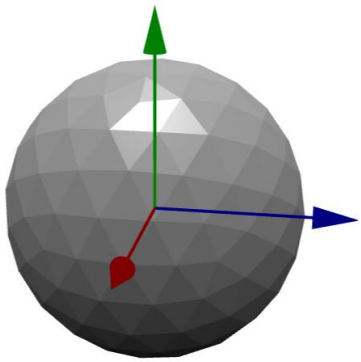
   // create the tiles
   for (var j = 0; j != 4; j++) {
      lat = Math.PI / 2 - d_2 - d * j;
      for (var i = 0; i != 8; i++) {
         lon = d * i;
         const name = "tile_" + lat + "_" + lon;
         buildTile(name, w, h, new BABYLON.Vector3(lon, lat, 0), size, RADIUS, BABYLON.Color3.Random(),
scene);
      }
   }  ...
```

L'ellipsoïde ainsi générée situe ses pôles sur l'axe Z (axe en bleue sur la figure), et je rappelle que pour OpengGL, l'axe vertical est Y (axe en vert sur la figure). Donc ne vous formaliser pas sur ce positionnement pour l'instant, nous y ferons attention lorsque nous placerons nos objets dans une scène. N'oubliez pas que différents repères existent pour différents logiciels, donc nous travaillerons ici avec un repère mathématique cartésien direct et nous pourrons ainsi avoir du code portable sur l'ensemble des plateformes, moyennant une petite transformation avant l'affichage.



Vous pouvez retrouver le code source complet en https://www.babylonjs-playground.com/#MMZ3GP#12

Une solution alternative est de représenter l'ellipsoïde à l'aide d'une « Ico-sphère ». Mais cela rend l'accès aux nombreuses sources de données beaucoup plus complexe, car le mécanisme de création est initié en 3D à l'aide des solides de Platon. Ainsi les sources de donnée existantes ne sont pas directement compatible. En revanche cette solution à l'avantage d'être homogène sur l'ensemble de l'ellipsoïde et ne présente pas de particularités aux pôles. J'ai réalisé beaucoup de simulation avec ce modèle d'ico-sphère et bien que cette démarche soit intellectuellement satisfaisante, la complexité sous-jacente n'est pas compatible avec la lisibilité et la maintenabilité du code.



On sacrifiera les pôles pour l'instant, sachant que les données de terrains à notre disposition (libre de droit ou accessibles) ne couvrent pas les pôles pour la grande majorité d'entre elles. En revanche une solution simple existe et nous la mettrons en œuvre afin d'apporter plus de qualité sur la representation des terrains aux pôles géographiques. Nous pensons en particulier au prochains travaux sur la lune qui se situerons en majorité près du pôle sud lunaire autour du cratère Shakleton (du nom d'Ernest Shakleton, explorateur britannique considéré comme l'une des principales figures de l'âge héroïque de l'exploration en Antarctique, fin 19ème debut 20ème siècle).

https://www.nasa.gov/sites/default/files/atoms/files/a_sustained_lunar_presence_nspc_report4220final.pdf

GPU

Ces mêmes transformations peuvent être réalisées au niveau du GPU. L'avantage est qu'alors, nous pouvons en utilisant les instances openGL, limiter le nombre d'appel (CPU vers GPU) pour dessiner une sphère complète avec ses tuiles.

Cela nécessite d'introduire la notion de shader OpenGL, et WebGL en particulier pour cet ouvrage.

GLSL

Les shaders WebGL 2.0 sont des programmes écrits en langage GLSL (OpenGL Shading Language) qui sont exécutés sur le processeur graphique (GPU) de l'ordinateur, et qui permettent de manipuler les données graphiques envoyées à la carte graphique pour afficher des images sur un navigateur web.

Le pipeline graphique en WebGL 2.0 se compose de plusieurs étapes. Tout d'abord, les données graphiques sont envoyées du processeur (CPU) vers la carte graphique, où elles sont stockées dans des tampons de mémoire appelés "buffers". Ensuite, le Vertex Shader est exécuté pour chaque vertex (sommet) de l'objet 3D. Le Vertex Shader permet de transformer la position des sommets dans l'espace 3D en position 2D sur l'écran, ainsi que de calculer d'autres données telles que la couleur et la texture.

Une fois que tous les sommets ont été transformés, le pipeline graphique passe à l'étape suivante : le Fragment Shader. Le Fragment Shader est exécuté pour chaque pixel de l'objet 3D. Le Fragment Shader permet de calculer la couleur finale de chaque pixel en fonction de diverses informations, telles que les coordonnées de texture et l'éclairage. La couleur finale est ensuite stockée dans un tampon appelé "framebuffer".

Enfin, le framebuffer est envoyé à l'écran pour afficher l'image finale à l'utilisateur.

Les Vertex Shader et Fragment Shader sont donc deux programmes clés du pipeline graphique en WebGL 2.0.

Voici un exemple basique de vertex shader en GLSL, qui déplace les sommets d'un objet de 0,5 unité sur l'axe des x :

```
attribute vec3 position;
void main() {
  gl_Position = vec4(position.x + 0.5, position.y, position.z, 1.0);
}
```

Ce shader prend en entrée un attribut "position", qui représente la position de chaque sommet de l'objet. Le shader utilise ensuite la fonction "vec4" pour définir la nouvelle position de chaque sommet en ajoutant 0,5 à la coordonnée x. La fonction "gl_Position" est une variable prédéfinie en GLSL qui définit la position finale de chaque sommet dans l'espace de projection.

Et voici un exemple basique de fragment shader en GLSL, qui définit la couleur de chaque pixel d'un objet en rouge :

```
void main() {
  gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0);
}
```

Ce shader est très simple : il ne prend pas d'entrée et définit simplement la couleur de chaque pixel en rouge. La fonction "gl_FragColor" est une variable prédéfinie en GLSL qui définit la couleur finale de chaque pixel.

Ces deux exemples basiques montrent comment les shaders peuvent être utilisés pour modifier l'apparence d'un objet 3D. Heureusement, les shaders peuvent être beaucoup plus complexes, avec des calculs avancés pour créer des effets de lumière, des ombres, des textures, des effets de distorsion, etc.

Paramètres

En GLSL, les attributs (ou "attributes" en anglais) sont des données d'entrée fournies à un shader pour décrire les propriétés géométriques d'un objet 3D. Les attributs sont utilisés pour définir les positions, les couleurs, les normales et autres propriétés des sommets d'un objet.

Voici quelques types d'attributs couramment utilisés en GLSL :

- "position" : Cet attribut est utilisé pour spécifier la position de chaque sommet d'un objet 3D dans l'espace 3D. La position est généralement définie comme un vecteur à 3 dimensions (x, y, z).
- "normal" : Cet attribut est utilisé pour spécifier la direction de chaque surface d'un objet 3D. Les normales sont généralement définies comme des vecteurs à 3 dimensions (x, y, z).
- "color" : Cet attribut est utilisé pour spécifier la couleur de chaque sommet d'un objet 3D. Les couleurs sont généralement définies comme des vecteurs à 3 ou 4 dimensions (rouge, vert, bleu, alpha).
- "uv" : Cet attribut est utilisé pour spécifier les coordonnées de texture pour chaque sommet d'un objet 3D. Les coordonnées de texture sont généralement définies comme des vecteurs à 2 dimensions (u, v).
- "tangent" et "bitangent" : Ces attributs sont utilisés pour spécifier la direction de la courbure d'une surface d'un objet 3D. Les tangentes et bitangentes sont généralement définies comme des vecteurs à 3 dimensions (x, y, z).
- "instance" : Cet attribut est utilisé pour spécifier des données d'instance pour chaque instance d'un même objet 3D. Les données d'instance sont généralement définies comme des vecteurs à 1 ou plusieurs dimensions.

Ces attributs sont souvent utilisés en combinaison avec des uniforms, qui sont des variables globales accessibles à tous les shaders de la scène. Les uniforms peuvent être utilisés pour définir des paramètres globaux tels que les matrices de transformation, les couleurs de la lumière, les textures, etc.

Structure

En GLSL, les structures sont des types de données personnalisés qui peuvent être utilisés pour organiser et regrouper plusieurs variables de différents types en une seule entité. Les structures sont utiles pour organiser des données connexes et pour améliorer la lisibilité et la maintenabilité du code.

Voici un exemple de déclaration de structure en GLSL :

```
struct Material {
  vec3 diffuseColor;
  vec3 specularColor;
  float shininess;
};
```

Dans cet exemple, la structure "Material" contient trois variables: "diffuseColor" et "specularColor" qui sont des vecteurs à 3 dimensions contenant les couleurs diffuse et spéculaire du matériau, et "shininess" qui est un scalaire représentant le degré de brillance du matériau.

Pour utiliser la structure définie ci-dessus dans un shader, vous pouvez créer une variable de type Material et y accéder en utilisant la notation pointée, par exemple :

```
Material myMaterial;
myMaterial.diffuseColor = vec3(1.0, 0.0, 0.0);
myMaterial.specularColor = vec3(0.5, 0.5, 0.5);
myMaterial.shininess = 32.0;
```

Dans cet exemple, nous avons créé une variable "myMaterial" de type Material et nous avons assigné des valeurs à ses champs "diffuseColor", "specularColor" et "shininess". Ces valeurs peuvent ensuite être utilisées dans le shader pour calculer la couleur finale de l'objet.

l est tout à fait possible de passer des structures en tant qu'uniforms en GLS

```
struct Material {
  vec3 diffuseColor;
  vec3 specularColor;
  float shininess;
};

uniform Material myMaterial;
```

Pour assigner une valeur à cet uniform dans le code JavaScript, nous pouvons utiliser les méthodes fournies par la bibliothèque WebGL pour accéder à l'uniform à l'intérieur du shader. Par exemple

```
const myMaterial = {
  diffuseColor: new BABYLON.Vector3(1, 0, 0),
  specularColor: new BABYLON.Vector3(0.5, 0.5, 0.5),
  shininess: 32.0
};

myShader.setUniform("myMaterial.diffuseColor", myMaterial.diffuseColor);
myShader.setUniform("myMaterial.specularColor", myMaterial.specularColor);
myShader.setUniform("myMaterial.shininess", myMaterial.shininess);
```

Dans cet exemple, nous avons défini une variable "myMaterial" contenant les valeurs des champs de la structure "Material", et nous avons passé ces valeurs à l'uniform "myMaterial" en utilisant la méthode "setUniform" de l'objet shader.

En utilisant des structures en tant qu'uniforms, nous pouvons donc facilement passer des ensembles de données complexes à nos shaders, ce qui nous sera particulièrement utile.

Shader Geodesique

Fort de ces informations, nous allons nous attacher dans les prochains paragraphe à introduire nos propres shaders afin de réaliser nos calculs de positionnement dans le GPU.

Nous rappelons que nous avons en entrées

- une grille normalisée,
- une ellipsoïde de référence.
- un vecteur d'arc.
- un vecteur de centre.

La grille et l'ellipsoïde de référence seront des uniforms, alors que les deux autres seront des attributs.

Tous d'abords nous introduisons un structure de type Ellipsoide

```
 // ellipsoid
  struct Ellipsoid {
     float _a;
     float _ee;
     float _p1mee;
  };
```

Puis une fonction de transformation afin de rendre le code modulable. Dans la version finale, ces types et fonction feront partis d'un module indépendant qui pourra être inclus dans le shader. Cette fonction de transformation est identique a celle portée par la classe Location, dont les formules mathématiques ont étés exposées dans les chapitres précédent.

```
vec4 toECEF(Ellipsoid ref, vec3 geo)
{
   float sin_lambda = sin(geo.y);
   float cos_lambda = cos(geo.y);
   float cos_phi = cos(geo.x);
   float sin_phi = sin(geo.x);
   float N = ref._a / sqrt(1.0 - ref._ee * sin_lambda * sin_lambda);
   float tmp = (geo.z + N) * cos_lambda;
   float x = tmp * cos_phi;
   float y = tmp * sin_phi;
   float z = (geo.z + ref._p1mee * N) * sin_lambda;
   return vec4(x,z,y,1.0);
}
```

En declarant la structure d'ellipsoide en tant qu'uniform, le code du vertex shader devient

```
// uniforms
uniform mat4 viewProjection;
uniform Ellipsoid ellipsoid;

// Varying
varying vec2 vUV;

void main(void) {
   // this line is specific to Babylonjs to include the logic and parameters behind instances
   #include<instancesVertex>

   float lat = center.y + position.y * size.y;
   float lon = center.x + position.x * size.x;
   float alt = 0.0;

   gl_Position = viewProjection * finalWorld * toECEF(ellipsoid,vec3(lon,lat,alt));

   float u = .5  + lon * oneOver2PI ;
   float v = .5  + lat * oneOverPI ;

   vUV = vec2(u, v);
}
```

Il est intéressant de noter à ce stade que le vecteur U,V est également calculé à la volée, et il n'est donc pas nécessaire de le traiter au niveau CPU , ni le transférer vers le GPU.

En utilisant une texture adaptée, le code du fragment shader devient

```
precision highp float;

varying vec2 vUV;
uniform sampler2D textureSampler;
```

```
  void main(void) {
    gl_FragColor = texture2D(textureSampler, vUV);
  }
```

L'appel depuis javascript dans le formalisme du Babylon sera donc

```
function createShaderMaterial(name, ellipsoid, scene){

  var shaderMaterial = new BABYLON.ShaderMaterial("shader", scene, {
    vertex: "custom",
    fragment: "custom",
    },
    {
      attributes: ["position", "center", "size"],
      uniforms: ["world", "worldView", "viewProjection", "view", "projection", "ellipsoid"]
    });
  var mainTexture = new BABYLON.Texture("textures/earth.jpg", scene);
  shaderMaterial.setTexture("textureSampler", mainTexture);
  shaderMaterial.setFloat("ellipsoid._a",ellipsoid._a);
  shaderMaterial.setFloat("ellipsoid._ee",ellipsoid._ee);
  shaderMaterial.setFloat("ellipsoid._p1mee",ellipsoid._p1mee);
  return shaderMaterial;
}
```

En utilisant les données suivantes, nous pouvons alors facilement représenter une maquette à l'échelle du soleil et des 8 planètes, plus la planète naine Pluton.

Astre	Diamètre à l'équateur (km)
Soleil	1391900
Mercure	4880
Vénus	12104
Terre	12756
Mars	6805
Jupiter	142984
Saturne	120536
Uranus	51312
Neptune	49922
Pluton	2300

Pluton a été déclassée comme planète en 2006 lorsqu'elle a été reclassée comme une "planète naine" par l'Union astronomique internationale (UAI). Cette décision a été prise car les astronomes ont découvert des corps célestes similaires à Pluton dans la ceinture de Kuiper, une région du système solaire au-delà de Neptune.

L'UAI a décidé que pour qu'un objet soit considéré comme une planète, il doit remplir trois critères :

1. Il doit orbiter autour du Soleil.
2. Il doit avoir une forme sphérique, ce qui signifie qu'il doit être suffisamment grand pour que sa propre gravité l'amène à adopter une forme sphérique.
3. Il doit avoir "nettoyé" son orbite, c'est-à-dire qu'il doit être suffisamment massif pour éliminer tout autre objet gravitant dans son orbite.

Pluton remplit les deux premiers critères, mais elle n'a pas "nettoyé" son orbite car elle partage son orbite avec d'autres objets de la ceinture de Kuiper. Par conséquent, elle a été reclassée comme une "planète naine".

Ce qui est assez flagrant sur la planche graphique suivante.

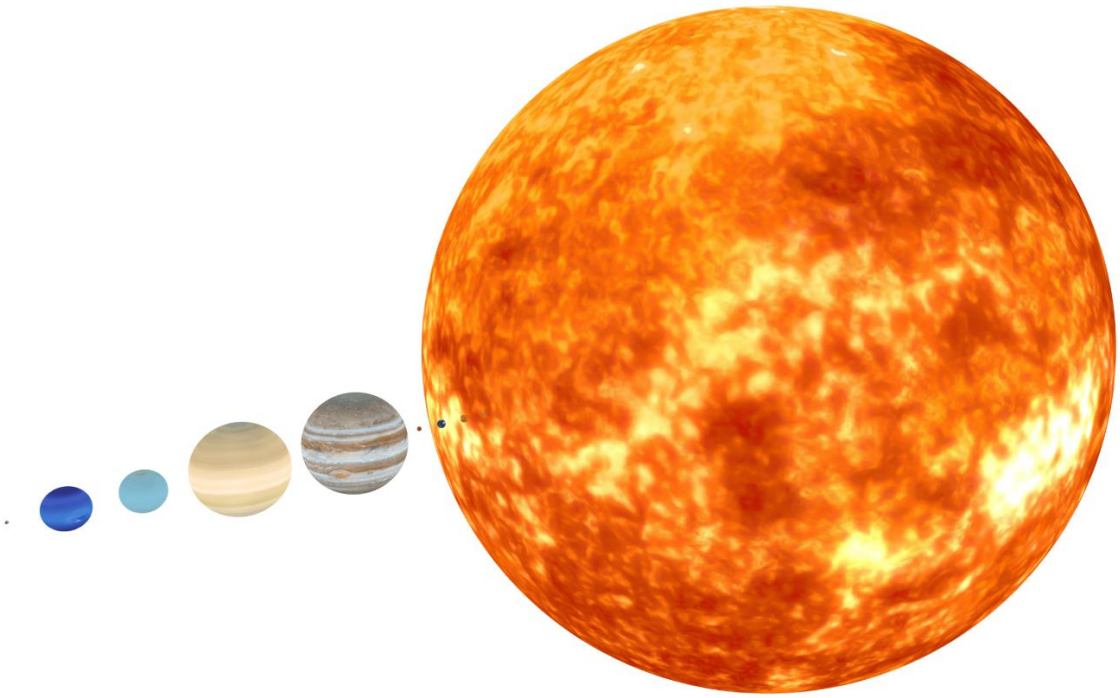
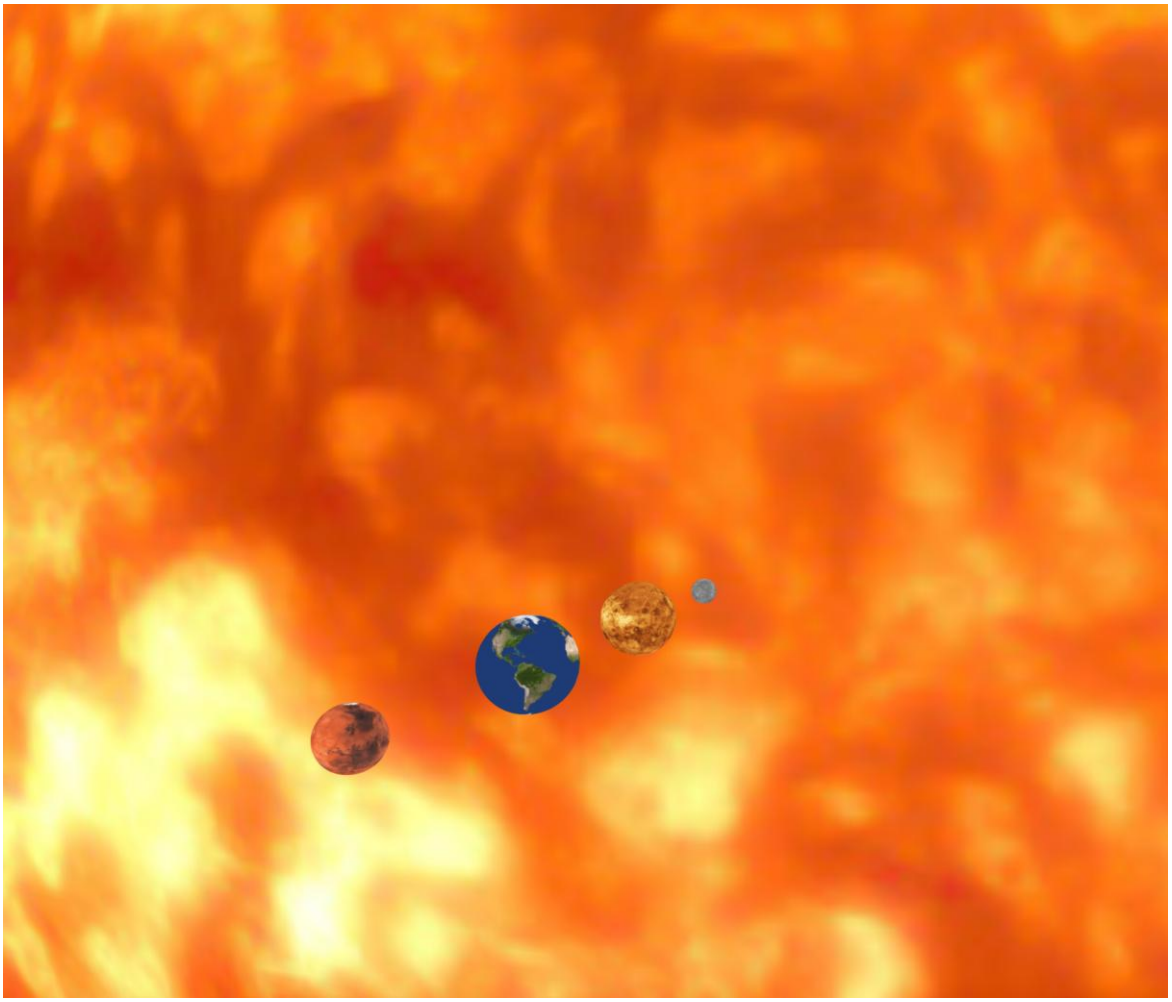


Figure 3: représentation à l'échelle des huit planètes du système solaire, avec pluton.



résumé

Dans ce chapitre, l'IA du Babylon nous a renseignée sur la topologie évoluée de la représentation des ellipsoïdes, et donc des planètes ou des étoiles. Elle a su distinguer de manière didactique, la théorie mathématique présentée de manière simplifié, le code nécessaire dans un langage simple et massivement utilisé pour adresser le processeur central. Et enfin le code nécessaire pour adresser le processeur graphique.

En utilisant des textures adaptées ainsi que quelques caractéristiques des huit planètes du système solaire, nous avons facilement crée une scène à l'échelle les représentants. A ce stade seule les positions des planètes était fantaisistes.

Vous trouverez le code sur le GIT de notre voyage mais également une version simplifiée sur le playground du vaisseau en https://www.babylonjs-playground.com/#KYVFI2#13

Sphere celeste : https://www.babylonjs-playground.com/#E5GE1C#8

https://svs.gsfc.nasa.gov/4851

L’IA nous a ensuite rappelé les calcul de dynamique spatiale vu précédemment et introduit le positionnement correct des corps célestes dans notre système, ainsi que leur animation en fonction du temps.

Dans un prochain rapport nous devons apprendre les techniques d’amélioration du rendu apportés par l’IA afin de présenter des versions holographiques correctes aux navigateurs

Rendu des corps célestes depuis l'espace

A ce stade, notre vaisseau, le Babylon, est positionné afin d'étudier les corps céleste depuis l'espace. Si l'on prend l'exemple de la terre, nous allons regarder la planète, au plus près amarré à la station spatiale ISS ou bien depuis Gateway.

Nous allons faire une pose à cette échelle, et allons travailler l'aspect esthétique et réalisme. A ces fins, nous allons modifier les shader afin d'y inclure les opérations nécessaire.

Les questions que nous devons nous poser sont les suivantes.

- Comment représenter une étoile
- Comment représenter la sphère céleste
- Comment représenter la galaxie
- Comment représenter une nébuleuse
- Comment représenter une planète
    - La lumière ambiante (soleil, réflexion sur les satellites,..)
    - Comment représenter la réflexion de la lumière sur les zone d'eau
    - Pour les planète habitées , comment représenter l'activité lumineuse nocturne.
    - Comment représenter l'atmosphère
        - Couverture nuageuse et ombre portée
        - Diffusion de la lumière.
    - Le relief si il est significatif fonction de la distance.

Tant de questions ou nous trouverons des réponses **approximatives** dans les prochains paragraphes. Je parle d'approximation car nous devons, afin de réaliser ce rendu en temps réel, trouver des « raccourcis » qui soit les moins gourmands en cycle CPU, GPU, et mémoire tout en offrant un réalisme suffisant pour nos besoins.

Rendu atmosphérique

L'atmosphère d'une planète est la couche de gaz qui entoure sa surface. Elle est maintenue en place par la gravité de la planète et peut varier en épaisseur et en composition chimique en fonction de la taille de la planète, de sa distance au soleil et d'autres facteurs environnementaux.

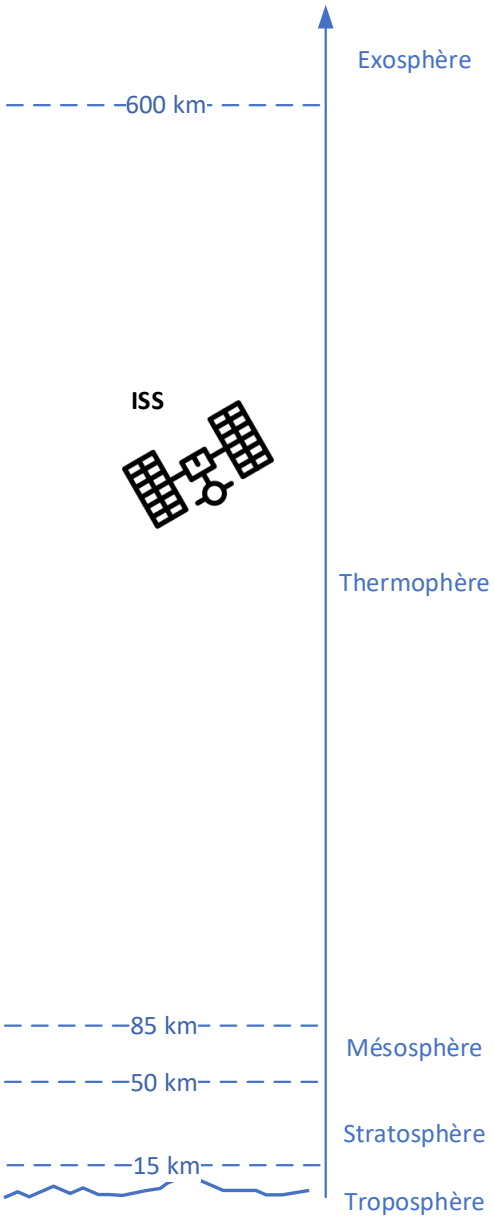
L'aspect extérieur d'une planète est grandement influencé par son atmosphère.

Pour la Terre, l'atmosphère est composée principalement d'azote (78%) et d'oxygène (21%), avec de petites quantités d'autres gaz tels que l'argon, le dioxyde de carbone et le néon. L'atmosphère terrestre contient également de la vapeur d'eau, des aérosols et d'autres polluants.

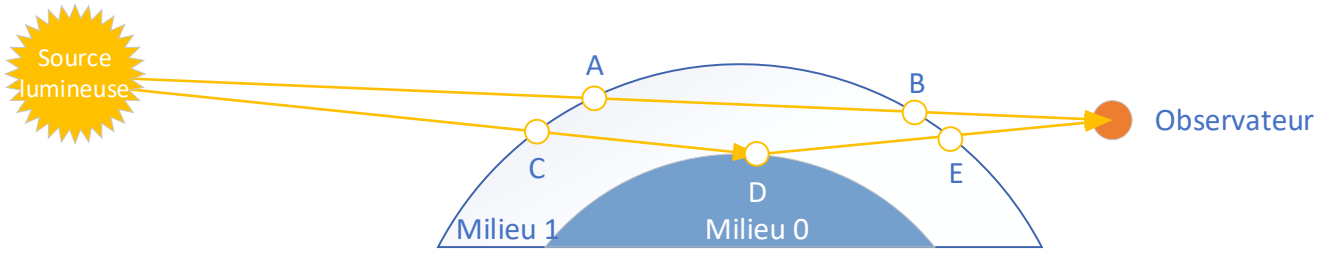
Les compositions chimiques des atmosphères des autres planètes varient considérablement. Par exemple, l'atmosphère de Vénus est principalement composée de dioxyde de carbone (96%), avec des quantités plus faibles de gaz sulfurique et d'azote. L'atmosphère de Mars est principalement composée de dioxyde de carbone (95%), avec de petites quantités d'azote et d'argon. Les atmosphères des géantes gazeuses comme Jupiter et Saturne sont principalement composées d'hydrogène et d'hélium, avec des quantités variables de gaz tels que le méthane et l'ammoniac.

Il y a cinq couches principales de l'atmosphère terrestre, chacune ayant ses propres caractéristiques et propriétés physiques. De bas en haut, elles sont:

1. La troposphère est la couche la plus proche de la surface de la Terre, où la plupart de la météo a lieu. Elle s'étend jusqu'à environ 10 à 15 km d'altitude et contient environ 80% de la masse de l'atmosphère.
2. Juste au-dessus de la troposphère, la stratosphère s'étend jusqu'à environ 50 km d'altitude. C'est ici que se trouve la couche d'ozone, qui absorbe la plupart des rayons ultraviolets nocifs du soleil.
3. La mésosphère, s'étendant de 50 à 85 km d'altitude, est la couche la plus froide de l'atmosphère et abrite de nombreux phénomènes lumineux tels que les étoiles filantes.
4. La thermosphère, s'étendant de 85 à 600 km d'altitude, est une couche très chaude en raison de son exposition directe aux rayons solaires.
5. L'exosphère est la couche la plus haute de l'atmosphère, s'étendant au-delà de 600 km d'altitude. Les gaz de l'exosphère s'amincissent rapidement et se mélangent avec l'espace interplanétaire.



Si l'on considère le modèle **ultra simplifié** suivant : deux milieux distincts éclairés par une source lumineuse, le milieu zéro représentant une planète et le milieu 1 son atmosphère.



La lumière emprunte plusieurs chemins pour arriver à l'observateur. Ce chemin peut être

- soit direct (A,B) à travers le milieu 1
- soit indirect (CDE) via une **réflexion** au point D.

Les points A, B, C et E sont des points ou la lumière passe d'un milieux à un autre et subit alors un phénomène de **réfraction**.

Réflexion

La réflexion lumineuse est le phénomène par lequel la lumière rebondit sur une surface solide ou un milieu transparent et change de direction. La lumière peut être réfléchie de différentes manières, selon la nature de la surface sur laquelle elle frappe.

Il est important de noter que la réflexion lumineuse ne modifie pas la longueur d'onde de la lumière, mais simplement sa direction. C'est pourquoi nous pouvons voir des objets qui nous entourent, car la lumière qui les éclaire est réfléchie dans nos yeux.

Cette réflexion en D peut donc être de différents type. Soit spéculaire, soit diffuse, soit de volume.

Réflexion spéculaire

La réflexion spéculaire, par exemple, se produit lorsque la surface sur laquelle la lumière frappe est lisse et régulière. Dans ce cas, la lumière est réfléchie de manière ordonnée et cohérente, de sorte que les rayons lumineux reflétés suivent une trajectoire précise et parallèle entre eux. Un exemple de réflexion spéculaire est celle que l'on observe sur une surface plane et polie comme un miroir.

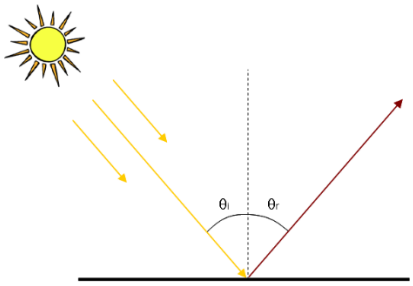


Figure 4: réflexion spéculaire

Réflexion diffuse

En revanche, si la surface est rugueuse ou irrégulière, la réflexion est dite diffuse. Dans ce cas, les rayons lumineux sont réfléchis dans différentes directions, créant un éclairage plus doux et uniforme.

il est important de noter que la direction de la lumière incidente peut influencer l'intensité et la distribution de la réflexion diffuse. Par exemple, si la lumière incidente frappe la surface de manière oblique plutôt que perpendiculaire, la quantité de lumière réfléchie peut être différente et la direction de la réflexion diffuse peut être légèrement biaisée dans la direction de l'incidence de la lumière.

L'indice de luminance d'une réflexion diffuse est une mesure qui quantifie la luminosité ou la brillance d'une surface qui réfléchit la lumière de manière diffuse. L'indice de luminance est utilisé pour décrire l'apparence visuelle de la surface et peut être mesuré à l'aide d'un luxmètre ou d'un photomètre.

L'indice de luminance est généralement exprimé en candelas par mètre carré (cd/m²) et indique la quantité de lumière qui est réfléchie par unité de surface. Plus l'indice de luminance est élevé, plus la surface apparaîtra lumineuse ou brillante.

L'indice de luminance d'une surface dépend de plusieurs facteurs, tels que la couleur de la surface, la quantité et la qualité de la lumière incidente, la texture et la géométrie de la surface. Par exemple, une surface blanche et lisse aura un indice de luminance plus élevé qu'une surface noire et rugueuse.

L'indicatrice de luminance d'une réflexion diffuse est une fonction mathématique qui décrit la relation entre la luminance d'une surface réfléchissante et l'angle d'observation. L'indicatrice de luminance est également connue sous le nom de fonction de réflectance ou de courbe de Lambert.

L'indicatrice de luminance (en pointillé dans le graphique suivant) est une propriété importante des surfaces diffusantes et permet de prédire la distribution de la lumière réfléchie par une surface en fonction de l'angle d'observation. Cette distribution est souvent représentée sous la forme d'un diagramme de polarisation ou d'un graphique en 3D appelé "fonction de réflectance bidirectionnelle" (BRDF).

La courbe de Lambert est souvent utilisée comme référence pour la réflexion diffuse. Elle décrit une réflexion diffuse idéale d'une surface parfaitement mate et isotrope (dont les propriétés sont les mêmes dans toutes les directions). La courbe de Lambert est caractérisée par une distribution uniforme de la lumière réfléchie dans toutes les directions, indépendamment de l'angle d'observation.

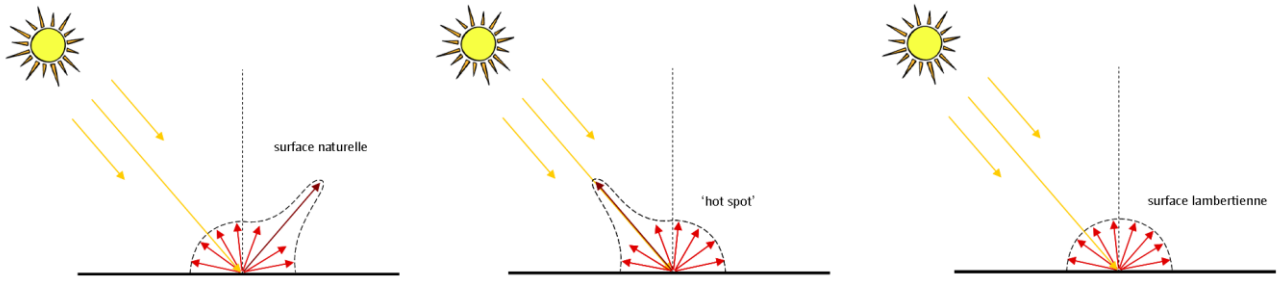


Figure 5: réflexion diffuse

En pratique, la réflexion diffuse réelle peut différer de la courbe de Lambert en raison de la texture et de la rugosité de la surface, de la longueur d'onde de la lumière incidente et de la nature du matériau réfléchissant. Cependant, l'indicatrice de luminance est un outil utile pour comprendre et modéliser la réflexion diffuse des surfaces

Un exemple courant de réflexion diffuse est celle de la lumière du soleil qui frappe la surface terrestre. Lorsque la lumière du soleil frappe le sol, elle est réfléchie dans toutes les directions, créant ainsi une lumière diffuse qui éclaire uniformément l'environnement.

Réflexion de volume

La réflexion de volume est un type de réflexion qui se produit à l'intérieur d'un matériau transparent et homogène, tel que le verre, l'eau ou le cristal. Contrairement à la réflexion de surface, qui se produit à l'interface entre deux milieux, la réflexion de volume se produit à l'intérieur du matériau lorsque la lumière pénètre dans celui-ci. Lorsque la lumière traverse un matériau transparent, une partie de celle-ci est réfléchie à l'interface entre le milieu et l'air (ou un autre milieu). Cette réflexion est appelée réflexion de surface. Cependant, une partie de la lumière continue de se propager à l'intérieur du matériau et peut être réfléchie plusieurs fois à mesure qu'elle rencontre différentes interfaces à l'intérieur du matériau. Cette réflexion multiple est appelée réflexion de volume.

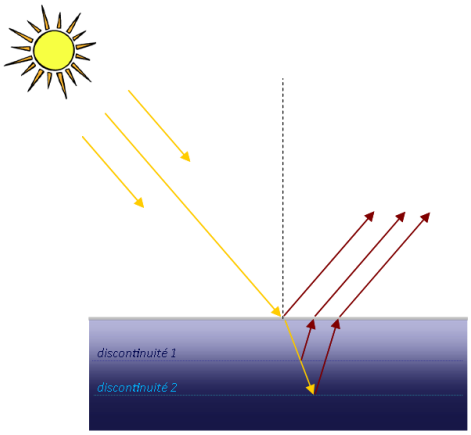


Figure 6: réflexion de volume

La réflexion de volume peut également se produire dans l'atmosphère, en particulier dans les couches supérieures de l'atmosphère. Cette réflexion est due à la propagation de la lumière à travers des couches d'air ayant des indices de réfraction différents.

Comme vus précédemment, l'atmosphère contient des couches de gaz ayant des densités et des indices de réfraction différents. Ces différences de densité et d'indice de réfraction peuvent conduire à la réfraction et la réflexion de la lumière dans l'atmosphère. Ce processus peut se produire plusieurs fois.

Réfraction

La réfraction est le phénomène lumineux au cours duquel la lumière dévie de sa trajectoire rectiligne en changeant de vitesse lorsqu'elle passe d'un milieu transparent à un autre. Elle se produit aux points A, B C et E de notre modèle précédent.

L'exemple le plus courant de réfraction atmosphérique qui permet de voir le Soleil après son coucher est appelé le phénomène de la lumière crépusculaire.

La lumière crépusculaire est un phénomène optique qui se produit lorsque la lumière du Soleil traverse l'atmosphère terrestre. L'atmosphère terrestre agit comme une lentille géante qui réfracte la lumière, courbant les rayons lumineux autour de la surface de la Terre. Cette réfraction fait apparaître le Soleil comme s'il était encore au-dessus de l'horizon, même s'il est déjà en dessous.

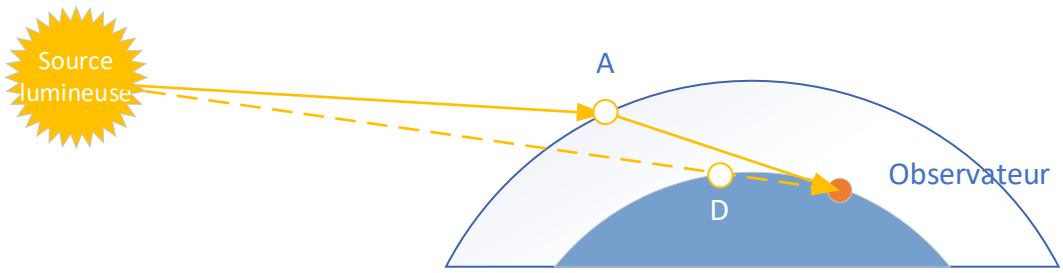


Figure 7: réfraction et lumière crépusculaire. La source lumineuse est située sous l'horizon de l'observateur en D, alors qu'elle reste visible grâce à la réfraction en A. Pour des raisons de simplification la source lumineuse est considérée ici comme ponctuelle.

Bien évidemment, ce phénomène de réfraction est beaucoup plus complexe dans la pratique, car comme vus précédemment, l'hétérogénéité de l'atmosphère crée un phénomène combinatoire difficilement quantifiable, avec des phénomènes locaux souvent prépondérants.

Diffusion

La diffusion est la propriété de la matière finement divisée de disperser la lumière dans toutes les directions. Cette diffusion ou étalement spatial se fait sans perte d'énergie donc sans changement de longueur d'onde (hormis le cas de la fluorescence non abordé ici). Elle dépend fortement de la forme des particules dites diffusantes et de leurs tailles. L'indice de réfraction est une mesure de la vitesse de la lumière dans un milieu donné, et il est déterminé par la densité et les propriétés électromagnétiques des particules dans le milieu.

Diffusion de Rayleigh

La diffusion de Rayleigh est un phénomène optique qui se produit lorsque des particules beaucoup plus petites que la longueur d'onde de la lumière incidente diffusent la lumière dans toutes les directions. Ce phénomène est nommé d'après le physicien britannique Lord Rayleigh, qui l'a étudié en premier.

La diffusion de Rayleigh est un phénomène courant dans l'atmosphère terrestre, où les molécules d'air, d'oxygène et d'azote, sont beaucoup plus petites que la longueur d'onde de la lumière visible. Lorsque la lumière blanche du Soleil traverse l'atmosphère terrestre, elle est diffusée dans toutes les directions par les molécules d'air. Cela donne au ciel une couleur bleue pendant la journée, car les molécules d'air diffusent davantage les longueurs d'onde courtes de la lumière visible (comme le bleu) que les longueurs d'onde plus longues (comme le rouge).

La diffusion de Rayleigh est également responsable de la couleur rouge des couchers de soleil et des levers de soleil. Lorsque le Soleil est près de l'horizon, les rayons de lumière doivent parcourir une distance plus longue à travers l'atmosphère avant d'atteindre l'observateur. Au cours de ce parcours, les molécules d'air diffusent donc davantage les longueurs d'onde plus courtes de la lumière (bleue et vert) , ce qui laisse passer davantage de longueurs d'onde plus longues comme le rouge et l'orange. Cela donne au ciel une couleur rougeâtre pendant ces moments de la journée.

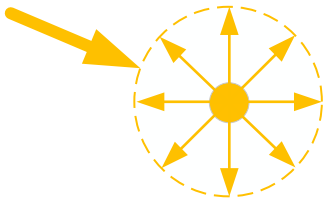


Figure 8: diffusion isotrope de Rayleigh

Diffusion de Mie

La diffusion de Mie est un phénomène optique similaire à la diffusion de Rayleigh, mais elle se produit lorsque les particules diffusantes sont de taille similaire ou supérieure à la longueur d'onde de la lumière incidente. Elle est nommée d'après le physicien allemand Gustav Mie, qui l'a décrite en 1908.

Contrairement à la diffusion de Rayleigh, qui est principalement causée par les molécules d'air dans l'atmosphère terrestre, la diffusion de Mie se produit lorsque la lumière rencontre des particules plus grosses telles que des gouttelettes d'eau, des particules de poussière, des cristaux de glace ou des molécules plus grosses. Les particules plus grosses diffusent la lumière dans toutes les directions, sans favoriser les longueurs d'onde plus courtes ou plus longues comme c'est le cas avec la diffusion de Rayleigh.

La diffusion de Mie est responsable de nombreux phénomènes optiques naturels tels que les halos solaires et lunaires.

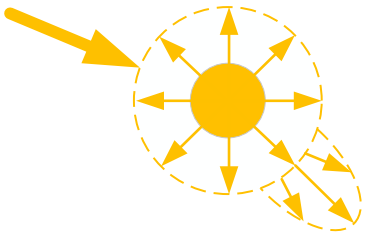


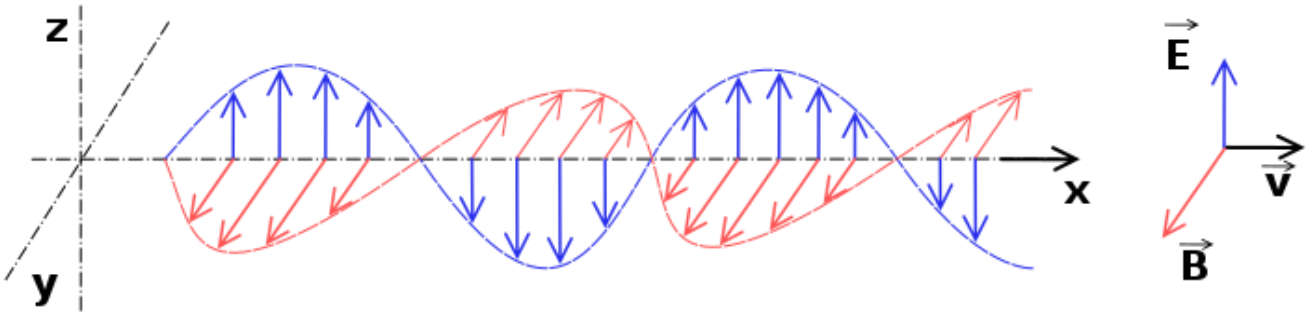
Figure 9: Diffusion vers l'avant de Mie

Contributions

En général, la diffusion de Rayleigh est plus importante à des altitudes plus élevées dans l'atmosphère, tandis que la diffusion de Mie est plus importante à des altitudes plus basses où les particules plus grosses sont plus fréquentes. Cependant, la contribution relative de ces deux types de diffusion dépend également de la longueur d'onde de la lumière incidente, car la diffusion de Rayleigh est plus forte pour les longueurs d'onde plus courtes, tandis que la diffusion de Mie est plus forte pour les longueurs d'onde plus longues.

Equation de phase

Une onde électromagnétique est une catégorie d'ondes qui se propagent dans des milieux tels que le vide ou l'air, avec une vitesse proche de celle de la lumière, soit environ 300 000 kilomètres par seconde. Ces ondes sont générées par des charges électriques en mouvement et sont caractérisées par des oscillations couplées d'un champ électrique et d'un champ magnétique, qui varient sinusoïdalement dans le temps. Les champs électriques et magnétiques oscillent perpendiculairement l'un par rapport à l'autre et se propagent à une vitesse constante dans toutes les directions, ce qui crée l'onde lumineuse.



La lumière est une onde électromagnétique.

La phase d'une onde lumineuse est une mesure de la position relative de l'onde par rapport à une référence de phase. Elle est définie comme la fraction de cycle complet que l'onde a parcourue depuis un point de référence dans son évolution temporelle. En d'autres termes, la phase de l'onde lumineuse indique à quel point elle est en train de vibrer, en commençant à partir d'un point spécifique de l'onde.

Pour une onde sinusoïdale, la phase est généralement mesurée en radians et peut prendre une valeur entre 0 et 2π. Une onde avec une phase de 0 rad est en phase avec la référence, tandis qu'une onde avec une phase de π rad est décalée de moitié par rapport à la référence.

La phase est importante dans la mesure où elle peut affecter la manière dont deux ondes lumineuses se superposent et interagissent entre elles. Par exemple, lorsque deux ondes sont en phase, elles se renforcent mutuellement et créent une onde résultante de plus grande amplitude. En revanche, lorsque deux ondes sont en opposition de phase, elles s'annulent mutuellement et créent une onde résultante de plus petite amplitude ou même nulle.

Dans la modélisation de la diffusion atmosphérique, l'équation de phase est utilisée pour décrire la propagation des ondes électromagnétiques dans l'atmosphère. Cette équation permet de déterminer comment la phase de l'onde varie en fonction de la position et du temps, en tenant compte des interactions de l'onde avec l'atmosphère.

L'équation de phase de Henyey-Greenstein est une équation mathématique utilisée pour décrire la distribution angulaire de la diffusion de la lumière par les particules, tels que les aérosols, dans un milieu. Cette équation est couramment utilisée dans la modélisation de la diffusion atmosphérique pour décrire comment la lumière est diffusée dans l'atmosphère par les particules atmosphériques.

L'équation de phase de Henyey-Greenstein est donnée par :

$$P(\theta) = (1 - g^2) / [4\pi(1 + g^2 - 2g\cos\theta)^{(3/2)}]$$

où P(θ) est la fonction de phase, θ est l'angle de diffusion, et g est le paramètre qui décrit la distribution angulaire de la diffusion de la lumière par les particules.

La valeur de g dépend des propriétés optiques des particules, telles que leur taille, leur indice de réfraction, leur forme et leur composition chimique.

il existe différentes versions de cette équation de phase qui ont été développées pour modéliser la diffusion de la lumière par des particules ayant des propriétés différentes.

Une des versions largement utilisée actuellement est la version développée par Sean O'Neil dans ses publications sur les shaders atmosphériques en 2005

https://developer.nvidia.com/gpugems/gpugems2/copyright

$$F(\theta, g) = \frac{3 * (1 - g^2)}{2 * (2 + g^2)} * \frac{1 + cos^2\theta}{(1 + g^2 - 2 * g * cos\theta)^{\frac{3}{2}}}$$

Avec cette équation on vérifie que

- La diffusion de Rayleigh peut être approximée avec une valeur de g=0
- La diffusion de Mie utilise des valeurs de g idéalement comprises entre -0.75 et -0.999.
- Les valeurs négative de g décrivent une diffusion plus importantes dans la direction de l'observateur alors que les valeurs de g positives décrivent un renvois de la lumière vers la source lumineuse.
- La valeur de g=1 ne doit jamais être utilisée car cela conduit à réduire l'équation à 0.

Modèle Global

Théorie

Implémentation

Modèle NERF

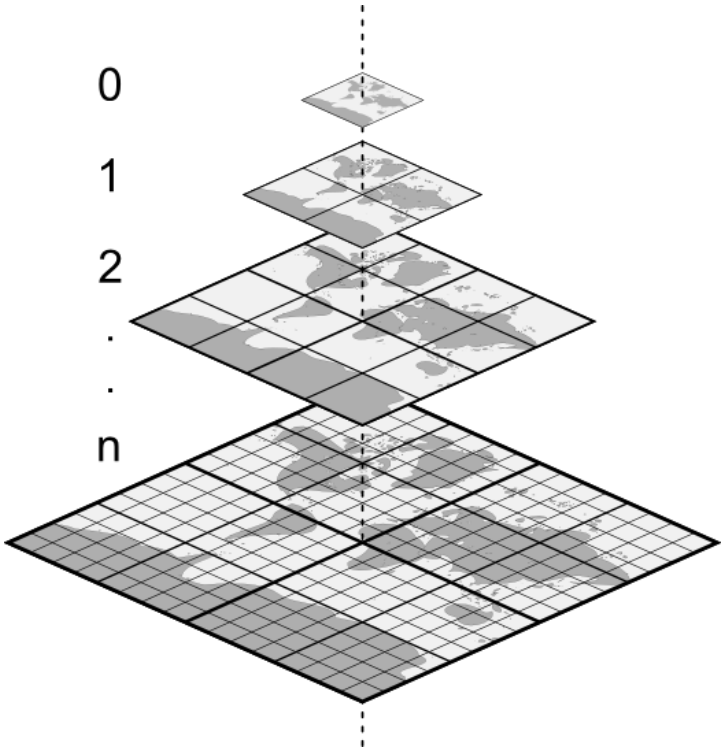
Vers la surface

La vue des planètes depuis leur surface, ou depuis leur atmosphère devrais-je dire, obéis à ses propres règles. En effet il y est nécessaire d'y afficher des niveaux de détails supplémentaires, notamment sur les reliefs, les couvertures, les surfaces liquide et les effets atmosphériques. Les modèles et les shaders s'en trouve modifiés.

Nous distinguerons alors les shaders Atmosphériques et les shaders Spatiaux.

Topologie

Nous venons de voir dans les chapitres précédent que la représentation d'une ellipsoïdes ou d'un plans partagent la même topologie. Seule la métrique change. Hors, nous savons qu'il est simple d'imaginer un mécanisme de tuile pour des plans, et l'on vient de constater que l'on peut tout simplement étendre cette notion aux ellipsoïdes.



On peut donc s'imaginer représenter une ellipsoïde par une multitude de tuile ayant chacune son centre, et son vecteur d'arc. Et même étendre ce mécanisme avec une capacitée de zoom équivalent à ce que l'on a sur les cartes WEB actuelles. Si vous avez compris ce concept, le reste n'est que détail d'implémentation.

Tuiles

Une tuile, dans le contexte cartographique et géospatial, est une subdivision régulière d'une surface plus grande, généralement représentée sous forme de grille carrée ou rectangulaire. Les tuiles sont utilisées pour diviser une carte ou un jeu de données géographiques en sections plus petites et plus gérables. Chaque tuile contient un ensemble de données spatiales spécifiques, tels que des images, des données vectorielles ou des modèles d'élévation numériques (DEM). Elles sont stockées sous forme de fichiers individuels, souvent dans des formats optimisés pour une récupération rapide et une gestion efficace. Les tuiles permettent une visualisation et une interaction rapides avec des cartes ou des données géospatiales en chargeant uniquement les tuiles nécessaires à un affichage ou une analyse donné. Elles constituent un moyen efficace de fournir des informations spatiales de manière interactive et évolutive.

Les tuiles sont souvent organisées de manière logique en utilisant une structure en pyramide, telle qu'un quadtree ou un octree. Cette approche hiérarchique divise la surface géographique en niveaux successifs, où chaque niveau représente une résolution spatiale spécifique. Au niveau le plus élevé, une tuile unique couvre toute la région avec une résolution plus basse, tandis qu'à mesure que l'on descend dans la pyramide, les tuiles sont subdivisées en quatre (quadtree) ou huit (octree) tuiles de résolution supérieure. Ainsi, les tuiles de la pyramide représentent les données géospatiales à différentes échelles, avec des niveaux supérieurs offrant une vue globale et des niveaux inférieurs fournissant des détails de plus en plus fins. Cette structure de tuiles en pyramide permet une récupération rapide des données appropriées pour l'échelle d'affichage ou l'analyse en cours, ce qui permet une exploration fluide des données géospatiales à différentes résolutions.

Les tuiles sont généralement de dimensions fixes et sont divisées en cellules qui contiennent les informations géospatiales. Dans le cas des tuiles de type raster, telles que les images ou les modèles d'élévation numérique (DEM), chaque tuile est composée d'un certain nombre de cellules. Ces cellules représentent les unités de base de données, telles que les pixels dans le cas d'une image ou les valeurs d'élévation dans le cas d'un DEM. Les tuiles raster sont souvent organisées en une grille régulière de W * H cellules, où W représente le nombre de cellules en largeur et H le nombre de cellules en hauteur. Chaque cellule stocke une valeur spécifique correspondant à un pixel de l'image ou à une valeur d'élévation dans le DEM. Ainsi, les tuiles fournissent une structure spatiale régulière et divisée en cellules pour représenter et stocker des informations géospatiales de manière efficiente et manipulable.

Il existe une différence entre les coordonnées des cellules dans la matrice (X, Y) et les coordonnées géographiques des cellules, en termes de référence spatiale utilisée.

Les coordonnées (X, Y) dans la matrice font référence à la position relative de chaque cellule par rapport à l'origine de la matrice. Habituellement, (0, 0) représente le coin supérieur gauche de la matrice, et les valeurs croissent vers le bas (axe Y) et vers la droite (axe X). Ces coordonnées sont utilisées pour accéder et manipuler les cellules dans la structure de données matricielle.

D'un autre côté, les coordonnées géographiques des cellules font référence à leur position sur la surface de la Terre. Selon la convention utilisée, ces coordonnées peuvent être prises par rapport au centre de la cellule ou à un coin spécifique (coin supérieur gauche, coin inférieur gauche, etc.) de la cellule. Les coordonnées géographiques utilisent des systèmes de coordonnées tels que la latitude et la longitude ou les projections cartographiques pour représenter la position des cellules dans le monde réel.

La convention choisie pour représenter la coordonnée géographique des cellules a une implication majeure lorsqu'il s'agit de représenter ces cellules en 3D. Prenons l'exemple de la référence au centre de la cellule. Lorsque les coordonnées géographiques sont basées sur le centre de la cellule, cela signifie que chaque cellule est positionnée en fonction de son point central dans l'espace 3D. Cependant, cela peut entraîner un problème lorsqu'on assemble plusieurs tuiles en 3D pour former un modèle continu.

Étant donné que chaque tuile est positionnée à partir de son centre, les cellules situées aux bords des tuiles présenteront un décalage par rapport aux cellules des tuiles adjacentes. Cela crée un espace vide ou un "gap" entre les tuiles.

Pour combler ce gap, le client doit télécharger les tuiles adjacentes pour créer un modèle 3D continu. Cela signifie qu'au lieu de télécharger une seule tuile, le client doit télécharger jusqu'à huit tuiles voisines (selon la méthode de subdivision utilisée) pour créer une représentation 3D continue.

Pour surmonter ce problème, Les serveurs de DEM fournissent souvent des tuiles avec un recouvrement d'une ou plusieurs cellules sur les bords. Cette approche est utilisée pour rendre chaque tuile indépendante et éviter les problèmes de bords discontinus lors de l'affichage ou de l'analyse des données. En ajoutant un recouvrement de pixels sur les bords, les tuiles se chevauchent légèrement, ce qui signifie que chaque pixel se trouve dans plusieurs tuiles adjacentes. Cela permet d'assurer la continuité des données entre les tuiles, même lorsqu'elles sont utilisées séparément. L'avantage de cette approche est que les utilisateurs peuvent travailler avec une tuile individuelle sans se soucier des valeurs d'élévation brusquement interrompues ou des artefacts aux frontières. Le recouvrement de pixels garantit que les données dans chaque tuile sont cohérentes et lisses, en minimisant les problèmes de bords visibles ou d'interpolation incorrecte des valeurs d'élévation.

Par exemple, les serveurs MapZen fournissait une tuiles de type 256 et une tuile de type 260 avec deux cellules de recouvrement sur chaque bords, le tout avec une référence de cellule au centre.

En ce qui concerne les tuiles de type image, la référence de la cellule ne pose généralement aucun problème. Lorsqu'il s'agit de représenter des images dans une structure de tuiles, les coordonnées des cellules sont souvent utilisées pour accéder directement aux pixels correspondants dans l'image. Dans ce contexte, les coordonnées de la cellule dans la tuile servent de référence pour localiser et extraire les valeurs de pixel correspondantes. Peu importe si la référence est basée sur le centre de la cellule ou sur un coin spécifique de la cellule, car les pixels de l'image peuvent être récupérés de manière indépendante et ne nécessitent pas de continuité spatiale entre les tuiles.

Dans le cas d'une représentation 3D, il est donc important de définir plusieurs paramètres clés pour les cellules, notamment leur taille, leur référence de coordonnée et la valeur optionnelle de recouvrement. Ces paramètres influencent la précision, la continuité et la qualité de la représentation 3D globale.

- Tout d'abord, la taille des cellules est un facteur crucial. Une taille de cellule plus petite permet une représentation plus détaillée et précise du relief ou de la structure 3D, mais cela peut entraîner un nombre plus important de cellules et des coûts de stockage et de traitement plus élevés. En revanche, une taille de cellule plus grande peut réduire la résolution, mais permet d'économiser des ressources. Il est donc important de trouver un équilibre entre la résolution souhaitée et les contraintes de ressources.
- Ensuite, la référence de coordonnée des cellules est cruciale pour la position spatiale précise de chaque cellule dans le modèle 3D. Il peut s'agir de la référence au centre de la cellule ou d'un coin spécifique de la cellule. Le choix dépend des besoins du système et de la manière dont les données sont traitées. La référence au centre de la cellule est souvent utilisée pour faciliter les calculs, tandis que la référence à un coin peut être préférée pour une meilleure correspondance spatiale avec d'autres données géospatiales.
- Enfin, la valeur de recouvrement optionnelle détermine la quantité de chevauchement entre les tuiles voisines dans le modèle 3D. Un recouvrement permet de créer une transition en douceur et d'éviter les lacunes ou les bords abrupts entre les tuiles. Cela peut être réalisé en incluant des cellules supplémentaires sur les bords de chaque tuile qui se chevauchent avec les cellules voisines. Le recouvrement garantit une continuité spatiale et améliore l'esthétique et la qualité visuelle de la représentation 3D.

```
enum CoordinateReference {
 center = 'center',
 nw = 'nw',
 ne = 'ne',
 sw = 'sw',
 se = 'se'
}
```

```
class Options {
  constructor(
    public cellSize: number = 1,
    public coordinateReference: CoordinateReference = CoordinateReference.center,
    public overlap: number = 0
  ) {}
}
```

Mécanisme de tuile.

Métriques.

Carte en projection de Mercator

Comme nous venons de le voir, Les systèmes de tuiles sont utilisés pour diviser une grande image en plusieurs petites images carrées appelées "tuiles". Cela permet de rendre l'affichage de l'image plus efficace en ne chargeant que les tuiles visibles à l'utilisateur plutôt que l'ensemble de l'image. Cela est particulièrement utile pour les images géospatiales, telles que les cartes, où l'affichage de l'ensemble de l'image en une seule fois peut être difficile ou impossible.

La projection de Mercator est l'une des projections cartographiques les plus couramment utilisées pour créer des tuiles. Cette projection présente l'avantage de conserver l'aspect carré des tuiles et de garantir que chaque tuile couvre une surface géographique égale en termes de longitude et de latitude. Cela signifie que les tuiles sont homogènes en termes de taille et de forme, ce qui facilite leur manipulation et leur affichage sur des écrans. Cependant, cette projection a également un inconvénient majeur : elle ne conserve pas les distances sur la surface de la Terre. En conséquence, les tuiles dans une projection de Mercator ne sont pas homogènes en termes de distance réelle entre les points. En d'autres termes, une même distance sur la carte peut correspondre à des distances différentes sur le terrain en fonction de la latitude. Cela peut entraîner des distorsions significatives dans les zones proches des pôles.

Cela dit, la métrique et les formules utilisé dans les serveurs de tuile limite la latitude a +/- 85.05112878 ce qui correspond à l'utilisation d'une opération Tangente dans le formule suivante.

```
latitude = 90 - (360 * Math.atan(Math.exp(-y * 2 * Math.PI))) / Math.PI;
```

Il est assez simple d'édicter des règles qui déterminent quels sont les tuiles visibles sur une représentation de type cartographique. Ces règles prennent en compte la position de l'utilisateur, le niveau de zoom et la taille de l'écran. Voici quelques-unes de ces règles :

1. La carte doit être centrée sur une certaine latitude et longitude. Cette position peut être spécifiée dans le code ou déterminée dynamiquement en fonction de la position de l'utilisateur.

2. Le niveau de zoom doit être défini. Plus le niveau de zoom est élevé, plus la carte est détaillée.
3. La taille de l'écran de l'utilisateur détermine le nombre de tuiles qui peuvent être affichées à la fois.
4. L'ensemble des calculs est réalisé dans l'espace de pixels 2D
5. La carte possède un cache extérieur qui détermine la visibilité des tuiles.
6. Le nombre de tuiles visible à l'instant T pour une taille de carte donnée varie avec le niveau de zoom. Les tuiles sont redimensionnées suivant différentes stratégie pour passer d'un zoom à un autre. Par exemple, Leaflet utilise en la propriété CSS transform pour redimensionner les images de tuiles de carte lorsqu'elles sont affichées à une nouvelle échelle La propriété transform permet de modifier la taille et la position des éléments HTML en utilisant des transformations CSS, telles que la mise à l'échelle, la rotation et la translation. Dans le cas des images de tuiles de carte de Leaflet, la propriété transform est utilisée pour mettre à l'échelle les images de tuiles de carte lorsqu'elles sont affichées à une nouvelle échelle, en modifiant la valeur de l'attribut scale de la transformation CSS.

   Lorsque vous interagissez avec la carte pour augmenter, ou dilinuer le niveau de zoom, Leaflet utilise cette propriété pour agrandir ou réduire les images de tuiles de carte. Cela permet à Leaflet d'ajuster rapidement la taille des images de tuiles, sans avoir besoin de générer de nouvelles images à la volée.

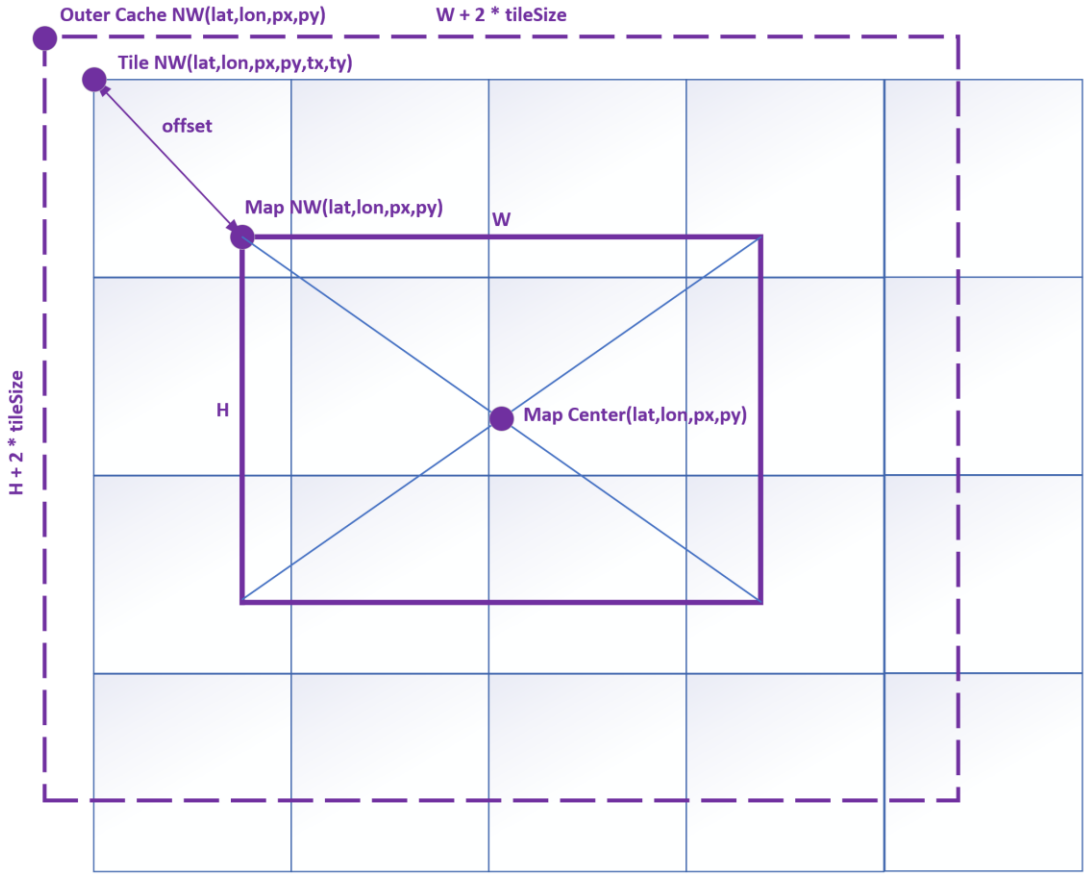


Figure 10: les tuiles dans leur contexte de visualisation cartographique 2D

Nombre de tuiles visibles

Le nombre de tuile visible ou utiles varie en fonction de la stratégie de zoom. En effet, le choix du moment de la transition entre deux niveaux de zoom est déterminant.

Mécanise de subdivision des grilles.

résumé

Coordonnées 3D des terrains

Nous travaillons maintenant à des altitudes qui permettent d'apprécier le relief. Il est donc temps d'introduire l'altitude (jusqu'à l'ors négligée et maintenue à zéro) dans nos représentations.

Les DEM (modèles numériques de terrain) sont des représentations numériques de la forme de la surface. Ils peuvent être utilisés pour visualiser, étudier et analyser les caractéristiques géographiques d'une région, telles que la topographie, la pente, l'altitude et la forme du terrain.

Sources de données

Vous aurez compris que nous parlons ici principalement des DEM de la terre, et qu'il existe d'autres sources de données pour la Lune, Mars et certains astéroïdes.

Terre

Les sources de DEM les plus courantes sur le web incluent :

- Agences gouvernementales : De nombreuses agences gouvernementales, telles que l'USGS aux États-Unis et l'IGN en France, offrent des données DEM gratuites en ligne.
- Services en ligne : Il existe également des services en ligne tels que OpenTopography et EarthExplorer qui permettent de télécharger des données DEM provenant de différentes sources.
- Satellite : Les images satellite peuvent également être utilisées pour générer des DEM. Les sources incluent des satellites tels que Landsat et Sentinel.
- Modèles commerciaux : Il existe également des entreprises qui vendent des données DEM de haute qualité, souvent produites à partir de sources satellites ou aériennes.

La NASA et l'Agence spatiale japonaise (JAXA) sont deux agences spatiales qui ont produit des données DEM de haute qualité à partir de leurs satellites.

- ASTER (Advanced Spaceborne Thermal Emission and Reflection Radiometer) est un instrument scientifique à bord du satellite Terra de la NASA. Il capture des images multispectrales de la Terre et peut être utilisé pour générer des DEM de haute précision. Les données ASTER sont disponibles

gratuitement en ligne et peuvent être utilisées pour diverses applications, telles que la reconnaissance des ressources naturelles, la gestion des catastrophes naturelles et la planification urbaine.

- ALOS (Advanced Land Observing Satellite) est un satellite de la JAXA conçu pour observer la surface terrestre. Il capture des images radar à haute résolution qui peuvent être utilisées pour générer des DEM précis. Les données ALOS sont également disponibles gratuitement en ligne et peuvent être utilisées pour des applications telles que la cartographie topographique, la surveillance des zones côtières et la gestion des ressources naturelles.

Les DEM produits par la NASA et la JAXA ont une précision d'un arc seconde, soit environ 30 mètres à l'équateur. Cela signifie que les données représentent la forme du terrain avec une précision de 30 mètres ou moins. Cependant, la précision peut varier en fonction de la latitude et de l'altitude de la zone étudiée.

Il est vrai que certaines entreprises spécialisées dans les données géospatiales, telles qu'Airbus, peuvent offrir des données DEM avec une précision plus fine, parfois sub métrique. Cela est possible grâce à l'utilisation de technologies plus avancées, telles que des systèmes de navigation par satellite et des caméras aériennes haute résolution. Les données de ce type sont souvent produites pour des applications spécifiques, telles que la planification urbaine et l'analyse du territoire, et peuvent être vendues à un coût plus élevé que les données disponibles en ligne.

Lune

La NASA a produit des données DEM pour la Lune dans le cadre de sa mission Lunar Reconnaissance Orbiter (LRO). Les données sont disponibles en ligne pour les chercheurs et les passionnés d'astronomie.

Mars

La NASA a également produit des données DEM pour Mars à partir de ses missions de reconnaissance orbitale Mars Reconnaissance Orbiter (MRO) et Mars Global Surveyor (MGS). Les données peuvent être utilisées pour étudier la topographie de la planète rouge et pour planifier les futures missions de surface.

Astéroïdes

Certaines missions d'exploration spatiale ont également produit des données DEM pour des astéroïdes, telles que Dawn de la NASA et Hayabusa de l'Agence spatiale japonaise (JAXA). Ces données peuvent être utilisées pour étudier la topographie et la géologie de ces corps célestes.

Serveurs de tuile

Les serveurs de tuiles DEM sont de plus en plus courants et sont utilisés pour fournir des données topographiques en temps réel pour des applications en ligne telles que les cartes interactives et les

visualisations 3D. Ils fonctionnent en divisant les données en petits morceaux, appelés tuiles, qui peuvent être chargés rapidement et facilement dans un navigateur web ou une application mobile. Les serveurs de tuiles sont très utiles pour les applications géospatiales qui nécessitent un accès rapide et fiable à des données topographiques à haute résolution. En utilisant un serveur de tuiles, nous pouvons accéder aux données topographiques de la zone d'intérêt en temps réel, sans avoir à télécharger de grandes quantités de données sur notre ordinateur local. Cela nous permet d'être plus efficaces et de travailler plus rapidement, ce qui est particulièrement utile pour les applications qui nécessitent une visualisation en temps réel des données topographiques.

Pour le propos de ce rapport, nous allons considérer que nous disposons d'un serveur de tuiles DEM à notre disposition, qui peut nous fournir des données topographiques à haute précision pour une zone géographique donnée.

Pour ce faire nous utiliserons l'ensemble des tuile de terrains de la Terre mis à disposition par l'entreprise MapZen.



Mapzen était une entreprise basée à New York, fondée en 2013, qui développait des services de cartographie et de localisation. Son objectif était de créer une plateforme de cartographie ouverte et accessible pouvant être utilisée par les développeurs, les chercheurs et d'autres organisations pour créer des applications basées sur la localisation. Elle offrait une variété d'outils et de services, notamment des tuiles vectorielles, de la géocodification et de la recherche, des itinéraires et des directions, ainsi que d'autres API liées à la cartographie. L'entreprise avait également créé des logiciels open-source tels que Tangram, un moteur de rendu de carte 2D/3D, et Valhalla, un moteur de routage.

Cependant, début 2018, Mapzen a annoncé qu'elle fermait ses opérations et rendait tous ses outils et services de cartographie disponibles en tant que logiciels open-source. Les données de cartographie de l'entreprise ont également été mises à disposition en téléchargement, et plusieurs organisations et entreprises se sont depuis chargées de maintenir et de mettre à jour ces ressources.

https://docs.sentinel-hub.com/api/latest/

tile metrics : les liens suivant fournissent une explication simple

https://wiki.openstreetmap.org/wiki/Slippy_map_tilenames

https://en.wikipedia.org/wiki/Web_Mercator_projection

https://registry.opendata.aws/terrain-tiles/

https://s3.amazonaws.com/elevation-tiles-prod/terrarium/13/1312/3167.png

https://s3.amazonaws.com/elevation-tiles-prod/terrarium/{z}/{x}/{y}.{extension}
https://s3.amazonaws.com/elevation-tiles-prod/normal/{z}/{x}/{y}.{extension}

https://www.mapzen.com/blog/terrain-tile-service/

full cpu https://www.babylonjs-playground.com/#1YHN5T#65
s
full gpu https://www.babylonjs-playground.com/#FTA0TS#52
avec normals en texture https://www.babylonjs-playground.com/#FTA0TS#62

https://learnopengl.com/Lighting/Basic-Lighting

basic shader to handle light and materials
https://www.babylonjs-playground.com/#689SUT#4

Les GPUs sont souvent basés sur des architectures SIMD (Single Instruction Multiple Data). Cela signifie que chaque opération de calcul est effectuée sur plusieurs données en même temps, ce qui permet de réaliser des calculs massivement parallèles.

Le problème dans ce cas avec l'implémentation des fonctions trigonométriques est que ces calculs sont souvent réalisés en utilisant des approximations plutôt que des valeurs précises, et le gain de pouvoir réaliser ces opérations en un seul cycle se paye au prix fort. En effet, ces approximations peuvent entraîner des erreurs de calcul, qui peuvent s'accumuler et entraîner des résultats imprécis et des représentations erronées. Cela peut être particulièrement préoccupant pour les applications qui nécessitent une grande précision, telles que les simulations physiques et les calculs scientifiques. Dans notre cas, lorsque les échelles changent, des sauts de valeurs apparaissent sur les transformations géodésiques, notamment avec les calculs ECEF et ENU. Ainsi des pans entiers d'élévation sont tronqués à intervalles régulier. Ce problème est connus et le comportement, qui plus est, change d'une machine sur l'autre.

Une solution courante pour résoudre ce problème consiste à utiliser une table de recherche, également connue sous le nom de lookup table. Cette méthode consiste à pré-calculer les valeurs des fonctions

trigonométriques à l'aide du CPU, et à les stocker dans une table de référence envoyée sous forme de buffer ou texture au GPU (CPU).

En revanche, cela complexifie l'approche par instance des grilles topologiques et nous oblige à précalculer une table de lookup pour chaque tuile.

- Lookup table latitude h x 1 sin, cos , N RGB_FLOAT
- Lookup table longitude w x 1 sin, cos RG_FLOAT
- Table altitudes w x h alt R_FLOAT
- Elllipsoid
- ENUTransform
- X,Y,LOD

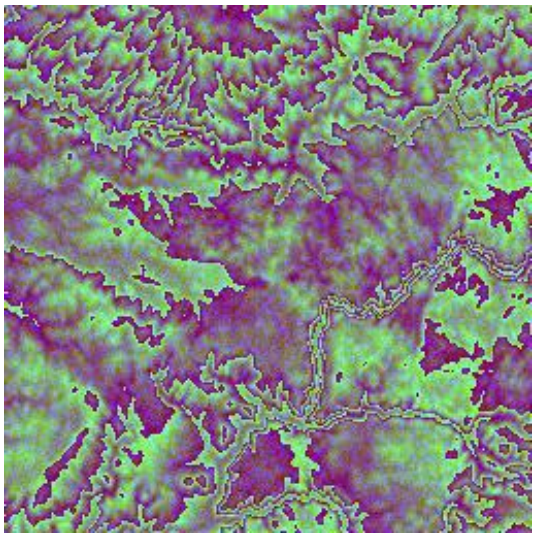
Dans un futur chapitre, nous explorerons comment créer nos propre tuiles en utilisant notamment les données d'ALOS (Advanced Land Observing Satellite) de l'Agence spatiale japonaise (JAXA). Cet exercice à été rendu nécessaire par l'exploration de nouveaux terrains.

Clients de tuiles

Les clients de tuiles sont la partie logique d'accès aux données des serveurs. Ils servent à transcrire une coordonnée géodésique en une requête vis-à-vis d'un serveur. Par définition, cette dernière requête est asynchrone.

Le formalisme d'une telle interface pourrait être

Format des données



Vous l'avez compris, une tuile de type DEM est une image ou l'élévation est encodée en utilisant les composantes RGB ou RGBA. Les formats d'image les plus utilisés sont les formats TIFF (avec une spécialisation en geoTiff), png et dernièrement webp Ce dernier format est beaucoup plus compact et conserve malgré tout l'intégralité des données comme png.

Les pixels dans une image sont stockés logiquement en matrice, c'est-à-dire en lignes et en colonnes. Chaque pixel dans une image représente une valeur de couleur unique pour une position donnée dans l'image.

Les pixels sont rangés dans l'ordre en ligne, en commençant par la première ligne de gauche à droite, puis la deuxième ligne de gauche à droite, et ainsi de suite. Cela permet d'accéder rapidement et facilement à n'importe quel pixel dans l'image en utilisant ses coordonnées (ligne et colonne).

La plupart des images numériques sont codées en utilisant un système de codage de couleur, tel que RGB (rouge, vert, bleu) ou CMYK (cyan, magenta, jaune, noir), qui utilise des valeurs numériques pour représenter les couleurs. Les valeurs de couleur pour chaque pixel sont stockées en binaire dans un fichier image, généralement sous forme de tableaux de données multidimensionnelles.

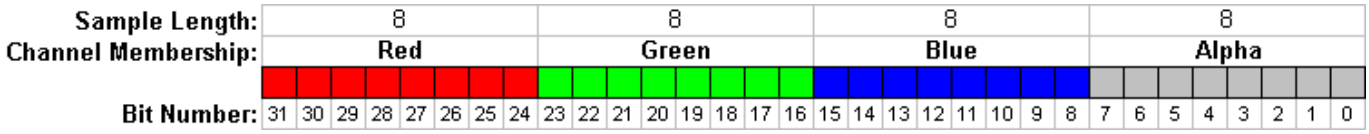
Nous allons nous abstraire de l'encodage de l'image et considérer cette dernière de la même manière que l'on considère une texture.

Il existe plusieurs manière de stocker l'information. Les plus courantes sont

- RGB
- RGBA

RGBA

Les canaux de couleurs sont disposés en mémoire sous forme d'un seul entier non signé de 32 bits . Le canal rouge dans les 8 bits les plus élevés, suivi du canal vert, du canal bleue et enfin du canal alpha dans les 8 bits les plus bas:



Selon la spécifications, le format PNG est toujours sous forme RGBA.

RGB

Pour un image de taille W, H, le pixel aux coordonnées x, y (avec x ∈ [0, W-1] et y ∈ [0,h-1) sera adressé par

V = (x * w + y) * stride

Avec stride la taille du pixel, 4 pour RGBA, 3 pour RGB, etc..

Nous avons donc basiquement ce pseudo code pour évaluer tous les pixels d'une image donnée

```
for(row=0; row!= h; row++) {
    for(column=0; column!= w; column++) {
        const r =  ( pixels[i]>> 24 ) & 0xFF ;
        const g = ( pixels[i] >> 16 ) & 0xFF ;
        const b = ( pixels[i] >> 8  ) & 0xFF ;
```

```
            const a =( pixels[i++] >> 0  ) & 0xFF ;
          }
        }
      }
```

Codage de l’altitude

Les tuiles de DEM sont des grilles régulières de points de mesure de l'altitude, généralement dans une projection géographique ou UTM. Chaque tuile est divisée en un nombre défini de pixels, et l'altitude pour chaque pixel est stockée en tant que valeur numérique associée à ses coordonnées géographiques ou UTM.

La valeur numérique représentant l'altitude pour chaque pixel dans une tuile de Digital Elevation Model (DEM) est généralement stockée sous forme de nombre entier ou flottant en binaire. Le nombre de bits utilisé pour représenter chaque pixel dépend de la précision requise pour les données d'altitude.

Topologie

Nous reprenons ici le travail fait précédemment sur les grilles normalisées (Chapitre sur les grilles de référence).

En préambule, nous définissons ici notre travail dans un repère cartésien Direct, soit

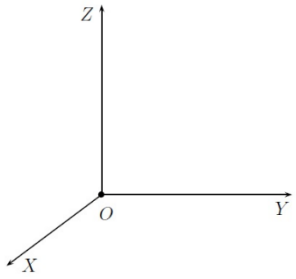


Figure 11: notre repère cartésien

En 3D, chaque pixel de l’image d’élévation représente une surface. La coordonnées relative (X,Y) de la valeur Z encodée sont généralement au centre de cette surface. Dans la théorie et selon les méthodologie employée par les fournisseurs des images ce positionnement relatif peut être

- En haut à gauche
- En haut à droite
- En bas à droite
- En bas à gauche
- Au centre.

Ainsi, on s’aperçois du décalage existant si l’on utilise la coordonnée centrale comme position X,Y de la valeur Z.

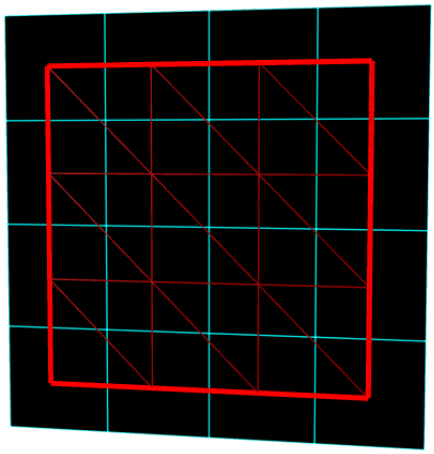
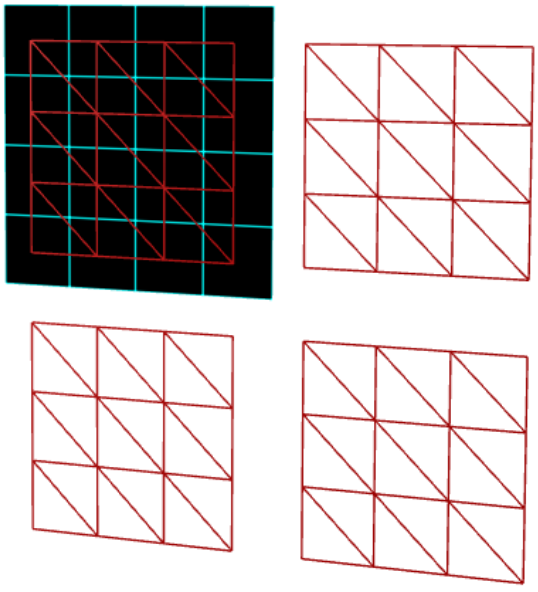


Figure 12: en rouge la grille de terrain, positionnée par rapport a l'image source en pixel pour une valeur située au centre du pixel.

https://playground.babylonjs.com/#CC0U0B

Dans l'espace 3D, on s'aperçois alors que cela génère un espace vide entre les tuiles successives, non comblé par la topologie.



Par exemple, une tuile de largeur 4 occupera un longueur de 3 unités ce qui laisse un ligne de 1 unité vide entre les tuiles.

Pour ce faire certains fournisseurs de tuiles proposent des tuiles avec un recouvrements sur les bordures, afin de ne pas obliger l'utilisateur à télécharger les voisins immédiat. L'IA du BABYLON travaille aussi bien sur des

tuiles dite étendues, au format w + n, h + n avec un recouvrement de n valeurs sur les bords, qu'avec des tuiles simple ou n est égal à zéro.

Stratégie de remplissage.

Ce gap doit donc être remplis pas une topologie. Nous pouvons donc :

- soit adjoindre des meshes additionnels.
- soit étendre la grille ;

l'adjonction de mesh est une méthode connue dans ce genre de situation et à été utilisée avec succès dans la littérature (Terrain rendering Using GPU- Based Geometry Clipmap, Arul Asirvatham, Hugues Hoppe, GPUGEM Tome 2, nvidia).

Dans notre cas, il semble plus judicieux d'étendre la grille vers l'est et le sud afin de remplir le vide entre les tuiles voisines. Ainsi, les grilles seront de taille w+1, h+1, avec une valeur d'élévation qui sera échantillonné sur les voisins immédiats.

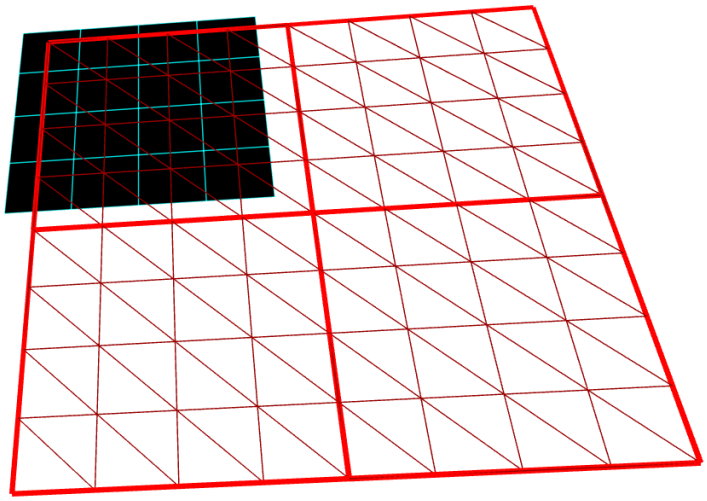


Figure 13: les grilles étendues à N+1

Considérons les coordonnées $(t, s) \in [0,1]$ dans une image de tuile.

Les tableaux suivants représentent les pixels d'une image carré de taille N et les valeurs associées aux coordonnée (t,s) dans l'image.

t=0, s=0	1/w, 0	2/w, 0	…	(w-1)/w, 0	1, 0
0, 1/h	1/w, 1/h	2/w, 1/h	…	(w-1)/w, 1/h	1, 1/h
0, 2/h	1/w, 2/h	2/w, 2/h	…	(w-1)/w, 2/h	1, 2/h
…	…	…	…	…	…
0, (h-1)/h	1/w, (h-1)/h	2/w, (h-1)/h	…	(w-1)/w, (h-1)/h	1, (h-1)/h
0, 1	1/w, 1	2/w, 1	…	(w-1)/w, 1	1, 1

Si nous rajoutons une ligne et une colonne à la matrice nous obtenons

t=0, s=0	1/w, 0	2/w, 0	…	(w-1)/w, 0	1, 0	1+1/w, 0
0, 1/h	1/w, 1/h	2/w, 1/h	…	(w-1)/w, 1/h	1, 1/h	1+1/w, 1/h
0, 2/h	1/w, 2/h	2/w, 2/h	…	(w-1)/w, 2/h	1, 2/h	1+1/w,2/h
…	…	…	…	…	…	…
0, (h-1)/h	1/w, (h-1)/h	2/w, (h-1)/h	…	(w-1)/w, (h-1)/h	1, (h-1)/h	1+1/w, (h-1)/h
0, 1	1/w, 1	2/w, 1	…	(w-1)/w, 1	1, 1	1+1/w, 1
0, 1+1/h	1/w, 1+1/h	2/w, 1+1/h	…	(w-1)/w, 1+1/h	1, 1+1/h	1+1/w, 1+1/h

Or, comme nous l'avons évoqué précédemment, la partie grisée de la matrice de coordonnée sort de la définition telle que $(t, s) \in [0,1]$.

C'est un cas d'usage classique dans les textures, qui est désigné sous le terme de WRAPING, et les moteurs de rendus définissent des stratégies pour traiter ce cas.

La stratégie la plus couramment utilisée est la répétition, Ainsi le moteur prends pour toute valeur > 1 le modulo 1 de la valeur, ce qui revient à répéter les coordonnées indéfiniment.

t=0, s=0	1/w, 0	2/w, 0	…	(w-1)/w, 0	1, 0	0, 0
0, 1/h	1/w, 1/h	2/w, 1/h	…	(w-1)/w, 1/h	1, 1/h	0, 1/h
0, 2/h	1/w, 2/h	2/w, 2/h	…	(w-1)/w, 2/h	1, 2/h	0,2/h
…	…	…	…	…	…	…
0, (h-1)/h	1/w, (h-1)/h	2/w, (h-1)/h	…	(w-1)/w, (h-1)/h	1, (h-1)/h	0, (h-1)/h
0, 1	1/w, 1	2/w, 1	…	(w-1)/w, 1	1, 1	0, 1
0, 0	1/w, 0	2/w, 0	…	(w-1)/w, 0	1, 0	0, 1+1/h

Une autre stratégie est de limiter les valeurs à 1, ce qui revient a répéter la même coordonnée indéfiniment. Ce mode est appelé CLAMP.

t=0, s=0	1/w, 0	2/w, 0	…	(w-1)/w, 0	1, 0	1, 0
0, 1/h	1/w, 1/h	2/w, 1/h	…	(w-1)/w, 1/h	1, 1/h	1, 1/h
0, 2/h	1/w, 2/h	2/w, 2/h	…	(w-1)/w, 2/h	1, 2/h	1,2/h
…	…	…	…	…	…	…
0, (h-1)/h	1/w, (h-1)/h	2/w, (h-1)/h	…	(w-1)/w, (h-1)/h	1, (h-1)/h	1, (h-1)/h
0, 1	1/w, 1	2/w, 1	…	(w-1)/w, 1	1, 1	1, 1
0, 1	1/w, 1	2/w, 1	…	(w-1)/w, 1	1, 1	1, 1

Ces modes d'adressages sont donc logiquement dédiés à une seule texture, tel que $(t, s) \in [0,1]$.

Hors notre logique de tuiles, sous-entend la manipulation d'images individuelles, téléchargées individuellement et traitée individuellement. Les représenter sous forme d'une seule et même texture est possible mais implique une opération d'agrégation supplémentaire.

La logique voudrait donc que l'on associe une coordonnée à une image particulière. Ainsi nous pouvons obtenir des coordonnées $(t, s, u) \in [0,1] \times [0,1] \times Z^+$ ou u représente l'index de l'image concernée.

En posant 4 images ayant pour index 0,1,2,3 avec 0 l'index de l'image courante

0	1
2	3

Nous obtenons

t=0, s=0, u=0	1/w, 0, 0	2/w, 0, 0	…	(w-1)/w, 0, 0	1, 0, 0	0, 0, 1
0, 1/h, 0	1/w, 1/h, 0	2/w, 1/h, 0	…	(w-1)/w, 1/h, 0	1, 1/h, 0	0, 1/h, 1
0, 2/h, 0	1/w, 2/h, 0	2/w, 2/h, 0	…	(w-1)/w, 2/h, 0	1, 2/h, 0	0,2/h, 1
…	…	…	…	…	…	…
0, (h-1)/h, 0	1/w, (h-1)/h, 0	2/w, (h-1)/h, 0	…	(w-1)/w, (h-1)/h, 0	1, (h-1)/h, 0	0, (h-1)/h, 1
0, 1, 0	1/w, 1, 0	2/w, 1, 0	…	(w-1)/w, 1, 0	1, 1, 0	0, 1, 1
0, 0, 2	1/w, 0, 2	2/w, 0, 2	…	(w-1)/w, 0, 2	1, 0, 2	0, 0, 3

Dans notre cas, et pour faire simple, l'IA du BABYLON utilise les texture de type 2D_ARRAY et/ou 3D (voir l'annexe) afin de stocker les texture individuellement et pouvoir ainsi y accéder garce à l'index u.

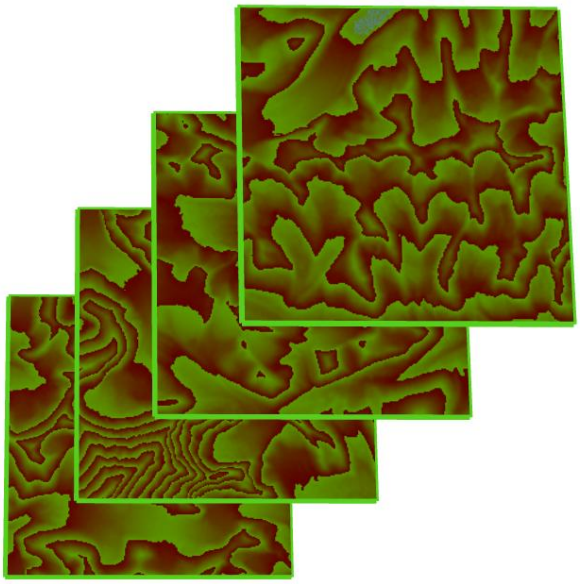
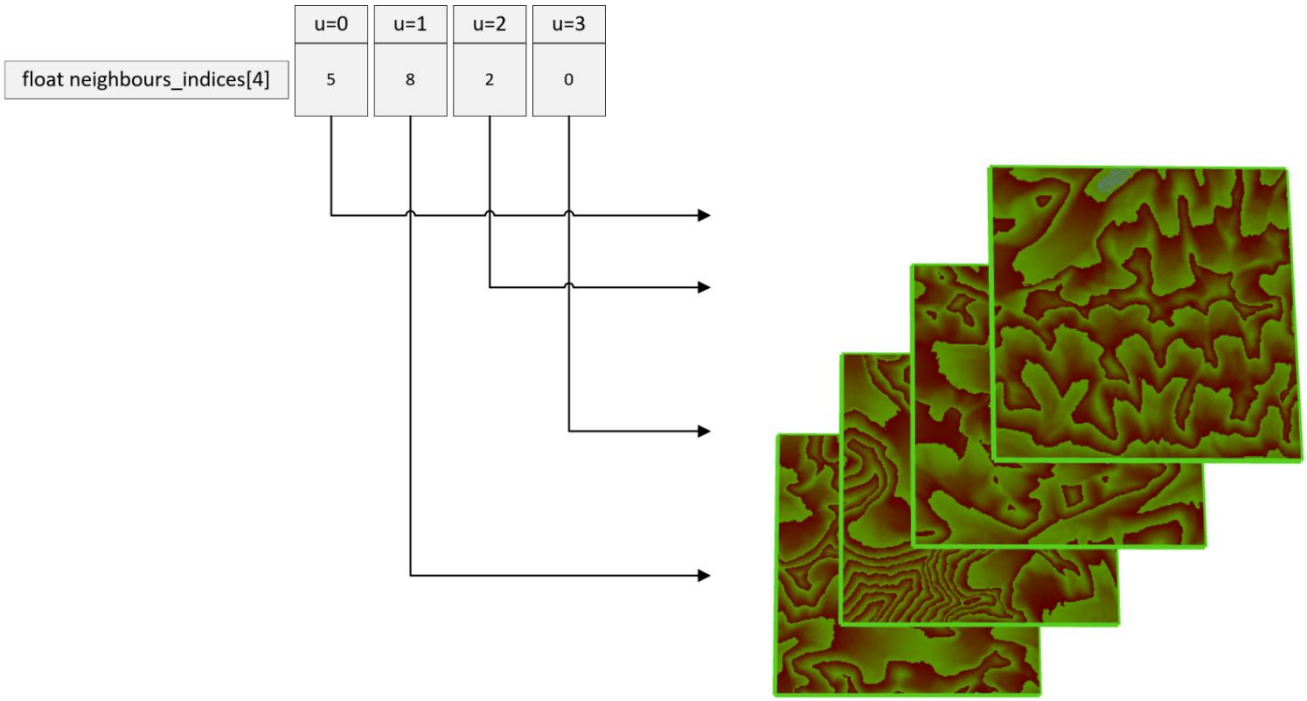


Figure 14 : une image de type 2D_ARRAY ou les tuiles sont rangées selon un index de profondeur

Dans la pratique, il est nécessaire de fournir une table de redirection pour les tuiles voisines de chaque tuile, ainsi nous passerons un vecteur de dimension 4 qui redirigera vers les index de profondeur (depth) de la texture.



Une valeur négative dans le tableau d'indices, signifies qu'il n'y a pas de texture disponible , c'est-à-dire pas de tuile voisines. Dans ce cas la valeur est reprise sur la bordure de la texture courante (index 0). A noter que par définition il n'est pas possible d'avoir un index zéro qui pointe sur une texture inexistante.

En considérant que nous avons des textures de type FLOAT, ou HALF_FLOAT (voir annexes) le vertex shader ressemblerais à

```
uniform highp Sampler2DArray u_altitudeSampler;
in vec4 neighbours_indices;
in vec3 tex_coordinates;

main(){

    float i = neighbours_indices[tex_coordinates.z];
    vec3 v = vec3(tex_coordinates.xy, i);
    if( i < 0.0) {
        v.x = v.x != 0.0 ? v.x : 1.0;
        v.y = v.y != 0.0 ? v.y : 1.0;
        v.z = 0.0;
    }
    float altitude = float(texture(u_altitudeSampler,v));
}
```

Tile stack : https://playground.babylonjs.com/#0FI51E#1

Coordonnées

Maintenant que nous avons la topologie de nos grilles de terrains, il est important d'expliquer comment les positionner dans une scène 3D.

Pour commencer, nous allons expliquer quelles sont les transformations impliquées sur la cartographie 2D. Dans la suite du raisonnement, nous partons du principe que la composante Z se trouve toujours dans la direction de l'observateur, c'est-à-dire vers le « haut ».

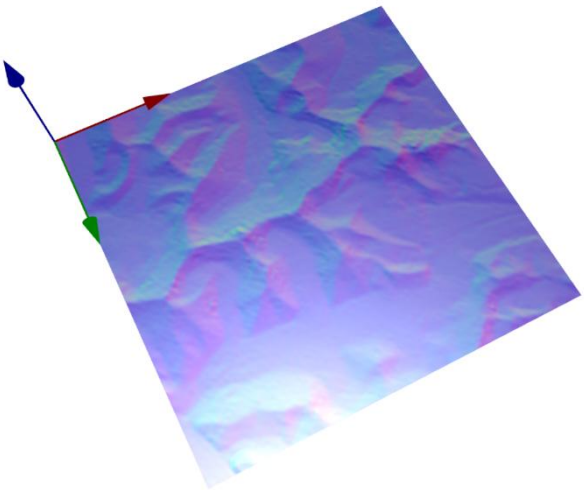


Figure 15 : l'orientation du système de coordonnée des tuiles Mercator.

La figure suivante résume l'ensemble des systèmes de

coordonnées utilisé dans la logique de positionnement d'une tuile 2D sur une surface. Nous verrons l'adressage de cette surface en 3D ultérieurement.

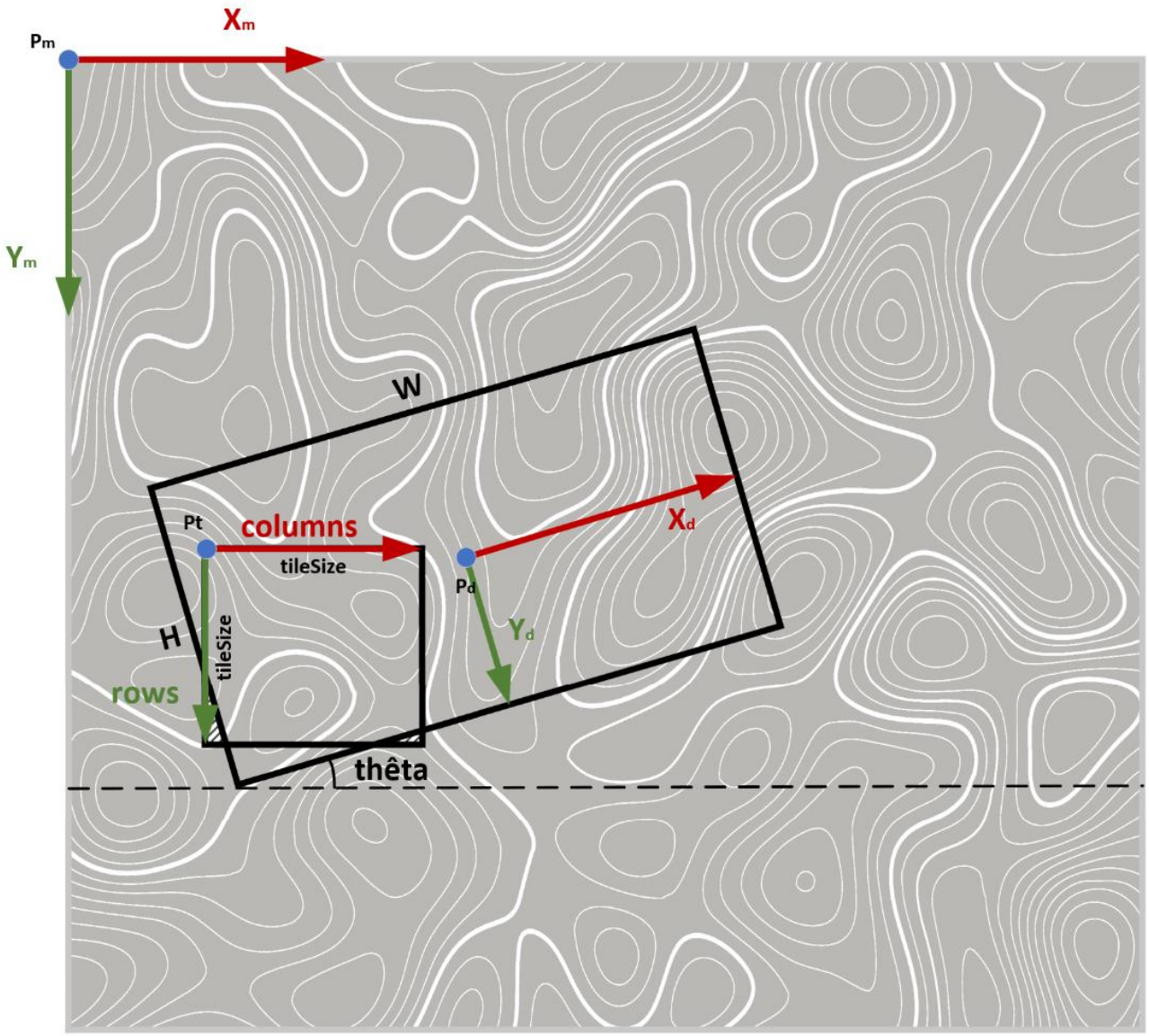


Figure 16 : Les différents repères de visualisation de la carttographie.

Soit

Soit M repère de cartographie de tuile, définis selon les métriques vues précédemment.

$M = \{P_m , aX_m , bY_m \mid a, b \in E\}$

où :

- M représente l'espace d'adressage des tuiles.
- P_m est le point d'origine du repère.
- X_m et Y_m sont les vecteurs de base du repère.

- a et b sont des coefficients entiers.
- E représente l'ensemble des nombres entiers.

Cette équation exprime que chaque point de l'espace d'adressage des tuiles M est obtenu en déterminant les coefficients entiers a et b, puis en multipliant les vecteurs de base X_m et Y_m respectivement par ces coefficients, et en les ajoutant à l'origine P_m.

Dans ce repère M, le point Pt est le point d'origine de la tuile qui est aussi le point d'origine de l'image sous-jacente. P_t est exprimé en Pixel dans M.

Soit V repère de visualisation de tuile qui est le filtre à travers lequel nous visualisons la cartographie.

Pour exprimer P_t dans V nous devons appliquer une transformations de repère. L'IA du Babylon travaille avec des matrices de type « Row-Major »

1. Translation de $-P_d$.

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -P_x & -P_y & 1 \end{bmatrix}$$

2. Rotation de – thêta (P_d est devenus le centre de rotation après la translation précédente) .

$$R = \begin{bmatrix} \cos & \sin & 0 \\ -\sin & \cos & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

On obtiens par T.R

$$T.R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -P_x & -P_y & 1 \end{bmatrix} \begin{bmatrix} \cos & \sin & 0 \\ -\sin & \cos & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos & \sin & 0 \\ -\sin & \cos & 0 \\ -P_x \cos + P_y \, sin & -P_x \sin - P_y \, cos & 1 \end{bmatrix}$$

Travail sur un Canvas

Si l'on travaille sur un élément de type canvas 2D , alors nous bénéficions déjà de ces opérations de changement de repère et il suffit d'appliquer une translation suivie d'une rotation au contexte du canvas.

Le code suivant donne un exemple simpliste d'utilisation.

```
protected invalidateDisplay(canvas, res, scale, rotation) {
  const ctx = canvas.getContext();
  if (ctx) {
    ctx.save();

    if (res) {
      ctx.translate(res.width / 2, res.height / 2);
    }
```

```
        if (scale) {
            ctx.scale(scale, scale);
        }
        if (rotation) {
            const angle = rotation * Scalar.DEG2RAD; // to radian
            ctx.rotate(angle);
        }
        .....
        ctx.restore();
    }
}
```

A noter que translate(), scale() et rotate() sont cumulatifs, il est donc nécessaire de sauver et de restaurer le contexte du canvas à chaque opération d'affichage. Nous aurions pu également directement affecter la transformation vue précédemment sous forme matricielle.

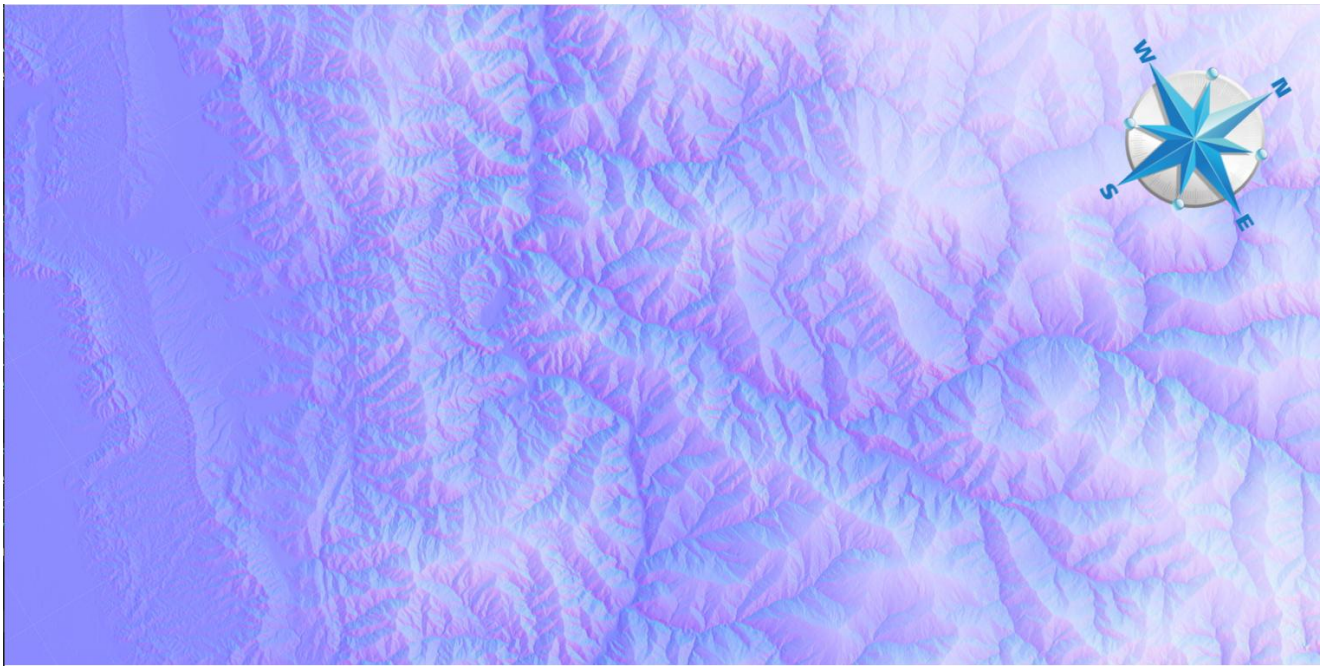


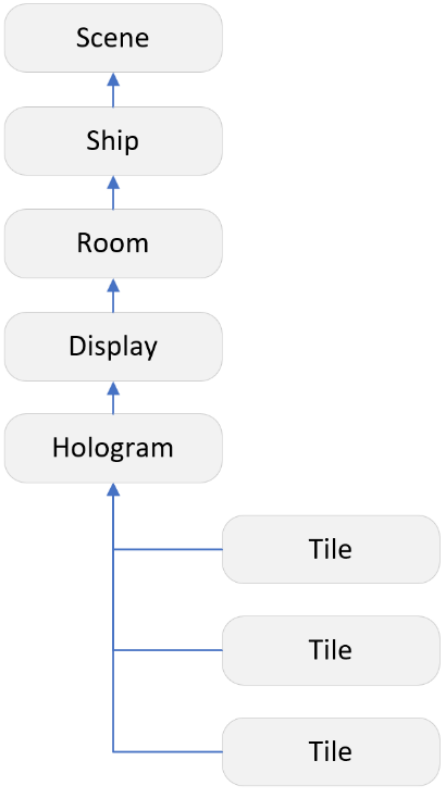
Figure 17: les tuiles visualisées dans un canvas 2D

Travail en 3D

En 3D, les formulations mathématiques sont conservées. Cela dit, les coordonnées homogènes sont exprimées sur des vecteurs à 4 composantes et des matrices carrés de dimension 4x4.

Sur le BABYLON, et de manière générale sur l'ensemble des vaisseaux, La suite des transformations peut être exprimée à travers la hiérarchie des objets de la scène. C'est-à-dire que les coordonnées d'un objet sont toujours exprimée dans le repère local de son parent.

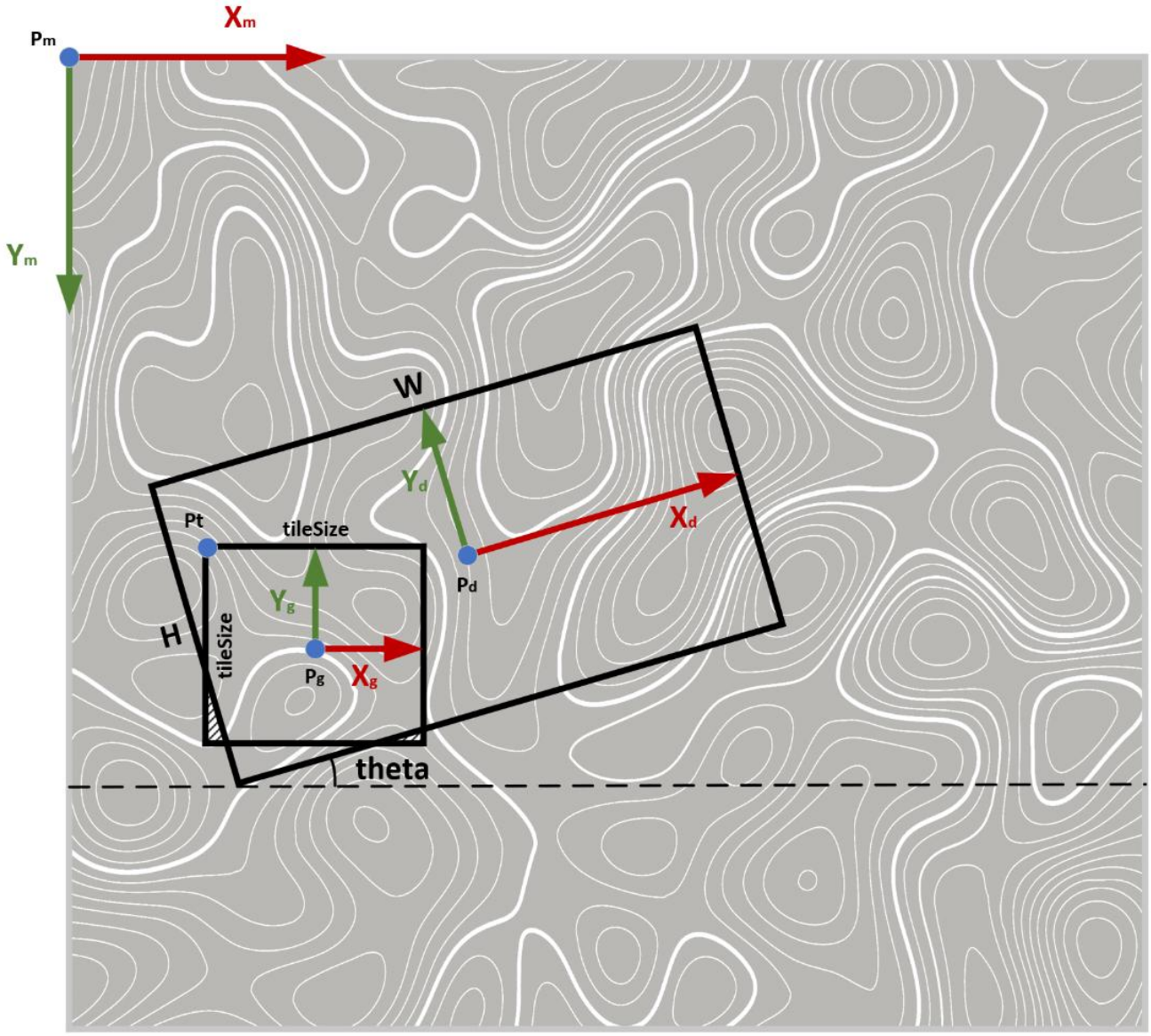
Nous pourrions donc avoir une hiérarchie du style



Si le display, est l'écrans physique au-dessus duquel projeter l'hologramme, l'hologramme contient toute les transformations nécessaires pour transformer les tuiles afin qu'elles soit correctement positionnées.

Si l'on reprends le graphique précèdent, légèrement modifié, nous pouvons affecter à la hiérarchie les différentes opérations.

Il est primordial de noter que dans le moteur de rendu du Babylon, l'ordre des transformations par défaut est le suivant : **T**ranslation, **R**otation, puis **S**cale (mise à l'échelle). Cela correspond à l'ordre **TRS.**



1. D'une part il faut déterminer la position des tuiles sur le Display, ce qui revient à effectuer la Translation de -P_d et inversion du repère Y.

   Il est important de noter que l'on considère maintenant le centre de la grille et non plus le coin supérieur gauche de la tuile. Notre transformation portera sur le point P_g et non P_t. On obtient le premier par l'adjonction de la moitié de la taille de la tuile.

$$Tg = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ tileSize/2 & tileSize/2 & 0 & 1 \end{bmatrix}$$

$$T0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -P_x & -P_y & -P_z & 1 \end{bmatrix} \quad S0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$Tg.T0.S0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ tileSize/2 & tileSize/2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -P_x & -P_y & -P_z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -P_x + tileSize/2 & -P_y + tileSize/2 & -P_z & 1 \end{bmatrix}$$

Cette transformation est indépendante de la hiérarchie des objets et donne juste la position du centre d'une grille dans le repère local du display.

2. L'hologramme prend en compte la rotation et l'éventuelle mise à l'échelle.

$$R1 = \begin{bmatrix} cos & sin & 0 & 0 \\ -sin & cos & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad S1 = \begin{bmatrix} Sx & 0 & 0 & 0 \\ 0 & Sy & 0 & 0 \\ 0 & 0 & Sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R1.S1 = \begin{bmatrix} cos & sin & 0 & 0 \\ -sin & cos & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} Sx & 0 & 0 & 0 \\ 0 & Sy & 0 & 0 \\ 0 & 0 & Sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} Sx\ cos & Sx\ sin & 0 & 0 \\ -Sy\ sin & Sy\ cos & 0 & 0 \\ 0 & 0 & Sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

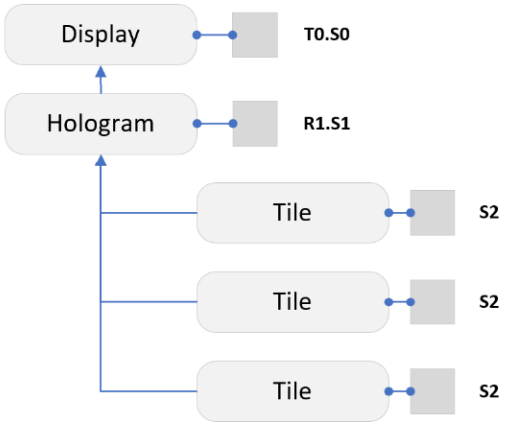
3. Finalement, les tuiles se voient affecter une transformation d'échelle, car elle sont représentée par une grille normalisée.

$$S2 = \begin{bmatrix} Sx & 0 & 0 & 0 \\ 0 & Sy & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \ avec\ Sx = Sy = tile\ size * cell\ size$$

Où

- *tile size* est la taille de la tuile, par exemple 256
- *cell size* est la taille du pixel dans l'environnement choisi. Cette valeur ne possède pas d'unité et est arbitrairement choisie en fonction de la scène. Nous y reviendrons plus tard lorsqu'il s'agira de normaliser les altitudes en rapport avec l'échelle de la carte.

Pour résumer nous pouvons affecter les opérations de transformations aux différents nœuds de la hiérarchie.



Echelle

La projection de Mercator est une projection cartographique couramment utilisée pour représenter la Terre sur une carte plane. Dans cette projection, les méridiens et les parallèles sont représentés par des lignes droites parallèles. La taille d'un pixel en mètres sur une carte web utilisant la projection de Mercator dépend du niveau de zoom de la carte et de la latitude à laquelle vous vous trouvez. La projection de Mercator conserve les angles, mais elle déforme les distances à mesure que l'on s'éloigne de l'équateur.

Pour calculer la taille d'un pixel en mètres, vous pouvez utiliser la formule suivante :

$$pixelSize = Ce * \frac{\cos(lat)}{tileSize * 2^{lod}}$$

Où :

Ce est la circonférence approchée de l'ellipsoïde à l'équateur en prenant l'axe majeur.

$$Ce = 2 * \pi * Ellipsoid.SemiMajorAxis$$

- lat est la latitude à laquelle vous vous trouvez, exprimée en radians.
- tileSize représente la taille d'une tuile en pixels. Par exemple 256 pixels.
- lod est le niveau de zoom de la carte.

Notez que cette formule est une approximation, car la projection de Mercator n'est pas une projection conforme à l'échelle partout sur la carte.

En utilisant cette formule, vous pouvez calculer la taille d'un pixel en mètres en fonction du niveau de zoom de la carte et de la latitude à laquelle vous vous trouvez.

On s'aperçoit donc que la résolution de la carte projeté en 2D évalue en fonction de

- la latitude,
- du facteur de zoom
- de l'ellipsoïde.

Considérant une carte « localement conforme » on peut obtenir l'échelle de la carte projeté si l'on connais sa taille métrique en X.

Ainsi, une carte de taille W en mètre avec une résolution ppm de N pixel par mètre aura une échelle calculée par les formules suivantes.

$$world\ size = \text{W * ppm * pixel Size}$$

$$scale = 1 /\ \text{ppm * pixel Size}$$

Transition

L'ors d'un passage à un niveau de détail supérieur ou inférieur, les tuiles sont amenées à évoluer dans leur contenus et leur représentation. C'est ce que l'on vas appeler ici une transition.

Pour schématiser, les tuiles pourrons évoluer pour représenter

- un contenu diffèrent qui est le principe intrinsèque de la hiérarchie de tuile et du niveau de détail souhaité dans le cas particulier des tuiles 3D.
- Une échelle différente afin de conserver une expérience utilisateur fluide.

La notion d'échelle introduit une valeur de niveau de détail en virgule flottante, a l'opposé du niveau de détail de la métrique du système de tuile qui est une valeur entière. Nous les appellerons respectivement **lodf** et **lod**.

Changer de zoom sur une carte reviens donc a appliquer une transformation d'échelle linéaire sur cette carte, tout en changeant de contenu régulièrement afin de bénéficier des données les plus précises pour un niveau de détail donné.

Cette transition de contenus doit donc être gérée en fonction de la valeur de lodf

$$lod = f(lodf)$$

La fonction la plus simple est la plus efficace et la fonction qui arrondit au nombre entier le plus proche. Dans la plupart des langage informatique elle est nommé round().

```
round(3.14159) // 3
round(3.5698) // 4
```

Elle nous sera très utile pour déterminer le passage d'un niveaux de tuile à un autre, tout en nous garantissant une transition d'échelle simple dans l'intervalle [N-0.5, N+0.5].

```
public static GetLodScale(lodf: number): number {
  // Trick to avoid floating point error.
  let lodOffset = (lodf * 1000 - Math.round(lodf) * 1000) / 1000;
  // scale corresponding to the decimal part
  let scale = lodOffset < 0 ? 1 + lodOffset / 2 : 1 + lodOffset;
  return scale;
}
```

La fonction précédente nous donne un facteur d'échelle dans l'intervalle [0.75, 1.5]. A noter le passage d'échelle effectué (x1000) pour éviter l'instabilité numérique lié à la manipulation de la partie décimale.

Pour résumer, nous avons 2 valeurs

$$\begin{pmatrix} lod = round(lodf) \\ scale = GetLodScale\ (lodf) \end{pmatrix}$$

Lorsque la valeur de lod change, nous chargeons alors un nouveau niveau de détail de la hiérarchie de tuile. Ce chargement est par définition asynchrone et représente un délais *dt* entre le moment ou la requête est effectuée auprès du serveur de tuile et le moment ou la tuile est disponible pour l'affichage. Ce délais est plus ou moins long en fonction :

- Du navigateur
- Du cache local
- Du serveur de tuile
- De la qualité de la connexion.

Pendant ce temps, sur l'affichage, l'espace réservé pour la tuile peut apparaitre vide.

Pour y remédier, il y a deux stratégies principales.

1. Travailler sur la diminution de dt
2. Utiliser les données locale afin de remplir les vides

Bien que la première option soit indispensable, elle comprend des paramètres non contrôlable. Ainsi le travail sur l'utilisation des donnée locales sera privilégié.

L'architecture sous-jacente de la hiérarchie de tuile nous permet assez simplement de déterminer que les tuiles enfants représentent une section de la tuile parent, et qu'une tuile parent représente l'assemblage de ses tuiles enfants.

Si l'on introduit la notion de section comme une figure géométrique plane à quatre côtés et quatre angles droits, on peut simplement décrire les coordonnées d'une section dans une tuile avec

- Le point supérieur gauche de la section par rapport au point supérieur gauche de la tuile.
- La taille (largeur, hauteur) de la section.

Dans la hiérarchie de tuile cela nous donne 4 sections par tuile avec les coordonnées suivantes :

On pose

$$\begin{pmatrix} s = \frac{tileSize}{2} \\ w = s \\ h = s \end{pmatrix}$$

$0 = \begin{pmatrix} x = 0 \\ y = 0 \\ w \\ h \end{pmatrix}$	$1 = \begin{pmatrix} x = s \\ y = 0 \\ w \\ h \end{pmatrix}$
$2 = \begin{pmatrix} x = 0 \\ y = s \\ w \\ h \end{pmatrix}$	$3 = \begin{pmatrix} x = s \\ y = s \\ w \\ h \end{pmatrix}$

On peux alors définir deux sections dans notre logique d'affichage.

1. Une section source qui définit quelle partie de la donnée source l'on utilise.
2. Une section cible qui définit quelle partie doit être dessinée.

Nous appellerons ces deux sections respectivement ***source*** et ***target***.

Nous introduisons alors une structure de type décorateur, qui pointe sur une donnée et lui affecte une section source et une section cible

```
…
export interface ITileSection extends ICartesian2, ISize2 {}

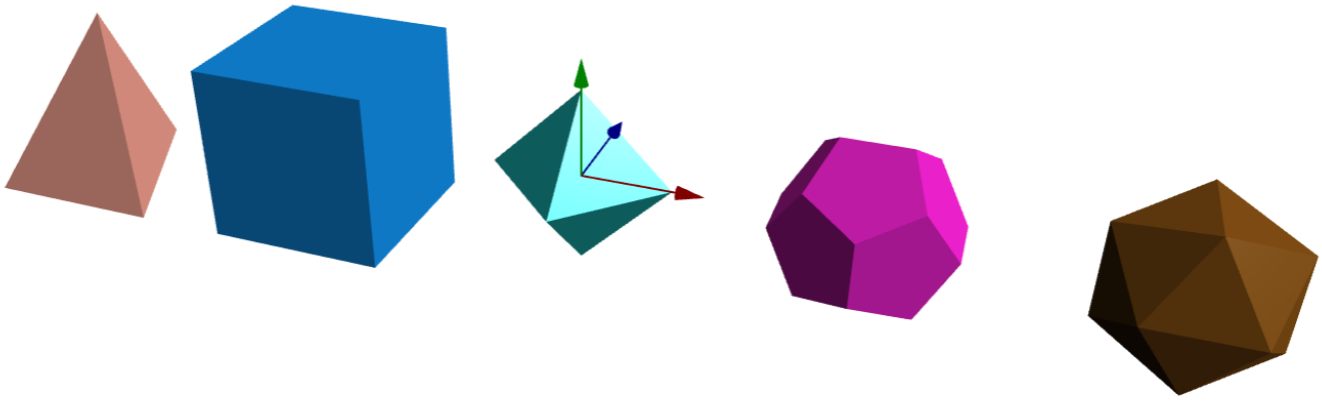
export interface ITileContentView<T> {
    source: Nullable<ITileSection>;
    target: Nullable<ITileSection>;
    data: Nullable<T>;
}
```

- Une action de zoomer se traduit par l'affectation temporaire à la tuile de la donnée du parent avec une section source définie par la position de la tuile courante et une section target nulle.
- Une action de dézoomer se traduit par l'affectation temporaire des tuiles enfants à la donnée de la tuile courante avec une définition des sections target, et des sections source nulles.
- Sans transition, source et target sont simplement nulles.

Annexes

Solides de Platon

Les solides de Platon son bien connus et leur représentation topologique est simple.



Le code ci-dessous génère ces topologies.

```
function CreateTetrahedron() {
    const data = new BABYLON.VertexData();
    const a = 1;
    const b = 1;
    const c = 1;
    data.positions = [0, c, 0, b, -c, -a, 0, -c, a, -b, -c, -a];
    data.indices = [0, 1, 2, 0, 2, 3, 0, 3, 1, 1, 3, 2];
    return data;
}

function CreateHexahedron() {
    const data = new BABYLON.VertexData();
    const i = 1;
    data.positions = [-i, -i, -i, -i, -i, i, i, -i, i, i, -i, -i, -i, i, -i, -i, i, i, i, i, i, i, -i];
    data.indices = [0, 2, 3, 0, 1, 2, 0, 5, 1, 0, 4, 5, 0, 7, 4, 0, 3, 7, 6, 1, 5, 6, 2, 1, 6, 3, 2, 6, 7, 3, 6, 4, 7, 6, 5, 4];
    return data;
}

function CreateOctahedron() {
    const data = new BABYLON.VertexData();
    const i = 1;
    data.positions = [0, 0, i, 0, 0, -i, i, 0, 0, -i, 0, 0, 0, i, 0, 0, -i, 0];
    data.indices = [4, 2, 0, 4, 1, 2, 4, 3, 1, 4, 0, 3, 5, 0, 2, 5, 2, 1, 5, 1, 3, 5, 3, 0];
    return data;
}

function CreateDodecahedron() {
    const data = new BABYLON.VertexData();
    const a = 1 / Math.sqrt(3);
    const b = Math.sqrt((3 - Math.sqrt(5)) / 6);
    const c = Math.sqrt((3 + Math.sqrt(5)) / 6);
```

```
    data.positions = [a, a, a, a, -a, a, -a, a, a, -a, -a, a, a, a, -a, a, -a, -a, -a, a, -a, -a, -a, -a, c, 0, b,
        c, 0, -b, -c, 0, b, -c, 0, -b, 0, b, c, 0, -b, c, 0, b, -c, 0, -b, -c, b, c, 0, -b, c, 0, b, -c, 0, -b, -c, 0];
    data.indices = [0, 9, 8, 0, 13, 12, 0, 17, 16, 8, 18, 1, 12, 10, 2, 16, 14, 4,
        9, 15, 5, 6, 10, 11, 3, 18, 19, 7, 5, 15, 7, 6, 11, 7, 3, 19, 0, 4, 9, 0, 1, 13,
        0, 2, 17, 8, 5, 18, 12, 3, 10, 16, 6, 14, 9, 14, 15, 6, 2, 10, 3, 1, 18, 7, 18, 5,
        7, 14, 6, 7, 10, 3, 0, 16, 4, 0, 8, 1, 0, 12, 2, 9, 5, 8, 12, 13, 3, 16, 17, 6,
        9, 4, 14, 6, 17, 2, 3, 13, 1, 7, 19, 18, 7, 15, 14, 7, 11, 10];
    return data;
}

function CreateIcosahedron() {
    const data = new BABYLON.VertexData();
    const a = (1 + Math.sqrt(5)) / 2;
    const b = Math.sqrt(1 + a * a);
    const t = a / b;
    const i = 1 / b;
    data.positions = [i, 0, t, i, 0, -t, -i, 0, t, -i, 0, -t, 0, t, i, 0, -t, i, 0, t, -i, 0, -t, -i, t, i, 0, -t, i, 0, t, -i, 0, -t, -i, 0];
    data.indices = [4, 8, 0, 10, 5, 0, 9, 4, 2, 5, 11, 2, 8, 6, 1, 7, 10, 1, 6, 9, 3, 11, 7, 3, 8, 10, 0, 10, 8, 1, 11, 9, 2, 3, 9,
11, 0, 2, 4, 2, 0, 5, 3, 1, 6, 1, 3, 7, 4, 6, 8, 6, 4, 9, 7, 5, 10, 5, 7, 11];
    return data;
}
```

Vous pouvez retrouver le code source complet en https://www.babylonjs-playground.com/#CX7LNY#6

Les coordonnées barycentriques

Tout d'abord, il est important de comprendre ce qu'est un barycentre.

Un barycentre est un point situé à l'intérieur d'un système constitué de plusieurs corps en mouvement. Ce point est le centre de masse du système, c'est-à-dire le point où la somme des masses de tous les corps du système est concentrée.

Dans le contexte de l'espace et des planètes, le barycentre est souvent utilisé pour décrire le mouvement de deux ou plusieurs corps en orbite l'un autour de l'autre. Par exemple, dans un système binaire de deux étoiles, chacune des étoiles orbite autour du barycentre du système, qui se situe quelque part entre les deux étoiles.

De manière plus générale, dans un système planétaire, le barycentre est le point autour duquel toutes les planètes du système orbitent. Ce point n'est pas nécessairement situé au centre de l'étoile autour de laquelle les planètes tournent. En effet, la masse des planètes peut avoir un impact sur la position du barycentre. Par exemple, la présence de Jupiter dans notre système solaire a un effet significatif sur la position du barycentre du système, qui est légèrement excentré par rapport au centre du Soleil.

En géométrie, un barycentre est un point situé à l'intérieur ou à l'extérieur d'une figure géométrique qui représente le centre de gravité de cette figure.

Maintenant, revenons aux coordonnées barycentriques. Les coordonnées barycentriques sont une méthode pour décrire la position d'un point dans un triangle en termes de trois poids relatifs. Ces poids relatifs représentent la proportion de chaque sommet du triangle nécessaire pour atteindre ce point.

Plus précisément, considérons un triangle ABC. Tout point P dans le plan du triangle peut être exprimé comme une combinaison linéaire des vecteurs reliant les sommets du triangle et le point P lui-même. Cela signifie que P peut être écrit comme:

$$P = \alpha A + \beta B + \gamma C$$

où α, β et γ sont des poids relatifs. Ces poids relatifs sont appelés les coordonnées barycentriques de P par rapport au triangle ABC.

Il est important de noter que les coordonnées barycentriques ont la propriété intéressante que la somme de leurs valeurs est toujours égale à 1. Cela signifie que la somme des poids relatifs doit être égale à 1 pour chaque point P.

Wireframe

Nous allons utiliser ces propriétés afin de créer un effet de type filaire en surimposition d'un shader de pixel (nous aurions pu utiliser le dessin de ligne en WebGL mais ce n'est pas très intéressant pour notre propos).

La première étape consiste à ajouter des coordonnées Barycentriques sous forme d'attribut à chaque sommet du triangle. Dans le cas d'Open GL cela peut être fait dans un shader de type Géométrie. En Web GL, les shaders de géométrie n'étant pas disponibles, il nous faut générer ces attributs via le CPU et les envoyer au shader.

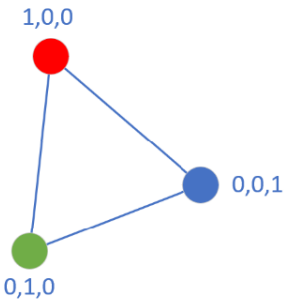


Figure 18 : poids barycentrique pour un triangle unique

Notez au passage que comme vu précédemment, la somme des coordonnées barycentriques est toujours égale à 1. Nous pourrions donc nous contenter de ne passer que deux coordonnées, en déduisant la troisième en soustrayant les deux autres de 1. Cela signifie que nous aurions à interpoler un nombre de moins, soit (x, y, 1-x-y).

Vertex Shader :

```
…
attribute vec2 barys;
varying vec2 vBarys;

 void main(void) {
   …
   vBarys = barys;
  …
}`;
```

Fragment Shader :

```
uniform float edgeThickness;
uniform vec4 edgeColor ;

varying vec2 vBarys;

float edgeFactor(){
    vec3 b = vec3( vBarys.xy, 1.0 - vBarys.x - vBarys.y);
    vec3 d = fwidth(b);
    vec3 a3 = smoothstep(vec3(0.0), d * edgeThickness, b);
    return min(min(a3.x, a3.y), a3.z);
}

void main(void) {
    gl_FragColor = …
    gl_FragColor = mix(edgeColor, gl_FragColor, edgeFactor());
```

```
}`
```

Pour créer l'effet wireframe, nous devons savoir à quel point nous somme proche d'une arête. Nous réalisons cela en prenant la valeur minimum des coordonnées barycentriques.

```
min(min(a3.x, a3.y), a3.z)
```

Nous allons également utiliser la fonction ***smoothstep*** et une variable d'épaisseur ***edgeThickness*** afin d'obtenir un effet visuel cohérent quel que soit les dimensions du triangle.

La fonction `smoothstep(float edge0, float edge1, floatx)` en OpenGL est une fonction de lissage qui est couramment utilisée pour créer des transitions fluides entre deux valeurs. Cette fonction prend trois arguments: un minimum, un maximum et une valeur. Elle renvoie une interpolation douce de la valeur, basée sur la fonction polynomiale de Hermite. Plus précisément, elle calcule d'abord une interpolation linéaire de la valeur "x" entre les limites "edge0" et "edge1". Elle évalue ensuite cette interpolation linéaire à l'aide de la fonction polynomiale de Hermite, qui crée une transition douce et sans à-coups entre les valeurs.

```
    vec3 a3 = smoothstep(vec3(0.0), d * edgeThickness, b);
    return min(min(a3.x, a3.y), a3.z);
```

notez que nous avons introduit un paramètre d'échelle qui permet de moduler l'épaisseur des trait obtenus.

Si l'on se contente de cette approche, nous allons apercevoir des discontinuité au niveau des sommets. Cela est du au changement brusque d'arête dans cette région du triangle. Cela introduit une discontinuité dans la fonction d'interpolation. Pour fixer cela, nus pouvons utiliser les dérivées individuelles des coordonnées barycentriques a la place des coordonnées.

La fonction `fwidth(vec3 v)` est une fonction mathématique en OpenGL qui permet de calculer une approximation de la variation de la valeur d'une fonction en deux dimensions de manière précise. Elle est souvent utilisée dans les shaders OpenGL pour réaliser des effets visuels comme des bords flous ou des ombres douces.

```
float edgeFactor(){
    vec3 b = vec3( vBarys.xy, 1.0 - vBarys.x - vBarys.y);
    vec3 d = fwidth(b);
    vec3 a3 = smoothstep(vec3(0.0), d * edgeThickness, b);
    return min(min(a3.x, a3.y), a3.z);
}
```

Nous obtenons alors notre fonction qui calcule le facteur à appliquer au pixel, afin de mixer éventuellement la couleur de fond, avec celle de l'arête.

Déterminer les coordonnées des sommets dans l'espace barycentrique.

Il est important de noter que le fonctionnement de ce shader wireframe est intimement lié aux poids barycentriques des sommets tel qu'elles sont passée au shader. En l'occurrence dans l'exemple précédent (1,0,0), (0,1,0), (0,0,1).

Considérant le triangle A,B,C, on peut remarquer que l'activation de l'arête A,C set fait par la mise à zéro des paramètres correspondant sur A et C et à 1 sur le sommet opposé B. Le corolaire et que si toute les valeurs du paramètres concernée sont à 1, l'arête n'est pas dessinée.

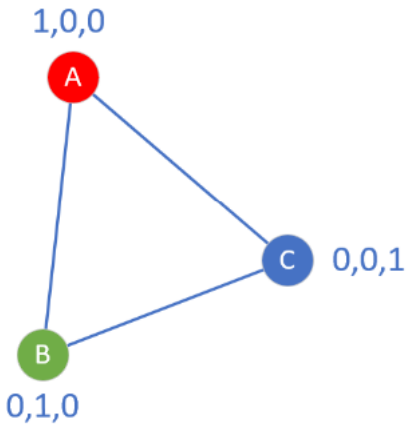


Figure 19 : dessin de toutes les arêtes

Considérant le triangle A,B,C, alors les paramètres (1,1,0), (0,1,0), (0,1,1) dessinent les arêtes A,B et B,C uniquement.

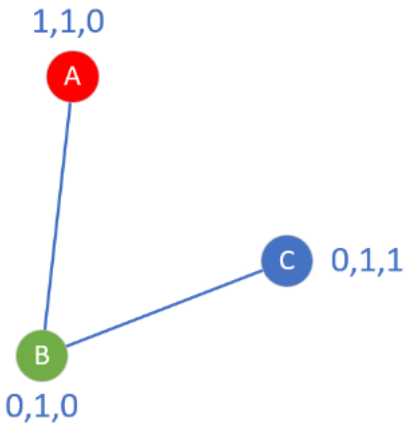


Figure 20: dessin partiel des arêtes

Cela fonctionne correctement si chaque triangle possède des sommets uniques non partagé avec d'autres triangles. Dans ce dernier cas, il faut revoir le poids des sommets en fonction de la topologie du mesh.

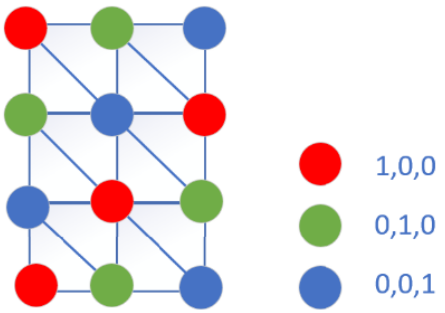


Figure 21 : poids barycentriques pour une topologie avec sommets partagés

Si l'on veut supprimer les diagonales alors cela devient plus complexe pour des sommets partagés et un changement de topologie devient souhaitable.

Calcul des valeurs barycentrique.

La génération des poids barycentrique peut être effectuée du coté CPU. Elle peut également être calculée à la volée du coté GPU. Pour ce faire il faudra identifier la fonction qui retourne le vecteur barycentrique en fonction de l'index du sommet et/ou de l'index de la face.

l est possible de connaître l'index de la face dans un shader OpenGL. Pour cela, vous pouvez utiliser la variable prédéfinie *gl_PrimitiveID* qui contient l'index de la primitive en cours de traitement par le shader.

Il convient de noter que la variable *gl_PrimitiveID* ne peut être utilisée que dans les shaders de géométrie et de fragment, car elle est calculée après l'étape de sommets. Dans les shaders de sommets, vous devez plutôt utiliser l'index de sommet *gl_VertexID* pour identifier la position de chaque sommet dans le maillage.

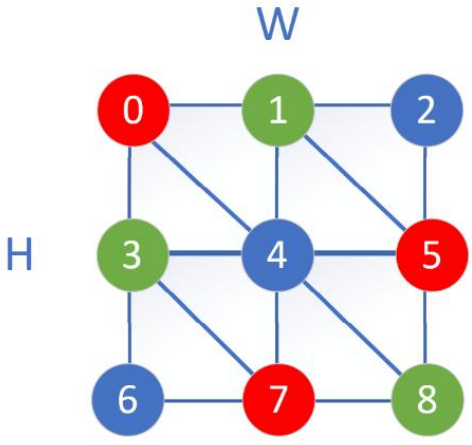


Figure 22: index des sommets sur une topologie classique

La table suivante nous donne les vecteurs barycentriques affectés aux sommets pour une topologie classique.

index	Line = floor(index/w)	Index table de poids (line+index)%3	poids barycentriques
0	0	0	1,0,0
1	0	1	0,1,0
2	0	2	0,0,1
3	1	1	0,1,0
4	1	2	0,0,1
5	1	0	1,0,0
6	2	2	0,0,1
7	2	0	1,0,0
8	2	1	0,1,0

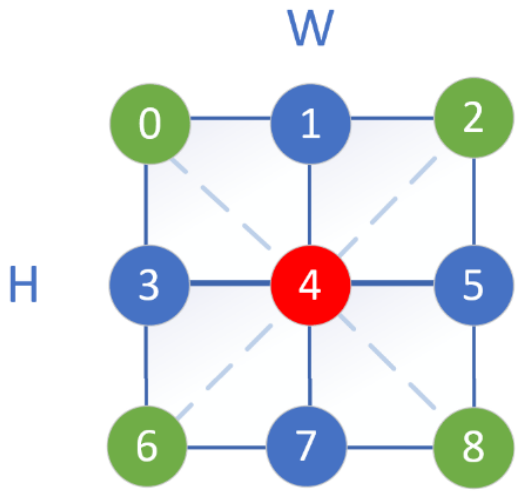


Figure 23: index des sommets sur une topologie en étoile.

La topologie en étoile est intéressante car elle permet d'effacer l'arête en diagonale par la mise à 1 de la troisième valeur des poids barycentriques pour l'ensemble des sommets. Et d'obtenir ainsi un wireframe « carré ». Sachant que cette troisième valeur est calculée, il est simple de lui affecter la valeur 1 systématiquement.

La table suivante nous donne les vecteurs barycentriques affectés aux sommets pour une topologie en étoile avec une table d'indexation intermédiaire [1,2,2,0] car la fonction de calcul n'est pas directe et implique un test que nous voulons éviter en GLSL.

index	i = index%2	Line = floor(index/w)	J = line*2 + (index+line)%2	Index table de poids	poids barycentriques
0	0	0	0	1	0,1,0
1	1	0	1	2	0,0,1
2	0	0	0	1	0,1,0
3	1	1	2	2	0,0,1
4	0	1	3	0	1,0,0
5	1	1	2	2	0,0,1
6	0	2	0	1	0,1,0
7	1	2	1	2	0,0,1
8	0	2	0	1	0,1,0

JS

En javascript, les fonctions sont

```
function createBarycentricDataSimple(w, h ){
    const barys = [] ;
    const barysPattern = [1,0, 0,1, 0,0];
    for(let i=0; i!= w*h;i++){
      const line = Math.floor(i/w);
      const index = (line+i)%3;
      barys.push(barysPattern[index*2],barysPattern[index*2 + 1]);
   }
  return barys;
}
```

```
function createBarycentricDataStar(w, h ){
    const barys = [] ;
    const barysPattern = [1,0, 0,1, 0,0];
    // it's better to introduce here an indirection to save cycles.
    const indexesPattern = [1,2,2,0];
    for(let i=0; i!= w*h;i++){
      const line = Math.floor(i/w)%2;
      const j = line*2 + (i+line)%2
      const index = indexesPattern[j];
      barys.push(index);
      //barys.push(barysPattern[index*2],barysPattern[index*2 + 1]);
```

```
  }
 return barys;
}
```

GLSL

Dans le shader de vertex, nous introduisons un ou plusieurs tableaux de constantes en fonction de la topolgie

```
  const vec2 BARYCENTRIC_VALUES[3] = vec2[](
    vec2(1.0, 0.0),
    vec2(0.0, 1.0),
    vec2(0.0, 0.0)
  );

  const int INDEX_PATTERN [4] = int[](1,2,2,0);

  uniform vec2 dim;
…

  vec3 barycentricWeight(){

    float vertexIndex = float(gl_VertexID);

    float line = floor(vertexIndex / dim.x) ; // x is width, y is height
    float col = mod(vertexIndex,dim.x);

    // calculate the index into indices table :
    // lineoffset * 2.0 is the offset into the table
    // vertexIndex % dim.x is the column
    // column % 2  is an alternate between 0 and 1 to add to the offset
    float lineoffset = mod(line,2.0) ; //  alternat 0 and 1 for even and odd line number
    float offset = lineoffset * 2.0;
    int j =  int( offset +  mod(col,2.0) );
    int i = INDEX_PATTERN[ j ];
    // z value is the alternate edge weight begining a 0 or 1 at each row.
    // this enable the partial draw of wireframe.
    return vec3(BARYCENTRIC_VALUES[i], mod(col + lineoffset, 2.0) ) ;
  } …
  void main(void) {
…
        vec3 b = barycentricWeight();
        vBarys = b.xy;
        vEdgeWeight = b.z;
  }`;
```

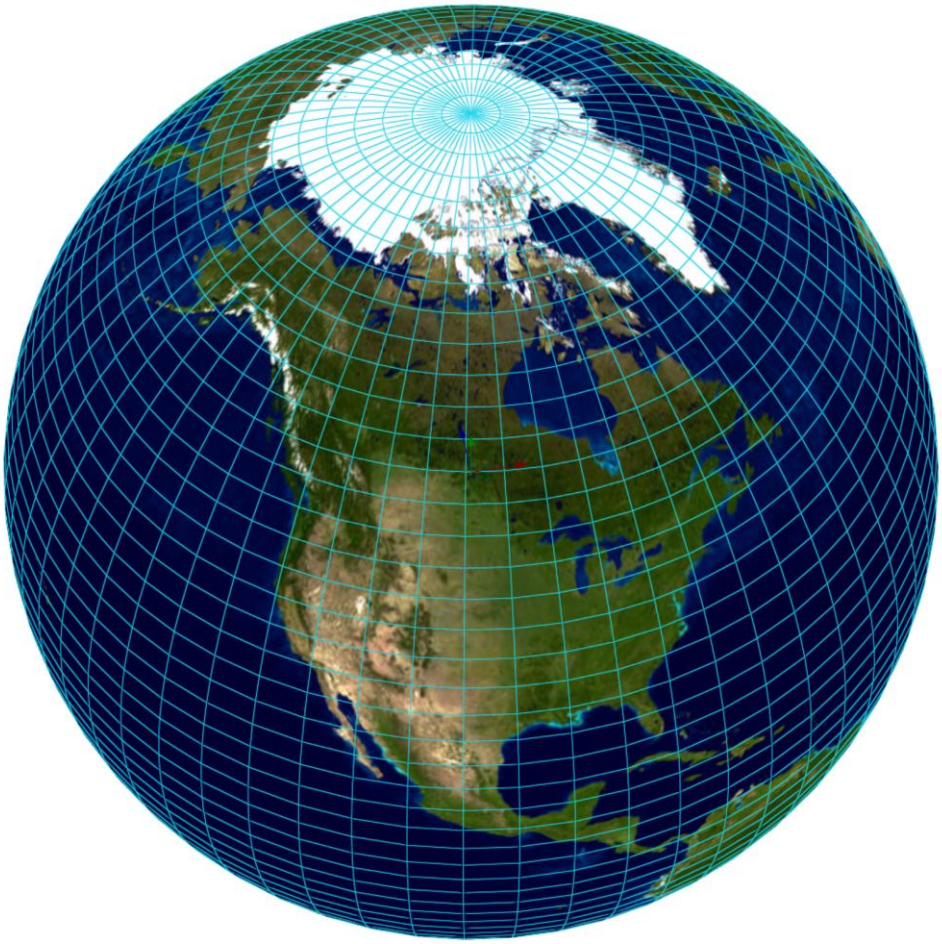


Figure 24: le shader avec wireframe e(suppression des diagonales

Vous pouvez retrouver le code de cet exemple sur https://www.babylonjs-playground.com/#99BSJR#33

En introduisant une poids supplémentaire alternatif 0 et 1 sur les colonnes de sommets et une dimension de segment dans l'intervalle [0,1] il est possible de réduire la longueur des segments filaires. Cet effet est très utile sur la visualisation des informations des modèles numériques de terrain car il permet de souligner la topologie tout en laissant place à l'information.

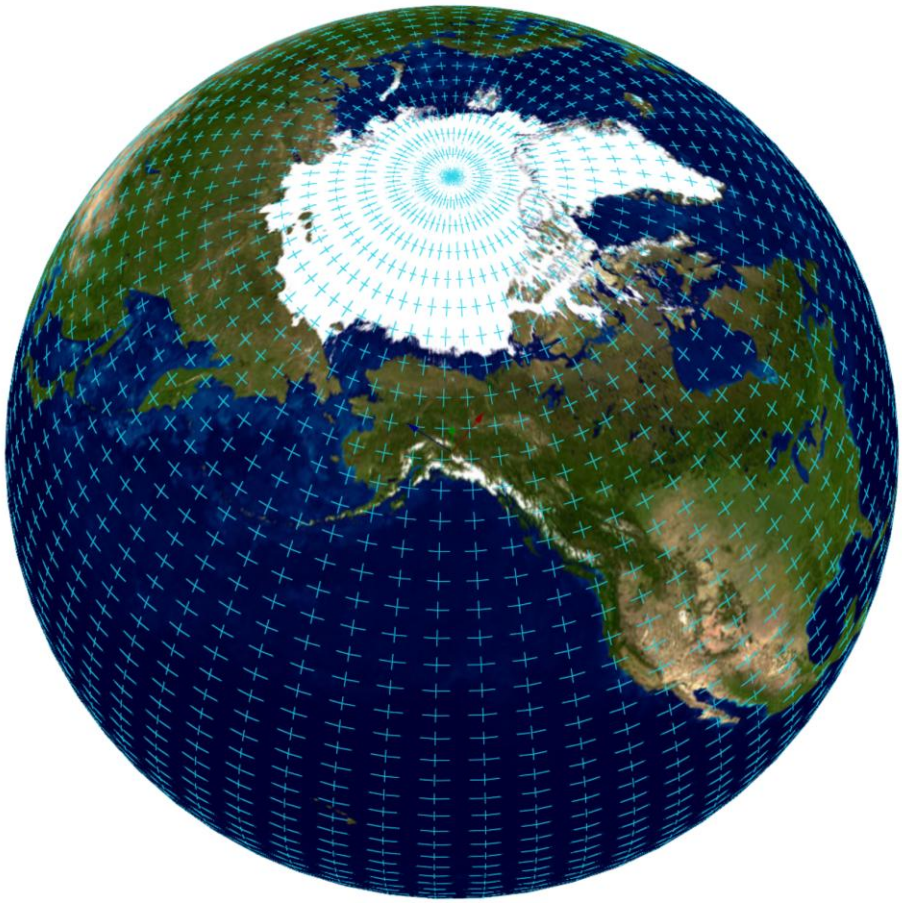


Figure 25: des segments filaire plus courts

Vous pouvez retrouver le code de cet exemple sur https://www.babylonjs-playground.com/#99BSJR#30

Les cartes numériques

Lorsqu'il s'agit d'une carte numérique, plusieurs concepts sont importants à comprendre : l'échelle, la résolution de l'écran et la taille de l'écran. Voici les rapports entre ces éléments :

- Échelle : L'échelle d'une carte numérique représente la relation entre les distances sur la carte et les distances réelles sur le terrain. Par exemple, une échelle de 1:10 000 signifie que 1 unité sur la carte représente 10 000 unités dans le monde réel. L'échelle peut être exprimée de différentes manières, telles que des fractions, des ratios ou des équivalences. Elle permet de mesurer la distance réelle entre des points spécifiques sur la carte.
- Résolution de l'écran : La résolution de l'écran fait référence à la densité de pixels affichés sur l'écran. Elle est généralement mesurée en pixels par pouce (PPI) ou en pixels par centimètre (PPCM). Une résolution plus élevée signifie qu'il y a plus de pixels par unité de mesure, ce qui se traduit par une meilleure qualité d'image et une plus grande précision d'affichage des détails. La résolution de l'écran peut influencer la clarté des éléments affichés sur la carte, tels que les lignes, les étiquettes ou les symboles.
- Taille de l'écran : La taille de l'écran fait référence aux dimensions physiques de l'écran, mesurées généralement en pouces (ou en centimètres). La taille de l'écran est importante car elle détermine la quantité d'informations qui peuvent être affichées simultanément. Une plus grande taille d'écran permet d'afficher une plus grande portion de la carte, ce qui facilite la navigation et la visualisation des détails. Cependant, la taille de l'écran ne doit pas être confondue avec la résolution, car deux écrans de même taille peuvent avoir des résolutions différentes.

Dans la projection de Mercator, l'échelle d'une carte dépend de la latitude. La projection de Mercator est une projection cylindrique conforme, ce qui signifie qu'elle préserve les angles locaux, mais entraîne des distorsions d'échelle aux latitudes élevées. Dans cette projection, l'échelle est constante le long de l'équateur. Cela signifie que les distances sur la carte sont proportionnelles aux distances réelles à l'équateur. Cependant, à mesure que l'on se déplace vers les latitudes plus élevées, la projection de Mercator provoque une distorsion d'échelle. Les distances en latitude deviennent de plus en plus exagérées à mesure que l'on se rapproche des pôles.

Pour le globe terrestre, le tableau suivant donne un exemple de ces échelles rapportées au niveaux de détail de la carte et à la latitude moyenne considérée. Et ce, une résolution d'écrans de 96 dpi.

Niveau de détail	Echelle à la latitude 0°	Echelle à la latitude 45°	Echelle à la latitude 80°
0	1 / 591658710,9	1 / 418365887	1 / 102740457

1	1 / 295829355,5	1 / 209182943	1 / 51370228,5
2	1 / 147914677,7	1 / 104591472	1 / 25685114,2
3	1 / 73957338,86	1 / 52295735,8	1 / 12842557,1
4	1 / 36978669,43	1 / 26147867,9	1 / 6421278,56
5	1 / 18489334,72	1 / 13073934	1 / 3210639,28
6	1 / 9244667,358	1 / 6536966,98	1 / 1605319,64
7	1 / 4622333,679	1 / 3268483,49	1 / 802659,82
8	1 / 2311166,839	1 / 1634241,74	1 / 401329,91
9	1 / 1155583,42	1 / 817120,872	1 / 200664,955
10	1 / 577791,7099	1 / 408560,436	1 / 100332,477
11	1 / 288895,8549	1 / 204280,218	1 / 50166,2387
12	1 / 144447,9275	1 / 102140,109	1 / 25083,1194
13	1 / 72223,96373	1 / 51070,0545	1 / 12541,5597
14	1 / 36111,98187	1 / 25535,0273	1 / 6270,77984
15	1 / 18055,99093	1 / 12767,5136	1 / 3135,38992
16	1 / 9027,995467	1 / 6383,75682	1 / 1567,69496
17	1 / 4513,997733	1 / 3191,87841	1 / 783,84748
18	1 / 2256,998867	1 / 1595,9392	1 / 391,92374
19	1 / 1128,499433	1 / 797,969602	1 / 195,96187
20	1 / 564,2497167	1 / 398,984801	1 / 97,980935
21	1 / 282,1248583	1 / 199,4924	1 / 48,9904675
22	1 / 141,0624292	1 / 99,7462002	1 / 24,4952338
23	1 / 70,53121458	1 / 49,8731001	1 / 12,2476169

Par exemple

Plus précisément, l'échelle de la projection de Mercator varie en fonction de la latitude selon la formule suivante :

$$\text{Échelle} = 1 / \cos(\text{latitude})$$

Cette formule indique que l'échelle diminue à mesure que la latitude augmente. Ainsi, à des latitudes élevées, l'échelle est plus petite, ce qui signifie que les distances sur la carte sont exagérées par rapport aux distances réelles. Il est important de prendre en compte cette distorsion d'échelle lors de l'utilisation de la projection de Mercator, en particulier lorsque vous travaillez avec des régions situées à des latitudes élevées.

Le corolaire est que si l'on dispose d'une carte interactive ou l'on peux naviguer le long des latitudes, l'échelle ne sera pas fixe.

Adaptation aux hologrammes

«.. En tant que capitaine du vaisseau Babylon, je suis fier de vous présenter notre technologie de rendu de terrain holographique. Grâce à cette avancée majeure, nous sommes en mesure de recréer des environnements virtuels réalistes, où les membres de notre équipage pourront explorer des planètes et des mondes avec une précision sans précédent. Nos hologrammes affichent des reliefs détaillés, des montagnes majestueuses, des

vallées profondes et des plaines étendues, créant ainsi une immersion totale dans ces environnements. Nous intégrons des données géospatiales précises, telles que des cartes topographiques et des informations LIDAR quantique, pour garantir une fidélité absolue dans la représentation du terrain. L'expérience visuelle que nous offrons grâce à notre rendu de terrain holographique est vraiment captivante, et nous permettra de vivre des expériences incroyables lors de la découverte de nouveaux mondes sans quitter le vaisseau Babylon. Soyez prêts à explorer l'inconnu et à interagir avec des environnements virtuels d'une réalité saisissante grâce à notre technologie de rendu de terrain holographique de pointe.. »

Capitaine D. Kosh. Année stellaire 3046.

Dans ce contexte, les concepts d'échelle et de résolution dans les hologrammes jouent un rôle crucial pour offrir des expériences visuelles immersives et détaillées. Voici comment ces concepts pourraient être envisagés:

- Échelle holographique : Les hologrammes peuvent être conçus pour afficher des objets et des environnements virtuels à différentes échelles. Par exemple, un hologramme pourrait présenter une scène miniature d'une ville entière, où chaque bâtiment, rue et détail est représenté avec précision, créant une sensation d'échelle réduite. De même, un hologramme pourrait présenter une échelle réelle, permettant aux utilisateurs d'interagir avec des objets virtuels à taille humaine. L'échelle holographique est soigneusement ajustée pour offrir des expériences immersives et réalistes.
- Résolution holographique : Dans un avenir où les hologrammes sont courants, la résolution holographique a atteint de nouveaux sommets. Les hologrammes haute résolution peuvent reproduire des détails extrêmement fins, tels que des textures, des contours et des nuances de couleur. Les objets virtuels affichés semblent réels et se confondent presque avec l'environnement physique. Les hologrammes haute résolution permettent de visualiser des informations complexes, de lire du texte clairement et de percevoir des détails subtils, offrant ainsi des expériences visuelles saisissantes.

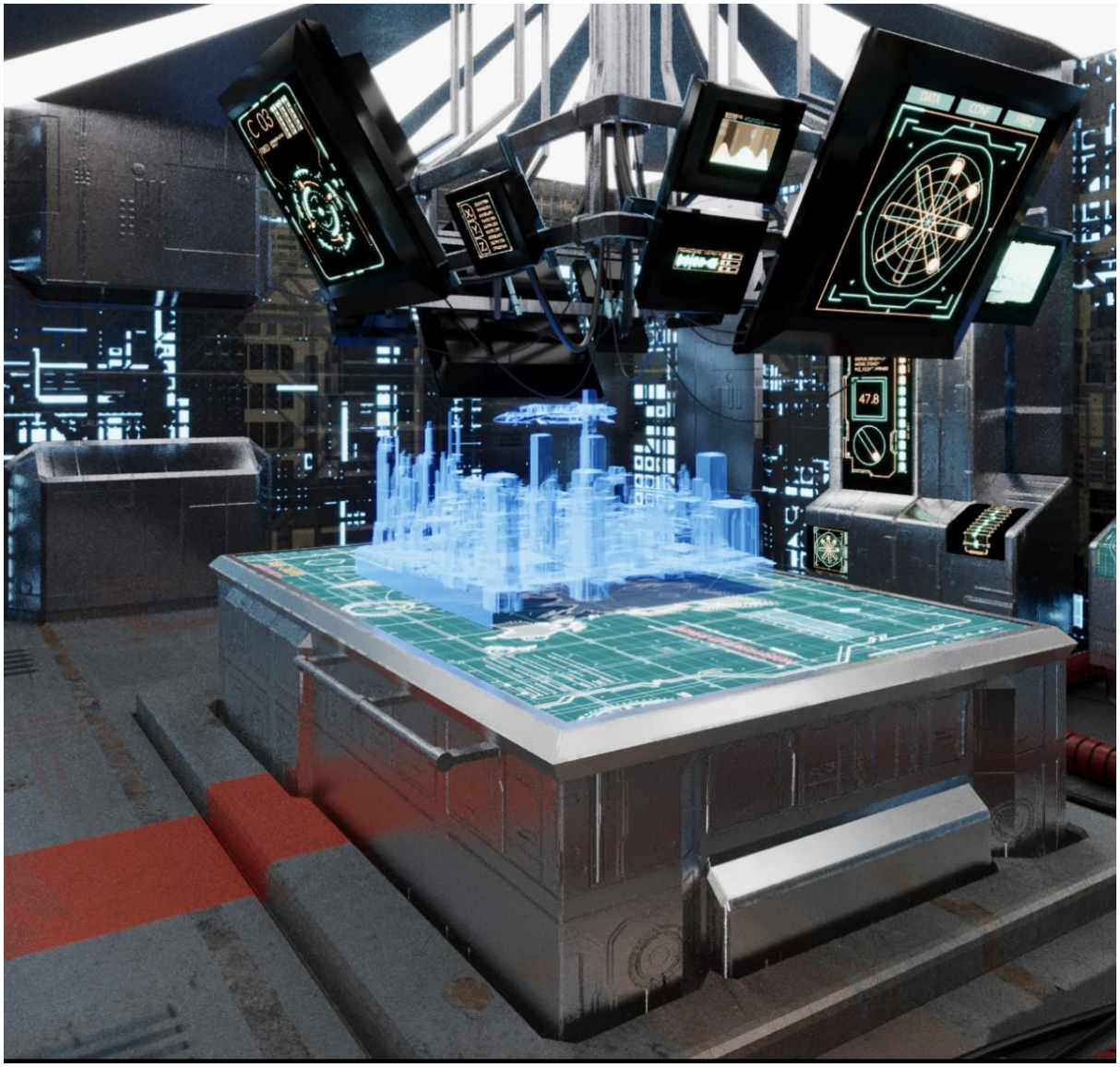


Figure 26: la table holographique du Babylon, sur la passerelle technique.

define virtual Display

https://playground.babylonjs.com/#65ZA2L

https://www.babylonjs-playground.com/#Z027U1#9

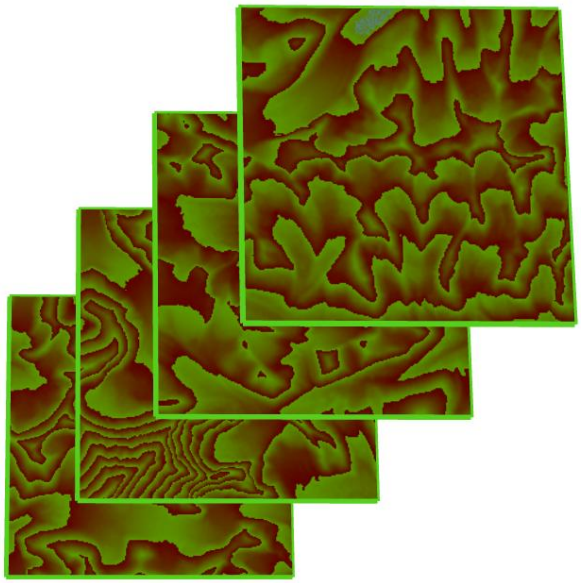
normal texture basic : https://www.babylonjs-playground.com/#X6RNX7

WEBGL

Les TEXTURE_2D_ARRAY en WebGL sont des types de textures apparues avec WebGL 2. Elles permettent de stocker et d'accéder à des données d'image dans un format particulier. Ils sont souvent utilisés lorsque vous avez besoin de gérer un ensemble de textures 2D liées d'une manière cohérente.

Dans le cas des atlases d'images, les TEXTURE_2D_ARRAY offrent un moyen efficace de stocker plusieurs images 2D dans une seule texture. Ce qui peut être avantageux en termes de performances et de gestion des ressources. Les coordonnées de texture utilisées pour accéder aux différentes images au sein de la texture peuvent être spécifiées en utilisant des coordonnées 2D classiques.

Les TEXTURE_2D_ARRAY peuvent également être utilisés pour représenter des images 3D, où chaque tranche de la texture représente une couche de l'image 3D. Par exemple, une texture 2D array de taille 256x256x4 pourrait être utilisée pour représenter une image 3D avec des dimensions de 256x256 pixels et 4 couches. Les coordonnées de texture utilisées dans ce cas peuvent être des coordonnées 3D, spécifiant à la fois la position horizontale et verticale sur la texture, ainsi que la couche dans la troisième dimension.



Sized internal formats are supported in WebGL 2.0 and *internalformat* is no longer required to be the same as *format*. Instead, the combination of *internalformat*, *format*, and *type* must be listed in the following table:

Internal Format	Format	Type

RGB	RGB	UNSIGNED_BYTE UNSIGNED_SHORT_5_6_5
RGBA	RGBA	UNSIGNED_BYTE, UNSIGNED_SHORT_4_4_4_4 UNSIGNED_SHORT_5_5_5_1
LUMINANCE_ALPHA	LUMINANCE_ALPHA	UNSIGNED_BYTE
LUMINANCE	LUMINANCE	UNSIGNED_BYTE
ALPHA	ALPHA	UNSIGNED_BYTE
R8	RED	UNSIGNED_BYTE
R16F	RED	HALF_FLOAT FLOAT
R32F	RED	FLOAT
R8UI	RED_INTEGER	UNSIGNED_BYTE
RG8	RG	UNSIGNED_BYTE
RG16F	RG	HALF_FLOAT FLOAT
RG32F	RG	FLOAT
RG8UI	RG_INTEGER	UNSIGNED_BYTE
RGB8	RGB	UNSIGNED_BYTE
SRGB8	RGB	UNSIGNED_BYTE
RGB565	RGB	UNSIGNED_BYTE UNSIGNED_SHORT_5_6_5
R11F_G11F_B10F	RGB	UNSIGNED_INT_10F_11F_11F_REV HALF_FLOAT FLOAT
RGB9_E5	RGB	HALF_FLOAT FLOAT
RGB16F	RGB	HALF_FLOAT FLOAT
RGB32F	RGB	FLOAT
RGB8UI	RGB_INTEGER	UNSIGNED_BYTE
RGBA8	RGBA	UNSIGNED_BYTE

SRGB8_ALPHA8	RGBA	UNSIGNED_BYTE
RGB5_A1	RGBA	UNSIGNED_BYTE UNSIGNED_SHORT_5_5_5_1
RGB10_A2	RGBA	UNSIGNED_INT_2_10_10_10_REV
RGBA4	RGBA	UNSIGNED_BYTE UNSIGNED_SHORT_4_4_4_4
RGBA16F	RGBA	HALF_FLOAT FLOAT
RGBA32F	RGBA	FLOAT
RGBA8UI	RGBA_INTEGER	UNSIGNED_BYTE

Les shaders Babylonjs

Bien sûr, pour créer un matériau personnalisé qui hérite de `PushMaterial` dans Babylon.js, vous devez surcharger certaines méthodes et propriétés pour définir le comportement de votre matériau personnalisé. Voici les méthodes et propriétés les plus couramment utilisées que vous pouvez surcharger :

- **isReady** : Cette méthode doit être surchargée pour indiquer si le matériau est prêt à être utilisé. Vous devez effectuer toutes les vérifications nécessaires ici, comme la vérification des textures chargées ou d'autres ressources.
- **getEffect** : Surchargez cette méthode pour définir les paramètres de rendu de votre matériau. Vous créez un nouvel effet (shader) ici et définissez les uniformes et attributs nécessaires.
- **bind**: Cette méthode est utilisée pour définir les valeurs des uniformes spécifiques au matériau avant le rendu. Vous devez surcharger cette méthode pour affecter les valeurs de vos propriétés personnalisées aux uniformes du shader.
- **unbind**: Cette méthode est utilisée pour réinitialiser les valeurs des uniformes après le rendu. Si vous avez modifié des uniformes dans la méthode `bind`, vous devez réinitialiser ces modifications ici.
- **dispose** : Vous devez surcharger cette méthode pour libérer toutes les ressources associées au matériau, telles que les textures ou les shaders. Cela permet de nettoyer correctement lorsque le matériau n'est plus nécessaire.
- **clone**: Si vous souhaitez prendre en charge la création de copies du matériau avec des propriétés spécifiques, surchargez cette méthode pour créer un clone du matériau avec les paramètres appropriés.
- **addUniformsToEffect** (facultatif) : Cette méthode est utilisée pour ajouter des uniformes supplémentaires à l'effet du matériau. Vous pouvez la surcharger si vous avez besoin d'uniformes personnalisés.

Voici un exemple simple de la façon dont vous pouvez surcharger ces méthodes dans votre classe de matériau personnalisé :

```
class MyCustomMaterial extends BABYLON.PushMaterial {
  constructor(name, scene) {
    super(name, scene);
  }

  // Surcharge de la méthode isReady
  isReady(mesh, useInstances) {
    // Effectuez vos vérifications de préparation ici
    // Renvoyez true si le matériau est prêt à être utilisé
```

```
  }

  // Surcharge de la méthode getEffect
  getEffect() {
    // Créez et retournez votre effet de rendu personnalisé ici
  }

  // Surcharge de la méthode bind
  bind(world, mesh) {
    // Affectez les valeurs des uniformes personnalisés ici
  }

  // Surcharge de la méthode unbind
  unbind() {
    // Réinitialisez les valeurs des uniformes personnalisés ici
  }

  // Surcharge de la méthode dispose
  dispose(forceDisposeEffect = false) {
    // Libérez les ressources ici
    super.dispose(forceDisposeEffect);
  }

  // Surcharge de la méthode clone
  clone(name) {
    // Créez et retournez une copie du matériau avec des propriétés spécifiques
  }

  // pour la serialization.
  getClassName() {
    return "MyCustomMaterial"
  }
}
```